

Från XLPM till agil IT-utveckling: arkitektens nya arbetssätt i statlig myndighet

Erland Lindmark

2026-05-12

Innehåll

Inledning	1
Kapitel 1: När kartan inte längre ritas färdigt i förväg	2
Kapitel 2: Arkitektens ansvar förändras — men försvinner inte	9
Kapitel 3: Från tidig säkerhet till löpande beslutsförmåga	16
Kapitel 4: När behovsbilden är osäker	22
Kapitel 5: När kravarbete blir löpande snarare än avslutat	33
Kapitel 6: När teamet vill börja bygga innan arkitekturen är klar	41
Kapitel 7: När design inte längre är en separat fas	52
Kapitel 8: När beroenden bromsar flödet	60
Kapitel 9: Dokumentation som stöd för lärande och ansvar	69
Kapitel 10: Säkerhet, regelefterlevnad och arkitektur i kortare cykler	78
Kapitel 11: Arkitekturforum, styrning och beslut utan flaskhalsar	87
Kapitel 12: Den agila arkitektens praktiska verktygslåda	98

Inledning

Den här boken handlar om vad som händer med arkitektrollen när IT-utveckling i en statlig myndighet rör sig från en mer XLPM- och fasbaserad utvecklingslogik till ett mer agilt, iterativt och teamnära arbetssätt.

Bokens utgångspunkt är inte att tidigare arbetssätt är fel. Fasorienterade modeller kan ge tydlighet, ansvar, struktur och beslutsstöd, särskilt i miljöer där regelefterlevnad, spårbarhet och långsiktig förvaltning är viktiga. Men när utvecklingen ska ske i kortare cykler, med mer återkoppling och större inslag av lärande under arbetets gång, behöver arkitekten ofta bidra på andra sätt än tidigare.

Boken riktar sig främst till lösningsarkitekter och IT-arkitekter i statlig myndighet. Den är skriven för dig som är van att vara med tidigt i behovsanalys, kravarbete, arkitektur och design av IT-stöd, och som nu behöver hitta ett fungerande förhållningssätt i mer agila utvecklingsflöden.

Fokus ligger på praktiska situationer snarare än på ett visst agilt ramverk. Du kommer därför inte att möta en manual för Scrum, SAFe eller Kanban. I stället följer boken återkommande dilemman som arkitekter ofta hamnar i: osäkra behov, löpande kravarbete, team som vill börja bygga innan allt är utrett, kontinuerlig design, beroenden, dokumentation, säkerhet, regelefterlevnad och beslutsforum.

Genom boken återkommer den fiktiva Myndigheten för samhällstjänster och lösningsarkitekten Sara. Sara är erfaren, strukturerad och van att skapa riktning tidigt. Hennes utmaning är inte att sluta vara arkitekt, utan att använda sin erfarenhet på ett nytt sätt: mer närvarande, mer dialogbaserat, mer stegvis och mer inriktat på att hjälpa organisationen fatta bra arkitektur- och designbeslut över tid.

Bokens grundfråga är:

Hur kan arkitekten bidra med riktning, kvalitet och långsiktighet utan att bromsa lärande, flöde och teamens förmåga att leverera stegvis?

Varje kapitel utgår från en igenkännbar situation, beskriver ett invariant sätt att agera, visar varför det kan skapa friktion i agil utveckling och föreslår ett mer agilt förhållningssätt. Kapitlen avslutas med reflektionsfrågor som kan användas individuellt, i arkitekturgemenskaper eller i samtal mellan arkitekter, team och ledning.

Kapitel 1: När kartan inte längre ritas färdigt i förväg

Varför detta kapitel finns

Du som har arbetat länge som lösningsarkitekt eller IT-arkitekt känner sannolikt igen värdet av en tydlig karta. Innan utveckling startar vill man förstå behov, beroenden, målbild, risker, informationsflöden, systemlandskap och möjliga lösningsvägar. Det är inte konstigt. I en myndighetskontext kan otydlighet snabbt få konsekvenser: felaktiga beslut, bristande spårbarhet, svag förankring, säkerhetsrisker eller lösningar som blir svåra att förvalta.

Samtidigt förändras förutsättningarna när organisationen vill arbeta mer agilt. Då räcker det inte alltid att först beskriva hela lösningen och därefter lämna över till utveckling. Behov kan visa sig vara annorlunda än man trodde. Användare kan reagera oväntat. Tekniska beroenden kan visa sig enklare eller svårare än analysen antydde. Nya regler, prioriteringar eller verksamhetsinsikter kan förändra vad som är klokast att göra.

Det här kapitlet handlar om det första skiftet i boken: från att se arkitektur som en karta som behöver ritas färdigt i förväg, till att se arkitektur som ett sätt att skapa tillräcklig riktning för att kunna lära sig säkert och ansvarsfullt under arbetets gång.

Målet är inte att ställa XLPM eller fasorienterade arbetssätt mot agila arbetssätt som om det ena vore rätt och det andra fel. Poängen är snarare att de bygger på olika logik. När logiken förändras behöver även arkitektens sätt att skapa trygghet, riktning och kvalitet förändras.

Situationen: uppstarten som känns både bekant och ny

Myndigheten för samhällstjänster ska modernisera ett äldre handläggningssystem. Systemet används av handläggare varje dag och har under många år byggts ut steg för steg. Det finns integrationer till andra interna system, beroenden till externa myndigheter, historiska datamodeller, gamla behörighetslösningar och flera manuella rutiner runt systemet.

Sara, erfaren lösningsarkitekt, blir inbjuden till ett tidigt uppstartsmöte.

På mötet finns Erik, produktägare för den nya digitala tjänsten, Lina som representerar utvecklingsteamet och Amir från informationssäkerhet. Verksamheten vill snabbt komma igång med en första digital tjänst för medborgare. Tanken är att en

begränsad version ska testas med en mindre användargrupp för att ge återkoppling på flöde, språk, handläggarstöd och tekniska antaganden.

Sara hör flera saker samtidigt.

Erik säger:

“Vi behöver få ut något litet tidigt, annars kommer vi fortsätta diskutera lösningen i flera månader utan att veta vad som faktiskt fungerar.”

Lina säger:

“Teamet kan börja med ett smalt flöde, men vi behöver veta vilka arkitekturramar som gäller. Annars riskerar vi att bygga något som vi får göra om.”

Amir säger:

“Vi kan inte testa med riktiga personuppgifter utan att ha kontroll på informationsklassning, åtkomst, loggning och rättsliga förutsättningar.”

Sara känner igen situationen. Hennes invanda reaktion är att skapa struktur. Hon vill kartlägga nuläget, identifiera beroenden, beskriva målbild, analysera risker och ta fram ett lösningsförslag som kan förankras. Det är ett ansvarsfullt sätt att agera.

Men den här gången förväntar sig organisationen också något annat: att arbetet börjar innan allt är färdiganalyserat.

Frågan blir då inte om Sara ska rita en karta eller inte. Frågan är vilken typ av karta som behövs när terrängen delvis ska upptäckas genom vandringen.

Det invanda sättet att agera

I en fasorienterad utvecklingslogik är det naturligt att arkitekten kommer in tidigt och bidrar till att skapa en stabil grund för kommande beslut. Ofta finns en tydlig rörelse från behov till krav, från krav till arkitektur, från arkitektur till design och därefter till utveckling, test och införande.

I ett sådant sammanhang blir arkitektens arbete ofta kopplat till att:

- förstå och beskriva nuläge,
- analysera behov och begränsningar,
- identifiera beroenden,
- ta fram målarkitektur eller lösningsarkitektur,
- dokumentera viktiga vägval,
- skapa underlag för beslut,
- minska osäkerhet innan större investeringar görs,
- möjliggöra överlämning till genomförande.

Det finns goda skäl till detta. Stora IT-satsningar i myndigheter behöver ansvar, spårbarhet, budgetlogik, förankring, riskhantering och samordning. En arkitekt som skapar tydlighet tidigt kan förhindra dyra misstag senare.

Det invanda arbetssättet kan därför sammanfattas så här:

Ju mer vi förstår och beskriver i förväg, desto tryggare kan genomförandet bli.

Detta är inte en orimlig tanke. Den har ofta varit både praktisk och nödvändig. Problemet uppstår när samma tanke används i situationer där en stor del av förståelsen bara kan uppstå genom återkoppling, test, teknisk utforskning och stegvis utveckling.

Då kan det som var tänkt att skapa trygghet i stället skapa väntan.

Varför det kan skapa friktion i agil utveckling

Agil utveckling bygger på en annan grundlogik. I stället för att utgå från att rätt lösning kan beskrivas tillräckligt komplett i början, utgår man från att förståelsen växer fram. Det betyder inte att man ska börja utan riktning. Det betyder att riktningen behöver kombineras med möjlighet att justera.

Här introducerar vi tre begrepp som följer med genom boken.

Fasorienterad utvecklingslogik betyder i den här boken ett arbetssätt där behov, krav, design, utveckling och införande ofta hanteras som tydligt avgränsade faser. Det kan ge struktur, ansvar och tydliga beslutspunkter.

Agil utvecklingslogik betyder ett arbetssätt där utveckling sker stegvis, med återkoppling, prioritering och lärande under arbetets gång. Det kan ge snabbare insikt, bättre anpassning och möjlighet att minska risk genom mindre steg.

Lärandebaserad styrning betyder att styrningen inte bara utgår från planer och tidiga antaganden, utan också från vad organisationen lär sig genom faktisk utveckling, användning, test och återkoppling.

Friktion uppstår när dessa logiker blandas utan att man pratar om det.

Organisationen kan säga att den vill arbeta agilt, men ändå förvänta sig samma typ av tidigt beslutsunderlag som i en fasorienterad modell. Teamet kan vilja börja bygga iterativt, men sakna tydliga ramar för vilka beslut de får fatta. Arkitekturforum kan vilja kvalitetssäkra, men göra det på ett sätt som skapar väntetider. Verksamheten kan vilja ha snabb nytta, men ändå förvänta sig hög förutsägbarhet kring kostnad, tid och omfattning.

För arkitekten kan detta bli särskilt svårt. Sara förväntas både möjliggöra fart och säkra helhet. Hon ska både släppa fram lärande och skydda organisationen från ogenomtänkta vägval. Hon ska både vara närvarande i teamets vardag och bidra till långsiktiga beslut.

Det är här den gamla kartmetaforen börjar skava. Om kartan måste vara färdig innan resan börjar kommer arbetet att gå långsamt. Om ingen karta finns alls riskerar teamet att gå vilse. Det agila arkitekturarbetet behöver därför en annan sorts karta: en som visar riktning, risker och kända begränsningar, men som också markerar vilka delar av terrängen som ännu behöver utforskas.

Ett mer agilt förhållningssätt

Ett mer agilt förhållningssätt innebär inte att Sara ska avstå från arkitektur. Det innebär att hon behöver ställa andra frågor tidigare.

I stället för att börja med frågan:

“Hur ska hela lösningen se ut?”

kan hon börja med frågor som:

- Vad behöver vi veta för att kunna ta nästa ansvarsfulla steg?
- Vilka beslut är svåra att ändra senare?
- Vilka antaganden behöver testas innan vi låser lösningen?
- Vilka risker måste vara hanterade innan teamet börjar bygga?
- Vilka ramar behöver teamet för att kunna fatta bra lokala beslut?
- Vilken dokumentation behövs nu för att stödja beslut, samarbete och ansvar?
- Vad är tillräckligt tydligt för att börja lära?

Skillnaden är subtil men viktig.

I en fasorienterad logik blir arkitektens uppgift ofta att reducera osäkerhet innan utveckling startar. I en mer agil logik blir arkitektens uppgift att hjälpa organisationen att hantera osäkerhet medan utvecklingen pågår.

Det kräver fortfarande analys, struktur och omdöme. Men analysen behöver ofta ske i mindre steg. Strukturen behöver kunna utvecklas. Omdömet handlar inte bara om att välja rätt lösning, utan om att avgöra vad som behöver väljas nu och vad som bör hållas öppet.

För Sara kan det innebära att hon inte tar fram en fullständig målarkitektur som första leverans. Hon kan i stället ta fram en första arkitektonisk orientering:

- vilka delar av nuläget som är mest relevanta,
- vilka system och informationsflöden som påverkas,
- vilka kvalitetskrav som inte får tappas bort,
- vilka vägval som verkar svårreversibla,
- vilka antaganden som behöver prövas,
- vilka risker som behöver följas upp,
- vilka ramar teamet behöver för första inkrementet.

Ett *inkrement* betyder här ett avgränsat steg i utvecklingen som ger ny funktion, ny kunskap eller båda. Det behöver inte vara en komplett lösning. Det viktiga är att det för arbetet framåt på ett kontrollerat sätt.

Exempel: Myndigheten för samhällstjänster

Efter uppstartsmötet väljer Sara att inte börja med att skriva en omfattande lösningsarkitektur. Hon bokar i stället ett kort arbetsmöte med Erik, Lina och Amir. Syftet är att skilja mellan tre typer av frågor:

1. Det vi måste veta innan teamet börjar.
2. Det vi kan undersöka genom ett begränsat första steg.
3. Det vi ännu inte behöver besluta.

De börjar med den första kategorin.

Amir lyfter att informationsklassning och hantering av personuppgifter måste vara tillräckligt tydligt innan någon verklig användardata används. Lina säger att teamet kan bygga en första teknisk prototyp med testdata. Erik vill att prototypen ska spegla ett verkligt användarflöde, men accepterar att den första versionen inte behöver användas skarpt.

Sara sammanfattar:

“Då verkar det som att vi inte behöver besluta hela framtida lösningen nu. Men vi behöver vara överens om vilka data vi inte får använda, vilka integrationer vi inte ska koppla på än och vilka arkitekturfrågor prototypen ska hjälpa oss att besvara.”

De identifierar tre frågor som första inkrementet ska ge svar på:

- Går det att skapa ett medborgarflöde som handläggare faktiskt kan följa upp i sitt befintliga system?
- Vilka uppgifter behöver tjänsten läsa från det äldre handläggningssystemet?
- Vilka säkerhets- och loggningskrav påverkar lösningens grundstruktur?

Sara dokumenterar inte detta som en fullständig målarkitektur. Hon dokumenterar det som en första arkitektonisk orientering och en lista över öppna antaganden.

Hon ritat en enkel bild över berörda system, markerar vilka kopplingar som är verkliga och vilka som bara är tänkta, och skriver en kort beslutsnotering:

Första inkrementet ska använda testdata och inte kopplas till produktionsmiljö. Syftet är att undersöka användarflöde, informationsbehov och tekniska beroenden. Beslut om slutligt integrationsmönster skjuts upp tills dessa frågor har prövats.

Det är inte frånvaro av arkitektur. Det är arkitektur i en annan rytm.

Vanliga fallgropar

- Fallgrop: Att tolka agil utveckling som att arkitektur ska vänta.
 - Varför den uppstår: Organisationen reagerar mot långa förberedelsefaser och vill snabbt börja leverera.
 - Vad arkitekten kan göra i stället: Visa vilken minimal arkitektonisk orientering som behövs för att arbetet ska kunna börja säkert.
- Fallgrop: Att försöka skapa samma arkitekturdokument som tidigare, men snabbare.
 - Varför den uppstår: Arkitekten vill behålla kvalitet och ansvar, men får mindre tid.
 - Vad arkitekten kan göra i stället: Skilj mellan dokumentation som behövs nu, dokumentation som kan växa fram och dokumentation som inte längre fyller ett tydligt syfte.
- Fallgrop: Att låsa lösningen för att skapa trygghet.
 - Varför den uppstår: Osäkerhet upplevs som risk, särskilt i myndighetsmiljö.
 - Vad arkitekten kan göra i stället: Synliggör vilka beslut som verkligen är svåra att ändra och vilka som kan hållas öppna.
- Fallgrop: Att låta teamet börja utan ramar.
 - Varför den uppstår: Organisationen vill undvika tunga processer och överlämningar.
 - Vad arkitekten kan göra i stället: Formulera tydliga startvillkor, risker och principer utan att detaljstyra lösningen.
- Fallgrop: Att prata om “agilt” utan att prata om vilken styrlogik som faktiskt gäller.
 - Varför den uppstår: Begreppet agilt används ofta positivt, men otydligt.
 - Vad arkitekten kan göra i stället: Ställ frågor om beslut, mandat, återkoppling och vad som får ändras när ny kunskap uppstår.

Frågor att ställa i situationen

När du som arkitekt hamnar i en uppstart där organisationen vill börja agilt men samtidigt efterfrågar tydlig arkitektur, kan följande frågor hjälpa:

- Vad behöver vara sant för att vi ska kunna ta ett första ansvarsfullt steg?
- Vilka risker är oacceptabla att skjuta framför oss?
- Vilka antaganden är så osäkra att de bör testas tidigt?
- Vilka delar av lösningen är dyra eller svåra att ändra senare?
- Vilka beslut kan teamet fatta själv, och vilka behöver bredare förankring?
- Vilken dokumentation behövs för att skapa gemensam förståelse just nu?
- Vilken återkoppling behöver vi efter första steget?
- Vad skulle få oss att ändra riktning?

Dessa frågor hjälper dig att flytta samtalet från “är arkitekturen klar?” till “är vi tillräckligt orienterade för att kunna lära oss på ett kontrollerat sätt?”.

Reflektionsfrågor

1. När har du själv upplevt att organisationen velat arbeta agilt men samtidigt förväntat sig ett traditionellt beslutsunderlag?
2. I vilka situationer brukar du vilja rita en mer komplett karta innan andra börjar agera?
3. Vilka delar av din arkitekturpraktik skapar trygghet på ett sätt som fortfarande behövs?
4. Vilka delar riskerar att skapa väntan eller låsa lösningen för tidigt?
5. Vad skulle en “första arkitektonisk orientering” kunna innehålla i ditt sammanhang?
6. Vilka beslut behöver du oftare markera som öppna, snarare än försöka lösa direkt?
7. Hur kan du prata med produktägare, team och säkerhetsroller om osäkerhet utan att den uppfattas som brist på kontroll?
8. Vad skulle vara ett litet men ansvarsfullt första steg i ett aktuellt initiativ du känner till?

Snabb sammanfattning

- Fasorienterad utvecklingslogik och agil utvecklingslogik bygger på olika antaganden om när förståelse uppstår.
- XLPM och liknande arbetssätt behöver inte beskrivas negativt; de kan ge struktur, ansvar och kontroll.
- Friktion uppstår när organisationen vill ha agil utveckling men behåller förväntningar på helt färdiga tidiga underlag.
- Arkitektens uppgift i agil utveckling är inte att släppa riktning, utan att skapa tillräcklig riktning för att möjliggöra lärande.
- En användbar första arkitektonisk orientering kan vara mer värdefull än en omfattande målarkitektur som försöker lösa för mycket för tidigt.
- Den centrala frågan blir: Vad behöver vi veta och besluta nu för att kunna ta nästa ansvarsfulla steg?

Nästa steg

I nästa kapitel går vi vidare till arkitektens ansvar. Om kartan inte längre ritas färdigt i förväg kan det kännas som att arkitektrollen försvagas. Men det är inte bokens utgångspunkt.

Tvärtom: när utvecklingen blir mer iterativ behöver arkitektens ansvar ofta bli mer aktivt och närvarande. Frågan är inte om arkitekten fortfarande behövs, utan hur arkitekten kan bidra utan att bli en flaskhals.

Kapitel 2: Arkitektens ansvar förändras — men försvinner inte

Varför detta kapitel finns

När en organisation börjar arbeta mer agilt uppstår ofta en osäkerhet kring arkitektrollen. Om teamen ska bli mer självständiga, om krav förfinas löpande och om lösningen får växa fram stegvis — vad ska arkitekten då göra?

En vanlig feltolkning är att agil utveckling gör arkitektur mindre viktig. I praktiken är det ofta tvärtom. När beslut fattas oftare, när fler personer deltar i designarbetet och när lösningen utvecklas i kortare steg blir behovet av arkitektoniskt omdöme större, inte mindre. Skillnaden är att omdömet behöver användas på ett annat sätt.

I en mer fasorienterad logik har arkitekten ofta skapat värde genom att analysera, strukturera, beskriva och förankra en lösning innan utvecklingen tar fart. Det arbetet kan fortfarande behövas. Men i en agil logik räcker det inte att arkitekturen finns som ett underlag vid sidan av teamets vardag. Arkitekturen behöver leva i samtalen, besluten, prioriteringarna och designvalen under hela utvecklingen.

Det här kapitlet handlar därför om ett viktigt skifte: från arkitekten som huvudsaklig producent av färdiga lösningsbeskrivningar till arkitekten som möjliggörare av bra arkitektur- och designbeslut över tid.

Situationen: teamet vill fatta fler beslut själv

Myndigheten för samhällstjänster har startat arbetet med den första avgränsade medborgartjänsten. Teamet har fått i uppdrag att ta fram ett smalt flöde där en medborgare kan lämna in kompletterande uppgifter digitalt i stället för via blankett.

Sara deltar i ett planeringsmöte tillsammans med Erik, Lina och några utvecklare. Teamet vill komma vidare med tekniska vägval för formulärhantering, åtkomstkontroll, integration mot handläggningssystemet och hur informationen ska visas för handläggaren.

Lina säger:

“Vi behöver kunna fatta fler beslut i teamet. Annars kommer varje fråga att behöva gå via arkitekturspåret, och då tappar vi tempo.”

Sara förstår poängen. Samtidigt ser hon riskerna. Formulärlösningen kan påverka informationsmodell, loggning, arkivering, integrationer och framtida återanvändning. Behörighetsfrågorna kan påverka både säkerhet och användarupplevelse. In-

tegrationerna kan skapa beroenden till det äldre handläggningssystemet som blir svåra att ändra senare.

Hon märker att hon dras mot två ytterligheter.

Den ena är att säga:

“Teamet får inte gå vidare förrän vi har rätt ut helheten.”

Den andra är att säga:

“Teamet får bestämma själva, så tar vi eventuella arkitekturfrågor senare.”

Ingen av ytterligheterna känns bra. Den första riskerar att göra arkitekturen till en broms. Den andra riskerar att viktiga vägval sker utan helhetsperspektiv.

Det Sara behöver är inte mer kontroll eller mindre ansvar. Hon behöver ett annat sätt att utöva sitt ansvar.

Det invanda sättet att agera

För en erfaren lösningsarkitekt är det naturligt att vilja skapa sammanhang innan beslut fattas. När teamet vill gå vidare med lösningsdetaljer kan det invanda agerandet vara att samla in mer information, analysera fler beroenden, beskriva målbilden tydligare och säkerställa att beslut är förankrade.

Det är ett rimligt beteende. Det kommer ofta ur ansvarstagande, inte ur ovilja att förändras. Arkitekten vet att till synes små beslut kan få stora följder. En integrationslösning kan påverka förvaltning i många år. En datamodell kan begränsa framtida tjänster. En behörighetsmodell kan bli dyr att ändra. En genväg i ett första inkrement kan bli normalfallet.

I fasorienterade sammanhang har arkitektens ansvar ofta uttryckts genom tydliga leveranser:

- lösningsbeskrivningar,
- arkitekturbilder,
- designprinciper,
- vägvalsunderlag,
- granskningskommentarer,
- beslutsrekommendationer,
- överlämningar till utveckling eller förvaltning.

När organisationen börjar arbeta mer agilt kan samma ansvar behöva uttryckas mer som närvaro och förmåga:

- att hjälpa teamet förstå konsekvenser,
- att skilja mellan beslut som kan tas lokalt och beslut som behöver lyftas,
- att formulera ramar som ger handlingsutrymme,
- att skapa dialog mellan team, verksamhet, säkerhet och förvaltning,
- att synliggöra risker utan att stoppa allt arbete,
- att dokumentera beslut i den form som faktiskt behövs.

Det invanda sättet är alltså inte fel. Men det kan bli otillräckligt om det bara bygger på att arkitekten först tänker färdigt och sedan förmedlar svaret.

Varför det kan skapa friktion i agil utveckling

Friktionen uppstår ofta när arkitektens ansvar är organiserat som en separat besluts punkt medan teamets arbete är organiserat som ett löpande flöde.

Teamet behöver fatta många små och medelstora beslut för att komma framåt. Arkitekten behöver säkerställa att besluten hänger ihop med helheten. Om alla beslut måste vänta på arkitekten uppstår köer. Om inga beslut involverar arkitekten uppstår risk för fragmentering. Utmaningen är att skapa ett arbetssätt där arkitekturen finns med utan att varje fråga blir ett godkännandeärende.

I praktiken syns friktionen ofta på några sätt.

Teamet väntar på arkitekturen, men vet inte exakt vad de väntar på. Arkitekten väntar på tydligare krav, men kraven blir tydligare först när teamet börjar bygga och testa. Produktägaren vill prioritera nytta, men arkitekten ser beroenden som inte syns i backloggen. Säkerhet och förvaltning behöver involveras, men bjuds in först när lösningen redan tagit form.

Det här betyder inte att någon roll gör fel. Det betyder att arbetet behöver kopplas ihop tätare.

Ett särskilt vanligt problem är att arkitektur reduceras till granskning. Arkitekten kommer in när teamet redan har format ett förslag och förväntas säga ja eller nej. Det gör arkitekten sen, reaktiv och ibland bromsande. Det gör också teamet beroende av att gissa vad arkitekten kommer att godkänna.

Ett annat vanligt problem är att teamnära arbete misstolkas som att arkitekten ska släppa helheten. Då fattas beslut nära koden, men utan tillräcklig koppling till långsiktiga egenskaper som förvaltningsbarhet, informationssäkerhet, återanvändbarhet, integration och spårbarhet.

Ett mer agilt arkitekturarbete behöver undvika båda problemen.

Ett mer agilt förhållningssätt

Ett mer agilt förhållningssätt börjar med att skilja mellan ansvar och kontroll.

Arkitekten har fortfarande ansvar för helhet, struktur, viktiga vägval och lösningens långsiktiga egenskaper. Men ansvar behöver inte alltid betyda att arkitekten själv fattar varje beslut eller godkänner varje detalj. Ansvar kan också betyda att skapa ramar, principer, dialoger och beslutsvägar som gör det möjligt för andra att fatta bra beslut.

Det är här tre begrepp blir viktiga: kontinuerligt arkitekturarbete, teamnära arkitektur och arkitekt som möjliggörare.

Kontinuerligt arkitekturarbete innebär att arkitekturen utvecklas och förvaltas löpande under arbetets gång. Den finns inte bara i ett dokument före utveckling eller i en granskning efteråt. Den finns i återkommande samtal, i principer, i beslut som dokumenteras när de fattas och i justeringar när ny kunskap uppstår.

Teamnära arkitektur innebär att arkitekturen kopplas till teamets vardag. Arkitekten behöver förstå vad teamet försöker åstadkomma nu, vilka beslut som står

framför dem och vilka risker som faktiskt är aktuella. Det betyder inte att arkitekten blir teamets chef eller detaljstyr designen. Det betyder att arkitekturen blir tillgänglig när beslut behöver fattas.

Arkitekt som möjliggörare innebär att arkitekten hjälper organisationen att fatta bättre beslut, inte bara fler beslut genom arkitekten. Ibland innebär det att själva ta fram ett vägval. Ibland innebär det att formulera en princip. Ibland innebär det att ställa rätt frågor. Ibland innebär det att föra samman personer som annars inte möts förrän för sent.

Ett praktiskt sätt att tänka är att arkitekten bidrar på tre nivåer:

1. **Riktning:** Vilka principer, begränsningar och mål behöver vara tydliga?
2. **Beslutsstöd:** Vilka beslut står vi inför, och vem har bäst kunskap för att fatta dem?
3. **Lärande:** Vad behöver vi pröva, följa upp eller ändra när vi vet mer?

Det här gör arkitektrollen mer aktiv, inte mindre. Men aktiviteten flyttar närmare flödet.

Exempel: Myndigheten för samhällstjänster

Sara och Lina bestämmer sig för att pröva ett annat upplägg. I stället för att Sara ska ta fram en fullständig lösningsbeskrivning innan teamet går vidare, bokar de ett kort arkitektursamtal två gånger i veckan under de första veckorna.

Syftet är inte att Sara ska godkänna varje detalj. Syftet är att fånga upp beslut som kan påverka helheten.

Inför första samtalet ber Sara teamet att ta med tre typer av frågor:

- beslut de tror att de kan fatta själva,
- beslut där de är osäkra på konsekvenserna,
- beslut som kan påverka andra team, system eller förvaltning.

Teamet kommer med frågor om formulärkomponenter, datalagring, integration och behörighet.

Sara föreslår att formulärkomponenterna kan hanteras lokalt av teamet så länge de följer myndighetens riktlinjer för tillgänglighet och återanvändning. Hon vill däremot att datalagring och integration hanteras mer medvetet, eftersom de kan påverka handläggningssystemet och framtida tjänster.

Tillsammans formulerar de tre enkla arkitekturramar för den första leveransen:

1. Den första versionen får bara omfatta ett smalt ärendeflöde.
2. Personuppgifter får inte lagras i den nya tjänsten utan tydligt beslut om ansvar, gallring och åtkomst.
3. Integration mot handläggningssystemet ska först göras genom ett avgränsat gränssnitt så att teamet inte bygger fast sig i interna detaljer.

Sara dokumenterar inte detta som en omfattande lösningsarkitektur. Hon skriver en kort beslutsanteckning och lägger till vilka frågor som fortfarande är öppna. Hon markerar också vilka frågor som behöver tas med Amir och vilka som kan hanteras av teamet.

På så sätt får teamet mer handlingsutrymme, samtidigt som arkitekturen blir mer närvarande. Sara har inte släppt sitt ansvar. Hon har gjort ansvaret användbart i teamets takt.

Vad arkitekten fortfarande behöver äga

Agil utveckling förändrar inte behovet av helhet. Vissa frågor behöver fortfarande någon som ser över teamgränser, systemgränser och tidshorisonter.

Arkitekten behöver ofta fortsatt ta ett tydligt ansvar för:

- att synliggöra strukturella konsekvenser av beslut,
- att identifiera beroenden mellan system, team och externa parter,
- att formulera principer som hjälper flera team att fatta samordnade beslut,
- att säkra att viktiga kvalitetsaspekter inte tappas bort,
- att bidra till långsiktig förvaltningsbarhet,
- att dokumentera viktiga vägval på ett sätt som går att förstå senare,
- att hjälpa beslutsforum att fokusera på rätt frågor.

Det här ansvaret ska inte blandas ihop med att arkitekten måste ha sista ordet i alla frågor. I en mer agil miljö är en viktig del av arkitektens professionalitet att veta när ett beslut bör fattas av arkitekten, när det bör fattas av teamet och när det bör fattas tillsammans med verksamhet, säkerhet, juridik, förvaltning eller styrning.

Ett enkelt test är att fråga:

- Påverkar beslutet bara teamets interna implementation?
- Påverkar det andra team, system eller framtida förändringsförmåga?
- Påverkar det myndighetens ansvar för säkerhet, spårbarhet, rättssäkerhet eller förvaltning?
- Är beslutet lätt att ändra senare, eller blir det dyrt att backa?
- Behöver beslutet förstås av andra än teamet om sex månader?

Ju mer beslutet påverkar helhet, långsiktighet eller ansvar, desto mer behöver arkitekten vara involverad. Men involverad behöver inte alltid betyda beslutande. Det kan betyda rådgivande, samordnande, principformulerande eller risktydliggörande.

Vanliga fallgropar

- Fallgrop: Arkitekten blir en sen granskare.
 - Varför den uppstår: Teamet vill arbeta självständigt och involverar arkitekten först när ett förslag redan finns.
 - Vad arkitekten kan göra i stället: Skapa tidiga och korta arkitektursamtal där beslut och risker fångas innan lösningen låses.
- Fallgrop: Teamnära arkitektur tolkas som detaljstyrning.
 - Varför den uppstår: Arkitekten kommer nära teamets vardag men saknar tydlig roll i samtalen.
 - Vad arkitekten kan göra i stället: Fokusera på principer, konsekvenser och beslut som påverkar helheten, inte på varje teknisk detalj.
- Fallgrop: Självständiga team lämnas utan arkitekturramar.
 - Varför den uppstår: Organisationen vill undvika gamla godkännandeprocesser och går för långt åt andra hållet.

- Vad arkitekten kan göra i stället: Formulera enkla ramar för beslut, beroenden, dokumentation och eskalering.
- Fallgrop: Arkitekturen blir ett parallellt spår.
 - Varför den uppstår: Arkitekten arbetar vidare med målbild och dokumentation utan tät koppling till teamets aktuella beslut.
 - Vad arkitekten kan göra i stället: Knyt arkitekturarbetet till teamets kommande vägval, risker och lärbehov.
- Fallgrop: Arkitekten släpper för mycket för att inte bromsa.
 - Varför den uppstår: Arkitekten vill stödja agil utveckling och undviker därför att lyfta svåra frågor.
 - Vad arkitekten kan göra i stället: Skilj mellan att bromsa och att tydliggöra konsekvenser. Viktiga risker ska synliggöras tidigt, men inte nödvändigtvis stoppa arbetet.

Frågor att ställa i situationen

När teamet vill fatta fler beslut själv kan arkitekten ställa frågor som hjälper utan att ta över:

- Vilka beslut står teamet inför de närmaste veckorna?
- Vilka av dessa beslut påverkar andra team, system eller framtida förändringsförmåga?
- Vilka beslut kan teamet fatta inom givna principer?
- Vilka beslut behöver dokumenteras så att andra kan förstå dem senare?
- Vilka kvalitetsaspekter riskerar att tappas bort om vi bara fokuserar på nästa leverans?
- Vilka frågor behöver säkerhet, juridik, förvaltning eller verksamhet vara med i tidigt?
- Vad behöver vara en gemensam arkitekturram, och vad kan vara lokalt teamansvar?
- Hur snabbt kan vi få återkoppling om beslutet visar sig vara fel?

Reflektionsfrågor

1. I vilka situationer blir du som arkitekt oftast en granskare snarare än en deltagare i beslutet?
2. Vilka typer av beslut brukar du vilja behålla själv, och varför?
3. Vilka beslut skulle ett team kunna fatta bättre om de fick tydligare principer i stället för fler godkännanden?
4. När har du sett självständiga team fatta beslut som senare skapade helhetsproblem?
5. Vad skulle krävas för att du skulle känna dig trygg med att flytta fler beslut närmare teamet?
6. Hur kan du vara mer närvarande i teamets beslut utan att bli en flaskhals?
7. Vilka samtal behöver du initiera tidigare med säkerhet, förvaltning eller verksamhet?
8. Vad är ett konkret steg du kan ta för att gå från kontrollpunkt till möjliggörare?

Snabb sammanfattning

Arkitektens ansvar försvinner inte i agil utveckling. Det förändras från att i första hand producera färdiga lösningsunderlag till att skapa förutsättningar för bra beslut över tid.

I en mer agil utvecklingslogik behöver arkitekturen vara närvarande i teamens vardag, inte bara i dokument och granskningar. Teamnära arkitektur innebär inte att arkitekten detaljstyr teamet. Det innebär att arkitektens helhetsperspektiv finns tillgängligt när beslut fattas.

En arkitekt som möjliggörare hjälper organisationen att skilja mellan beslut som teamet kan fatta själv, beslut som behöver gemensam dialog och beslut som behöver formell förankring. På så sätt kan teamet få mer handlingsutrymme utan att helhet, säkerhet, förvaltning och långsiktighet tappas bort.

Nästa steg

I nästa kapitel går vi vidare till frågan om beslut i sig. När arkitektens roll blir mer kontinuerlig och teamnära räcker det inte att fråga vem som fattar beslut. Vi behöver också förstå vilka beslut som behöver fattas tidigt, vilka som kan vänta och vilka som bör hållas öppna tills organisationen har lärt sig mer.

Nästa kapitel handlar därför om skiftet från tidig säkerhet till löpande beslutsförmåga.

Kapitel 3: Från tidig säkerhet till löpande beslutsförmåga

Varför detta kapitel finns

I en fasorienterad utvecklingslogik skapas ofta trygghet genom att viktiga vägval görs tidigt. Arkitektur tas fram, granskas och förankras innan utvecklingen går vidare. Det kan vara ett rationellt sätt att minska osäkerhet när uppdraget, lösningsrymden och ansvarsfördelningen är relativt stabila.

I agil IT-utveckling är förutsättningarna ofta mer rörliga. Behov, prioriteringar, tekniska möjligheter, regelkrav och användarinsikter kan förändras medan arbetet pågår. Då räcker det inte att försöka fatta alla viktiga beslut tidigt. Arkitekten behöver i stället bidra till en löpande beslutsförmåga: förmågan att fatta rätt beslut i rätt tid, med rätt personer och med lagom mycket information.

Det betyder inte att beslut ska skjutas upp av bekvämlighet. Det betyder att arkitekten behöver skilja mellan beslut som måste fattas tidigt, beslut som kan vänta och beslut som bör utforskas innan de låses.

Situationen: när alla vill ha trygghet men ingen vet tillräckligt

Sara är med i arbetet med den nya medborgartjänsten på Myndigheten för samhällstjänster. Tjänsten ska låta medborgare följa sitt ärende digitalt och komplettera uppgifter utan att kontakta handläggare manuellt.

Redan i början uppstår flera arkitekturfrågor:

- Ska tjänsten läsa data direkt från det äldre handläggningssystemet eller via ett mellanlager?
- Hur ska behörighet och åtkomstkontroll hanteras?
- Vilka händelser behöver loggas för spårbarhet?
- Ska tjänsten byggas som en fristående komponent eller nära befintlig plattform?
- Hur mycket av informationsmodellen behöver vara stabil innan teamet börjar bygga?

Flera personer vill ha ett tydligt svar. Produktägaren vill veta vad som går att leverera först. Teamet vill veta vad de kan börja bygga. Säkerhetsfunktionen vill förstå riskerna. Förvaltningen vill undvika en lösning som blir svår att ta över. Styrgruppen vill ha trygghet i att vägvalen är genomtänkta.

Sara känner igen mönstret. I tidigare projekt hade hon ofta tagit fram en mer komplett lösningsarkitektur innan utvecklingen startade. Det gav en känsla av kontroll. Men den här gången är flera grundantaganden fortfarande osäkra. Om hon låser för mycket nu riskerar lösningen att bygga på gissningar. Om hon inte låser något alls riskerar teamet att gå åt olika håll.

Frågan blir därför inte: "Vilken arkitektur ska vi bestämma nu?"

Frågan blir snarare: "Vilka beslut behöver vi fatta nu, vilka behöver vi förbereda, och vilka behöver vi lära oss mer om innan vi låser?"

Det invanda sättet att agera

När osäkerheten är hög är det vanligt att arkitekten försöker skapa trygghet genom att göra mer analys. Det kan innebära att samla in fler krav, beskriva fler lösningsalternativ, rita fler vyer, boka fler avstämningar och försöka få fler principbeslut på plats innan teamet börjar bygga.

Det invanda beteendet är ofta välmenande. Arkitekten vill undvika kostsamma fel, sena omtag och otydliga ansvar. I en myndighetsmiljö är det dessutom rimligt att vara försiktig. Beslut kan påverka rättssäkerhet, informationssäkerhet, spårbarhet, upphandling, förvaltning och samverkan med andra aktörer.

Men i agil utveckling kan det invanda beteendet skapa tre problem.

För det första kan arkitekten råka behandla alla beslut som om de vore lika viktiga. Då läggs mycket tid på att låsa detaljer som senare visar sig vara lätta att ändra, medan verkligt svåra beslut kanske inte blir tillräckligt synliga.

För det andra kan arbetet hamna i vänteläge. Teamet väntar på arkitekturen. Arkitekten väntar på krav. Kravarbetet väntar på verksamhetsbeslut. Verksamheten väntar på att se vad som är möjligt. Alla väntar på någon annans tydlighet.

För det tredje kan tidiga beslut få för hög status. När ett beslut väl är dokumenterat och förankrat kan det bli svårt att ändra, även om ny kunskap visar att beslutet byggde på fel antaganden.

Varför det kan skapa friktion i agil utveckling

Agil utveckling bygger på att viss kunskap uppstår genom arbetet. Det går ofta inte att tänka sig fram till alla svar. Vissa svar kommer först när teamet bygger något, testar en integration, möter användare, analyserar faktisk data eller upptäcker en begränsning i befintlig miljö.

Om arkitekturarbetet utgår från att osäkerhet alltid ska reduceras genom mer förarbete, kan det bromsa lärandet. Arkitekturen blir då något som ska vara färdigt innan utvecklingen får börja, i stället för något som hjälper utvecklingen att lära sig på ett kontrollerat sätt.

Samtidigt vore det oansvarigt att säga att "allt kan växa fram". Vissa beslut är dyra att ändra. Vissa säkerhets- och informationsfrågor behöver hanteras tidigt. Vissa integrationer eller datamodeller påverkar många delar av lösningen. Vissa vägval kan skapa beroenden som blir svåra att lösa senare.

Friktionen uppstår alltså inte för att arkitektur behövs eller inte behövs. Den uppstår när organisationen saknar ett gemensamt sätt att avgöra vilka beslut som behöver tas när.

Här blir arkitektens roll central. Arkitekten behöver hjälpa organisationen att sortera beslut, inte bara fatta dem.

Ett mer agilt förhållningssätt

Ett mer agilt förhållningssätt är att arbeta med beslutsförmåga snarare än enbart beslutspunkter.

En beslutspunkt är ett tillfälle där organisationen säger ja, nej eller välj alternativ A. Beslutsförmåga är bredare. Det handlar om att skapa förutsättningar för bra beslut över tid.

Det innebär att arkitekten hjälper organisationen att:

- se vilka beslut som finns framför den,
- förstå vilka beslut som är brådskande och vilka som kan vänta,
- skilja mellan reversibla och svårreversibla beslut,
- dokumentera beslut på ett sätt som gör dem möjliga att följa upp,
- synliggöra antaganden bakom beslut,
- skapa lärande innan svåra beslut låses,
- se när ett tidigare beslut behöver omprövas.

Reversibla och svårreversibla beslut

Ett reversibelt beslut är ett beslut som går att ändra senare utan orimlig kostnad, risk eller påverkan. Exempel kan vara en lokal komponentstruktur, ett namn på en intern modul eller en mindre implementationsteknik som bara påverkar ett team.

Ett svårreversibelt beslut är ett beslut som blir dyrt eller svårt att ändra. Exempel kan vara ett integrationsmönster mellan flera system, en grundläggande informationsmodell, en säkerhetsmodell, ett plattformsval eller ett beroende till en extern leverantör.

Poängen är inte att alla beslut kan delas perfekt i två kategorier. Poängen är att frågan hjälper arkitekten och teamet att föra ett bättre samtal.

En enkel fråga kan vara:

Om vi har fel om detta beslut, hur svårt blir det att ändra senare?

Om svaret är "ganska enkelt" kanske beslutet kan ligga närmare teamet och fattas snabbare. Om svaret är "mycket svårt" behöver beslutet troligen mer gemensam analys, tydligare dokumentation och bredare förankring.

Lagom sena beslut

I traditionellt projektarbete talar man ofta om vikten av tidiga beslut. I agil utveckling är det ofta mer användbart att tala om lagom sena beslut.

Ett lagom sent beslut fattas inte tidigare än nödvändigt, men inte heller så sent att det skapar väntan, risk eller omarbete.

Det kräver omdöme. Om beslutet fattas för tidigt kan det bygga på svaga antaganden. Om det fattas för sent kan teamet sakna riktning eller bygga in fel struktur.

Arkitektens bidrag blir att formulera beslutets "senaste ansvarsfulla tidpunkt". Det är inte ett exakt datum i alla lägen, utan en praktisk bedömning:

- När behöver teamet veta detta för att kunna fortsätta?
- Vilka andra beslut påverkas?
- Vilken risk ökar om vi väntar?
- Vilken kunskap kan vi få om vi väntar lite?
- Vad kan vi göra nu för att hålla flera alternativ öppna?

Beslut, antaganden och lärande

I agil utveckling är det viktigt att skilja mellan beslut och antaganden.

Ett beslut är något organisationen väljer att agera utifrån.

Ett antagande är något organisationen tror, men ännu inte vet.

Många arkitekturdiskussioner blir svåra för att antaganden behandlas som beslut. Någon säger till exempel: "Det äldre systemet klarar nog inte Realtidsanrop." Om detta skrivs in som en begränsning utan att testas kan hela lösningen formas runt något som kanske inte stämmer. Om det i stället behandlas som ett antagande kan teamet planera en teknisk undersökning, en dialog med förvaltningen eller en begränsad testimplementation.

Arkitektens fråga blir då:

Vet vi detta, har vi beslutat detta, eller antar vi detta?

Den frågan kan låta enkel, men den förändrar samtalet. Den hjälper teamet att se vad som är fast, vad som är valt och vad som behöver undersökas.

Exempel: Myndigheten för samhällstjänster

Sara samlar produktägaren Erik, teamrepresentanten Lina och säkerhetsspecialisten Amir för att prata om medborgartjänstens åtkomst till ärendedata.

Från början låter diskussionen som en vanlig lösningsdiskussion.

Erik vill att medborgarna snabbt ska kunna se status på sina ärenden. Lina vill veta vilket API teamet ska bygga mot. Amir vill säkerställa att åtkomstkontroll,loggning och spårbarhet inte blir efterhandskonstruktioner. Sara ser att valet av integrationsmönster kan påverka både säkerhet, prestanda, förvaltning och framtida tjänster.

I stället för att direkt föreslå en målarkitektur ritar Sara upp tre kolumner på tavlan:

1. Beslut som behöver fattas nu.
2. Beslut som kan vänta.
3. Antaganden som behöver undersökas.

I den första kolumnen hamnar att medborgartjänsten inte får läsa mer information än vad medborgaren har rätt att se, och att alla åtkomster till ärendeinformation

behöver vara spårbara. Det är inte detaljerad design, men det är arkitektoniska startvillkor.

I den andra kolumnen hamnar exakt teknik för hur statusinformationen ska presenteras i gränssnittet. Det påverkar inte de svåraste vägvalen ännu.

I den tredje kolumnen hamnar antagandet att det äldre handläggningssystemet inte klarar den typ av anrop som tjänsten skulle behöva. Teamet får i uppgift att undersöka detta genom en begränsad teknisk spike och dialog med förvaltningen.

Sara dokumenterar inte detta som en färdig lösningsarkitektur. Hon dokumenterar det som en beslutsbild:

- vad som är beslutat,
- varför det är beslutat,
- vad som fortfarande är öppet,
- vilka antaganden som behöver testas,
- när frågan behöver tas upp igen.

Efter mötet har teamet tillräcklig riktning för att börja undersöka lösningen. Samtidigt har de viktigaste riskerna inte gömts undan. De har blivit synliga.

Vanliga fallgropar

- Fallgröp: Alla beslut behandlas som arkitekturbeslut.
 - Varför den uppstår: Arkitekten vill säkra helheten och undvika lokala beslut som får oväntade konsekvenser.
 - Vad arkitekten kan göra i stället: Skilj mellan beslut som påverkar helhet, kvalitet och handlingsutrymme och beslut som teamet kan fatta lokalt.
- Fallgröp: Beslut skjuts upp utan plan.
 - Varför den uppstår: Organisationen vill vara agil och undvika att låsa sig för tidigt.
 - Vad arkitekten kan göra i stället: Formulera vad som behöver läras innan beslutet tas och när beslutet senast behöver fattas.
- Fallgröp: Antaganden dokumenteras som fakta.
 - Varför den uppstår: I tidiga diskussioner blandas erfarenhet, oro och tidigare sanningar ihop med verifierad kunskap.
 - Vad arkitekten kan göra i stället: Märk tydligt vad som är känt, vad som är beslutat och vad som är antaget.
- Fallgröp: Tidiga beslut blir svåra att ompröva.
 - Varför den uppstår: När ett beslut har presenterats i styrning eller dokumentation får det ofta organisatorisk tyngd.
 - Vad arkitekten kan göra i stället: Dokumentera beslutets grund, giltighet och omprövningspunkter.
- Fallgröp: Teamet tolkar öppna beslut som att allt är fritt.
 - Varför den uppstår: Arkitekten vill ge teamet handlingsutrymme men är otydlig med ramarna.
 - Vad arkitekten kan göra i stället: Beskriv tydliga startvillkor, principer och riskområden även när detaljlösningen är öppen.

Frågor att ställa i situationen

När du som arkitekt hamnar i en osäker beslutssituation kan följande frågor hjälpa:

1. Vilket beslut är det egentligen vi försöker fatta?
2. Vad händer om vi väntar med beslutet?
3. Vad händer om vi fattar beslutet nu och har fel?
4. Är beslutet reversibelt eller svårreversibelt?
5. Vilka antaganden ligger bakom beslutet?
6. Vilken kunskap saknas för att fatta ett bättre beslut?
7. Kan vi lära oss det genom en begränsad undersökning?
8. Vem har bäst kunskap för att vara med i beslutet?
9. Vem påverkas av beslutet senare?
10. Hur behöver beslutet dokumenteras för att vara användbart?

Reflektionsfrågor

1. Vilka typer av beslut brukar du vilja få på plats tidigt?
2. Vilka av dessa beslut är faktiskt svårreversibla?
3. Vilka beslut i din vardag fattas ofta för tidigt av vana?
4. Vilka beslut skjuts ofta upp för länge?
5. Hur brukar ni dokumentera varför ett arkitekturbeslut fattades?
6. När omprövar ni tidigare arkitekturbeslut?
7. Vilka antaganden behandlas ofta som fakta i din organisation?
8. Vad skulle förändras om ni oftare frågade: "Vad behöver vi lära oss innan vi låser detta?"

Snabb sammanfattning

- I agil utveckling behöver arkitekten bidra till löpande beslutsförmåga, inte bara till tidiga beslutspunkter.
- Alla beslut är inte lika viktiga. Arkitekten behöver hjälpa organisationen att sortera beslut efter risk, påverkan och reversibilitet.
- Reversibla beslut kan ofta fattas snabbare och närmare teamet.
- Svårreversibla beslut kräver mer omsorg, tydligare förankring och bättre dokumentation.
- Ett lagom sent beslut fattas när tillräcklig kunskap finns, men innan väntan, risk eller omarbete blir för stort.
- Antaganden bör inte smygas in som fakta. De bör synliggöras och undersökas.
- Arkitektens uppgift är inte att eliminera all osäkerhet, utan att göra osäkerheten möjlig att hantera.

Nästa steg

Nästa kapitel går in i en situation där osäkerheten ofta börjar: när behovsbilden ännu inte är stabil. Där ser vi hur arkitekten kan bidra redan innan verksamheten har formulerat ett tydligt kravpaket, utan att för tidigt låsa lösningen.

Kapitel 4: När behovsbilden är osäker

Varför detta kapitel finns

Många arkitekter är vana vid att komma in när ett behov har börjat ta form. Det kan finnas en förstudie, ett projektdirektiv, en effektbeskrivning, en kravbild eller åtminstone en relativt tydlig formulering av vad verksamheten vill åstadkomma. Utifrån det kan arkitekten analysera konsekvenser, identifiera beroenden, formulera lösningsalternativ och skapa beslutsunderlag.

I mer agil IT-utveckling är behovsbilden ofta mindre stabil när arbetet börjar. Det är inte alltid ett tecken på bristande styrning. Ibland är det själva poängen: organisationen vill snabbare undersöka vad som faktiskt skapar nytta, vad användare behöver, vilka regelmässiga begränsningar som är avgörande och vilka tekniska vägval som är rimliga.

Det kan kännas obekvämt för en erfaren arkitekt. Om behovet är oklart blir också lösningsrymden otydlig. Om lösningsrymden är otydlig är det svårt att veta vilka arkitekturbeslut som behöver fattas. Och om besluten inte fattas i tid kan teamet bygga något som senare visar sig vara svårt att ändra, förvalta, säkra eller samordna med andra system.

Det här kapitlet handlar om hur arkitekten kan bidra när behovsbilden är osäker. Inte genom att kräva fullständig klarhet innan något får börja, men inte heller genom att acceptera oklarhet som något teamet får lösa längs vägen. Arkitektens bidrag blir i stället att göra osäkerheten begriplig, prövbar och hanterbar.

Situationen: “Vi behöver bara en enklare digital tjänst”

På Myndigheten för samhällstjänster vill verksamheten skapa en ny digital tjänst där medborgare ska kunna följa sitt ärende och komplettera uppgifter digitalt. I den första beskrivningen låter behovet relativt enkelt:

“Medborgaren ska kunna logga in, se status i sitt ärende och ladda upp kompletterande underlag.”

Erik, som är verksamhetsnära produktägare, vill gärna komma igång. Han ser att dagens manuella hantering leder till många telefonsamtal, långa väntetider och frustration både hos medborgare och handläggare. Lina och teamet ser också att det borde gå att skapa ett första inkrement ganska snabbt: en inloggad vy, några statusmeddelanden och en möjlighet att skicka in en fil.

Sara lyssnar och ser direkt flera öppna frågor:

- Vilka ärendetyper ska tjänsten stödja först?
- Är statusinformationen begriplig för medborgaren, eller är den skriven för interna handläggare?
- Vilka uppgifter får visas digitalt?
- Vilka kompletteringar får lämnas in utan manuell kontroll?
- Hur ska handlingar kopplas till rätt ärende?
- Vilka data kommer från det äldre handläggningssystemet?
- Behöver tjänsten fungera för ombud eller bara för den person ärendet gäller?
- Vilka krav finns på spårbarhet, arkivering och informationssäkerhet?

I ett fasorienterat arbetssätt hade Saras naturliga reaktion kanske varit att föreslå en fördjupad behovsanalys innan lösningsarbetet går vidare. Det hade inte varit orimligt. Behovet påverkar många delar av lösningen. Men i den här situationen vill organisationen arbeta mer iterativt. Den vill inte vänta flera månader på en komplett behovsbild innan den lär sig något av användare och handläggare.

Frågan blir därför inte om behovsbilden är tillräckligt komplett för att göra hela arkitekturen. Frågan blir: vad behöver vi förstå nu för att kunna ta nästa ansvarsfulla steg?

Det invanda sättet att agera

När behovsbilden är osäker kan arkitektens invanda arbetssätt ofta se ut så här:

1. Be om tydligare behovsbeskrivning.
2. Efterfråga mer kompletta krav.
3. Identifiera luckor och beroenden.
4. Vänta med lösningsförslag tills verksamheten har preciserat vad den vill ha.
5. Ta fram flera lösningsalternativ när behovet är mer stabilt.
6. Förankra ett alternativ genom beslutsunderlag.

Detta sätt att agera har flera styrkor. Det minskar risken att arkitekten designar mot fel problem. Det hjälper verksamheten att se konsekvenser av otydliga behov. Det kan också skydda organisationen från att starta utveckling på alltför lösa grunder.

Men i en agil kontext kan samma arbetssätt skapa friktion. Om varje oklarhet gör att arkitekturen väntar på mer analys kan teamet tappa fart. Om arkitekten blir den som hela tiden pekar på allt som ännu inte är utrett kan rollen uppfattas som bromsande. Och om behovsanalysen försöker skapa för mycket säkerhet i början kan organisationen ändå upptäcka senare att den analyserade fel saker.

Den viktiga skillnaden är att osäkerhet inte alltid ska elimineras innan utveckling startar. Ofta behöver den struktureras så att den kan undersökas.

Varför det kan skapa friktion i agil utveckling

Agil utveckling utgår från att viss kunskap uppstår genom arbetet. Det gäller inte bara teknisk kunskap, utan också verksamhetskunskap. Användare kan ha svårt att beskriva vad de behöver innan de ser något konkret. Handläggare kan beskriva ett arbetssätt som det borde fungera, men först i mötet med ett nytt digitalt stöd

blir undantag, specialfall och tysta regler synliga. Juridiska eller säkerhetsmässiga frågor kan också bli tydligare när lösningalternativen konkretiseras.

Det betyder inte att arkitekten ska acceptera vaga behov. Det betyder att arkitekten behöver skilja mellan tre typer av oklarhet:

1. Oklarhet som måste reduceras innan start

Vissa frågor kan vara så grundläggande att arbetet inte bör gå vidare innan de är tydligare. Exempel:

- Vilken målgrupp tjänsten gäller.
- Om informationen över huvud taget får visas digitalt.
- Om det finns rättsligt stöd för den tänkta behandlingen av personuppgifter.
- Om ett centralt system kan tillhandahålla nödvändiga data.
- Om lösningen kräver en förändring som ligger utanför teamets mandat.

Här behöver arkitekten hjälpa till att stoppa, pausa eller avgränsa arbetet tills frågan är hanterad. Det är inte att vara bakåtsträvande. Det är att skydda organisationen från att bygga på en grund som kan visa sig felaktig.

2. Oklarhet som kan undersökas genom ett begränsat steg

Andra frågor behöver inte lösas helt innan arbetet börjar. De kan undersökas genom ett smalt inkrement, en prototyp, ett tekniskt test, en användardialog eller ett begränsat lösningsspår. Exempel:

- Vilken statusinformation medborgare faktiskt förstår.
- Vilka kompletteringar som är vanligast.
- Hur handläggarna vill bli notifierade.
- Vilken integrationslösning som fungerar bäst i praktiken.
- Hur mycket av det äldre systemets datamodell som behöver exponeras.

Här blir arkitektens uppgift att hjälpa teamet att formulera vad man behöver lära sig, vilka risker som ska hållas begränsade och vilka beslut som inte bör låsas för tidigt.

3. Oklarhet som är acceptabel tills vidare

En tredje kategori är frågor som är verkliga men inte avgörande just nu. Exempel:

- Hur tjänsten ska skalas till alla ärendetyper på sikt.
- Exakt hur förvaltningsorganisationen ska se ut efter införandet.
- Hur framtida analysfunktioner kan kopplas på.
- Vilka rapporteringsbehov som kan uppstå om två år.

Dessa frågor ska inte ignoreras, men de behöver kanske inte styra dagens designbeslut. Arkitekten kan dokumentera dem som antaganden, framtida beslut eller bevakningspunkter.

Friktionen uppstår ofta när alla tre typer av oklarhet behandlas på samma sätt. Om allt måste utredas före start blir arbetet tungt och långsamt. Om inget behöver utredas före start blir arbetet riskfyllt. Arkitektens värde ligger i att kunna göra skillnaden synlig.

Ett mer agilt förhållningssätt

Ett mer agilt förhållningssätt till osäkra behov innebär att arkitekten rör sig från att fråga:

“Är behovet tillräckligt tydligt för att vi ska kunna ta fram lösningen?”

till att fråga:

“Vilken del av behovet behöver vi förstå bättre för att kunna ta nästa kloka steg?”

Det förändrar arkitektens bidrag på flera sätt.

Från krav på klarhet till struktur på osäkerhet

Sara behöver inte acceptera att behovet beskrivs som “en enklare digital tjänst”. Men hon behöver inte heller kräva en komplett behovsspecifikation innan något kan hända. Hon kan i stället hjälpa gruppen att dela upp osäkerheten:

- Vad vet vi?
- Vad tror vi?
- Vad antar vi?
- Vad måste vara sant för att lösningen ska fungera?
- Vad behöver vi undersöka först?
- Vilka antaganden kan bli dyra om de är fel?

Den typen av frågor gör behovsbilden mer användbar utan att låtsas att den är färdig.

Från lösningsförslag till lösningsrymd

Ett centralt begrepp i detta kapitel är **lösningsrymd**. Med lösningsrymd menar vi det möjliga utrymme av lösningar som kan möta ett behov, innan ett visst vägval har låsts.

När behovet är osäkert är det ofta för tidigt att välja en detaljerad lösning. Däremot är det värdefullt att beskriva lösningsrymden:

- Vilka lösningstyper verkar möjliga?
- Vilka vägval skulle begränsa oss snabbt?
- Vilka alternativ håller flera möjligheter öppna?
- Vilka begränsningar är verkliga redan nu?
- Vilka kvalitetskrav påverkar alla rimliga alternativ?

För Sara kan det innebära att hon inte direkt ritar en målarkitektur för hela medborgartjänsten. Hon kan i stället beskriva tre möjliga vägar:

1. En enkel visningstjänst som läser status från handläggningssystemet.

2. En mer självständig tjänst med ett mellanlager som skyddar det äldre systemet.
3. En stegvis väg där första inkrementet bara hanterar en ärendetyp och en begränsad mängd information.

Poängen är inte att välja allt direkt. Poängen är att ge samtalet struktur.

Från kravinsamling till hypotesdriven behovsanalys

Ett annat begrepp är **hypotesdriven behovsanalys**. Det betyder att behov, antaganden och lösningssidéer behandlas som något som behöver prövas och förfinas, inte bara samlas in och dokumenteras.

I stället för att säga:

“Medborgare behöver kunna följa sitt ärende digitalt.”

kan teamet formulera en hypotes:

“Om medborgare kan se tydlig status och vilka kompletteringar som saknas, minskar antalet telefonsamtal till handläggarna och medborgaren upplever större kontroll.”

Den formuleringen gör flera saker tydliga. Den säger vem nyttan gäller. Den antyder vilken funktion som behövs. Den kopplar funktionen till en effekt. Den visar också vad som behöver följas upp.

För arkitekten är detta användbart eftersom arkitekturfrågor kan kopplas till hypotesen:

- Vilken information krävs för att visa status?
- Hur aktuell behöver informationen vara?
- Vilka kompletteringar får visas eller skickas in digitalt?
- Vilka risker uppstår om statusen är otydlig eller fel?
- Vilket integrationsmönster räcker för att pröva hypotesen?

Hypotesdriven behovsanalys gör alltså inte arkitekturen mindre viktig. Den ger arkitekten ett tydligare sätt att koppla arkitekturfrågor till verksamhetslärande.

Från tekniska konsekvenser till arkitektoniska konsekvenser av behov

Det tredje begreppet är **arkitektoniska konsekvenser av behov**. Ett behov är sällan bara en önskad funktion. Det kan påverka information, integrationer, säkerhet, ansvar, processer, prestanda, datakvalitet, förvaltning och framtida förändringsförmåga.

När verksamheten säger “medborgaren ska kunna komplettera sitt ärende digitalt” hör Sara flera möjliga konsekvenser:

- Kompletteringen behöver kopplas till rätt ärende.
- Filformat och filstorlekar behöver hanteras.
- Det kan krävas viruskontroll eller annan teknisk kontroll.
- Handläggare behöver se vad som har kommit in.
- Händelsen behöver loggas.

- Informationen kan behöva diarieföras eller arkiveras.
- Det behöver vara tydligt vem som får komplettera.
- Det kan finnas krav på tillgänglighet och begriplighet.
- Tjänsten kan skapa nya belastningsmönster på äldre system.

Arkitektens uppgift är inte att göra alla dessa frågor till hinder. Uppgiften är att visa vilka konsekvenser ett till synes enkelt behov kan få, så att gruppen kan välja ett klokt första steg.

Exempel: Myndigheten för samhällstjänster

Sara föreslår ett kort arbetsmöte med Erik, Lina, Amir från informationssäkerhet och två handläggare från verksamheten. Syftet är inte att skriva en komplett kravspecifikation. Syftet är att förstå vilken del av behovet som är mest värdefull och mest riskfylld att undersöka först.

Hon ritar upp tre kolumner på en digital tavla:

1. Det vi vet.
2. Det vi antar.
3. Det vi behöver undersöka.

Under "det vi vet" skriver gruppen:

- Många medborgare ringer för att fråga om status.
- Handläggare lägger mycket tid på återkommande frågor.
- Det äldre handläggningssystemet har statuskoder, men de är skrivna för intern användning.
- Vissa ärenden innehåller känsliga personuppgifter.
- Myndigheten vill börja med en begränsad ärendetyp.

Under "det vi antar" skriver gruppen:

- Medborgare vill främst veta om något saknas.
- En enkel statusvy kan minska antalet telefonsamtal.
- Handläggningssystemets statuskoder går att översätta till medborgarvänlig text.
- Det går att läsa status utan att påverka det äldre systemets stabilitet.
- De flesta kompletteringar gäller ett fåtal dokumenttyper.

Under "det vi behöver undersöka" skriver gruppen:

- Vilka statuskoder som faktiskt kan visas externt.
- Vilka formuleringar medborgare förstår.
- Hur aktuell statusinformationen behöver vara.
- Om kompletteringar ska ingå i första inkrementet eller vänta.
- Vilka säkerhets- och dataskyddsfrågor som måste vara hanterade innan test med riktiga användare.

Sara märker att samtalet förändras. I stället för att diskutera om behovet är "klart" börjar gruppen diskutera vilket lärande som behövs först.

Lina föreslår att teamet börjar med att visa status för en enda ärendetyp, utan möjlighet till komplettering. Erik är först tveksam eftersom kompletteringarna var en viktig del av nyttan. Men handläggarna säger att många samtal faktiskt handlar

om status, inte kompletteringar. Amir påpekar att en ren statusvy kan vara enklare att riskbedöma än filuppladdning.

Sara formulerar ett första arkitektoniskt antagande:

Första inkrementet ska pröva om en extern statusvy för en avgränsad ärendetyp kan minska osäkerheten kring informationsvisning, begrepp och integration, utan att samtidigt införa de risker som följer av digital komplettering.

Det är inte en färdig arkitektur. Men det är ett tydligt steg. Det säger vad man ska pröva, vad man medvetet avgränsar bort och varför.

Att ställa bättre frågor tidigare

När behovsbilden är osäker behöver arkitekten ofta bidra med frågor som öppnar samtalet snarare än stänger det. Det är skillnad på frågor som indirekt säger "det här är inte tillräckligt utrett" och frågor som hjälper gruppen att förstå vad som behöver bli tydligare.

Mindre hjälpsamma frågor kan vara:

- "Var finns den fullständiga kravspecifikationen?"
- "Har verksamheten bestämt exakt vad den vill ha?"
- "Vem har godkänt behovsbilden?"
- "Hur ska jag kunna ta fram arkitektur utan tydliga krav?"

De frågorna kan vara begripliga, men de riskerar att leda till väntan eller försvar. Mer hjälpsamma frågor är ofta:

- "Vilken nytta vill vi kunna se först?"
- "Vilket antagande är mest riskabelt om det visar sig vara fel?"
- "Vilka behov påverkar arkitekturen mest?"
- "Vad behöver vi veta innan teamet bygger något som blir svårt att ändra?"
- "Vilken del av lösningen kan hjälpa oss att lära utan att låsa för mycket?"
- "Vilka begränsningar gäller oavsett vilket lösningsalternativ vi väljer?"

Sådana frågor gör att arkitekten fortfarande tar ansvar för kvalitet och helhet, men på ett sätt som hjälper arbetet framåt.

Att skilja mellan behov, lösning och begränsning

En vanlig källa till otydlighet är att behov, lösningsidéer och begränsningar blandas ihop.

Verksamheten kan säga:

"Vi behöver en app."

Det kan låta som ett behov, men är egentligen en lösningsidé. Behovet kanske är att medborgaren ska kunna få snabb återkoppling, lämna kompletteringar eller förstå vad som händer i ärendet. En app kan vara en lösning, men kanske inte den bästa eller första.

Verksamheten kan också säga:

“All information måste hämtas i realtid.”

Det kan låta som ett krav, men kan vara ett antagande. Kanske behöver vissa uppgifter vara aktuella direkt, medan andra kan uppdateras med viss fördröjning. Skillnaden påverkar arkitekturen kraftigt.

Eller så kan någon säga:

“Det äldre systemet klarar inte fler integrationer.”

Det kan vara en verklig begränsning, men det kan också vara en historisk erfarenhet som behöver undersökas. Kanske gäller det bara vissa typer av belastning. Kanske går det att skydda systemet med ett mellanlager. Kanske är begränsningen organisatorisk snarare än teknisk.

Arkitektens roll blir att hjälpa gruppen att sortera:

- Behov: Vad ska bli bättre, för vem och varför?
- Lösningssidé: Hur skulle behovet kunna mötas?
- Begränsning: Vad sätter ramar för möjliga lösningar?
- Antagande: Vad tror vi, men behöver pröva?
- Beslut: Vad väljer vi att göra nu?

Den sorteringen kan verka enkel, men den förändrar ofta samtalet. Den gör det möjligt att börja utan att låsa sig.

Arkitektens balans: tillräckligt kritisk, tillräckligt nyfiken

När behovsbilden är osäker behöver arkitekten hålla två perspektiv samtidigt.

Det första är det kritiska perspektivet. Arkitekten behöver se risker, beroenden, motsägelser och konsekvenser. Det är en viktig del av rollen. Om ingen ställer de svåra frågorna kan teamet bygga snabbt men fel.

Det andra är det nyfikna perspektivet. Arkitekten behöver också hjälpa organisationen att lära. Alla oklarheter är inte fel. Vissa oklarheter är en naturlig del av att arbeta med verkliga behov i komplexa miljöer.

Om arkitekten bara är kritisk uppstår väntan. Om arkitekten bara är nyfiken uppstår risk. Balansen ligger i att vara tydlig med vad som är farligt att lämna oklart och vad som är klokt att undersöka stegvis.

En användbar fråga är:

“Vad är det minsta vi behöver förstå för att kunna ta nästa steg utan att skapa onödig framtida inlåsning?”

Den frågan håller ihop ansvar och lärande.

Vanliga fallgropar

- Fallgrop: Att kräva komplett behovsbild innan något får börja.
 - Varför den uppstår: Arkitekten vill undvika felaktiga lösningsbeslut och skydda organisationen från omarbete.

- Vad arkitekten kan göra i stället: Skilj mellan frågor som måste klargöras före start, frågor som kan undersökas genom ett begränsat inkrement och frågor som kan bevakas senare.
- Fallgrop: Att acceptera vaga behov för att inte uppfattas som bromsande.
 - Varför den uppstår: Arkitekten vill stödja det agila arbetssättet och undvika att bli en kontrollpunkt.
 - Vad arkitekten kan göra i stället: Hjälp gruppen att formulera antaganden, hypoteser och arkitektoniska konsekvenser utan att kräva en fullständig kravspecifikation.
- Fallgrop: Att omvandla behovsosäkerhet till teknisk lösning för snabbt.
 - Varför den uppstår: Tekniska lösningsskisser kan kännas mer konkreta och hanterbara än otydliga verksamhetsbehov.
 - Vad arkitekten kan göra i stället: Beskriv först Lösningssrymden och vilka vägval som bör hållas öppna.
- Fallgrop: Att behandla alla osäkerheter som lika viktiga.
 - Varför den uppstår: Erfarna arkitekter ser många konsekvenser samtidigt och vill fånga helheten.
 - Vad arkitekten kan göra i stället: Prioritera osäkerheter utifrån risk, beslutspåverkan och kostnad om antagandet visar sig fel.
- Fallgrop: Att låta "agilt" betyda att behov inte behöver analyseras.
 - Varför den uppstår: Organisationen vill bort från långa förstudier och tunga överlämningar.
 - Vad arkitekten kan göra i stället: Gör behovsanalysen mer löpande, mer hypotesdriven och närmare teamets lärande.

Frågor att ställa i situationen

När du som arkitekt möter en osäker behovsbild kan följande frågor hjälpa:

Frågor om nytta

- Vilken förändring vill vi åstadkomma för medborgare, handläggare eller myndigheten?
- Hur skulle vi märka att första steget skapar nytta?
- Vilken del av nyttan är viktigast att pröva först?

Frågor om antaganden

- Vad bygger vår lösningssidé på för antaganden?
- Vilket antagande är mest riskabelt om det är fel?
- Vilket antagande kan vi testa genom ett begränsat steg?

Frågor om arkitektoniska konsekvenser

- Vilka behov påverkar information, integrationer, säkerhet eller förvaltning?
- Vilka beslut skulle bli svåra att ändra senare?
- Vilka vägval bör vi undvika att låsa för tidigt?

Frågor om avgränsning

- Vilken ärendetyp, målgrupp eller process kan vara ett klokt första avgränsat område?
- Vad ska första inkrementet medvetet inte hantera?
- Vilka risker accepterar vi genom avgränsningen, och vilka minskar vi?

Frågor om ansvar

- Vilka roller behöver vara med i behovsdialogen redan nu?
- Vem kan avgöra om ett antagande är rimligt?
- Vem behöver förstå konsekvenserna av det första steget?

Reflektionsfrågor

1. När har du själv varit med om att ett till synes enkelt behov visade sig ha stora arkitektoniska konsekvenser?
2. Hur brukar du reagera när verksamheten beskriver behov på en mycket övergripande nivå?
3. Vilka typer av oklarheter brukar du vilja reda ut innan teamet börjar bygga?
4. Vilka oklarheter skulle i stället kunna undersökas genom ett begränsat inkrement?
5. Hur kan du hjälpa en produktägare att se skillnaden mellan behov, lösningssidé och begränsning?
6. Vilka antaganden i din nuvarande miljö behandlas som sanningar utan att någon nyligen har provat dem?
7. Hur kan du formulera arkitekturfrågor så att de stödjer lärande snarare än stoppar arbetet?
8. Vad skulle det innebära i din vardag att se behovsanalys som något mer hypotesdrivet?

Snabb sammanfattning

- Osäker behovsbild är inte alltid ett problem som måste lösas före start. I agil utveckling är det ofta en del av arbetet.
- Arkitektens uppgift är att göra osäkerheten begriplig, prövbar och hanterbar.
- All oklarhet är inte likadan. Vissa frågor måste klargöras tidigt, andra kan undersökas genom begränsade steg och vissa kan bevakas tills vidare.
- Lösningssrymd hjälper gruppen att se möjliga vägar utan att låsa en lösning för tidigt.
- Hypotesdriven behovsanalys kopplar behov, antaganden och arkitekturfrågor till lärande.
- Arkitektoniska konsekvenser av behov gör det möjligt att se hur även enkla verksamhetsönskemål kan påverka information, integrationer, säkerhet, ansvar och förvaltning.
- Ett mer agilt arkitektbeteende är inte att släppa behovsanalysen, utan att göra den mer löpande, prövande och kopplad till nästa kloka steg.

Nästa steg

I nästa kapitel går vi vidare från osäkra behov till löpande kravarbete. När behovsbilden inte är färdig från början blir kraven inte heller något som kan betraktas som avslutat innan design och utveckling startar. Det förändrar hur arkitekten behöver samarbeta med produktägare, kravanalytiker, verksamhet och team.

Nästa kapitel handlar därför om situationen: när kravarbete blir löpande snarare än avslutat.

Kapitel 5: När kravarbete blir löpande snarare än avslutat

Varför detta kapitel finns

För många arkitekter är kravarbete en central del av vägen mot en fungerande lösning. Krav hjälper till att avgränsa, prioritera, analysera konsekvenser och skapa underlag för design. Särskilt i en myndighetsmiljö kan krav dessutom behöva bära spårbarhet, rättssäkerhet, informationssäkerhet och ansvar. Det är därför naturligt att arkitekten vill att kravbilden ska vara tillräckligt tydlig innan viktiga arkitekturbeslut fattas.

I ett mer fasorienterat arbetssätt är det vanligt att kravarbete får karaktären av en leverans. Behov analyseras, krav dokumenteras, lösningsförslag tas fram och först därefter startar utvecklingen på allvar. Det ger en känsla av ordning: krav först, design sedan, utveckling därefter.

I agil IT-utveckling förändras detta. Kravarbete upphör inte, men det blir mer löpande. Krav preciseras, omprövas och kompletteras när organisationen lär sig mer. En *backlog*, alltså en prioriterad lista över sådant som kan behöva göras, ersätter ofta den färdiga kravspecifikationen som det huvudsakliga arbetsunderlaget för teamet. Det kan vara både kraftfullt och frustrerande.

För arkitekten uppstår en viktig fråga: hur kan arkitektur och design ta höjd för krav som ännu inte är fullständigt formulerade?

Det här kapitlet handlar om just den situationen. Fokus är inte att överge kravarbete, utan att förändra hur arkitekten samverkar med krav, verksamhet och team när detaljer växer fram över tid.

Situationen: “Det ligger i backloggen, men det är inte färdigkravställt”

På Myndigheten för samhällstjänster fortsätter arbetet med den nya digitala medborgartjänsten. Teamet har börjat bygga en första begränsad vy där medborgaren ska kunna se status för sitt ärende. Erik, produktägaren, har prioriterat ett antal backloggposter som han menar ger snabb nytta:

- Visa aktuell ärendestatus.
- Visa senaste datum för komplettering.
- Ge möjlighet att ladda upp en bilaga.
- Skicka avisering när status ändras.

Sara tittar på backloggen och ser att varje post är kortfattad. Hon saknar flera saker som hon normalt hade förväntat sig i en kravspecifikation:

- Vilka statusvärden får visas för medborgaren?
- Är statusvärdena samma som handläggarna ser internt?
- Ska olika ärendetyper visa olika information?
- Hur ska bilagor viruskontrolleras, klassificeras och kopplas till ärendet?
- Vilka metadata behöver sparas?
- Vilka händelser ska loggas?
- Vilka krav finns på tillgänglighet och svarstider?
- Hur ska aviseringar hanteras om kontaktuppgifter saknas eller är felaktiga?

Lina från teamet säger:

“Vi vet att allt inte är färdigt. Men vi kan börja med den enklaste ärendetypen och lära oss därifrån.”

Erik fyller i:

“Vi vill inte skriva en full kravspecifikation innan vi vet om medborgarna ens förstår statusinformationen.”

Sara förstår poängen. Samtidigt ser hon risken. Om teamet börjar bygga utifrån för tunna krav kan viktiga arkitekturfrågor komma för sent. Om bilagehantering byggs enkelt nu men senare visar sig kräva starkare spårbarhet, informationsklassning och arkivering kan det bli dyrt att rätta. Om statusinformationen bygger direkt på interna koder i det gamla systemet kan tjänsten bli svår att förändra.

Det invanda sättet att agera vore att be om mer kravarbete innan teamet fortsätter. Men i den här situationen är frågan inte om kraven ska bli tydligare. Det ska de. Frågan är hur tydliggörandet ska ske utan att arbetet förvandlas till en ny kravfas vid sidan av utvecklingen.

Det invanda sättet att agera

När kravbilden är ofullständig är det lätt att arkitekten försöker återskapa den trygghet som en komplett kravspecifikation brukar ge. Det kan visa sig på flera sätt.

Arkitekten kan be om att kraven “måste bli klara” innan designen kan fortsätta. Det kan vara rimligt när viktiga rättsliga, säkerhetsmässiga eller tekniska förutsättningar saknas. Men det kan också bli ett sätt att skjuta lärandet framför sig.

Arkitekten kan börja fylla i luckorna själv. Det är vanligt att erfarna arkitekter snabbt ser vad som saknas och därför börjar formulera antaganden om informationsmodell, integrationsmönster, behörighet, loggning och förvaltningsbarhet. Det kan hjälpa arbetet framåt, men det kan också leda till att arkitekten i praktiken tar över kravtolkningen från verksamheten.

Arkitekten kan också försöka skapa en separat lista över arkitekturkrav vid sidan av teamets backlogg. Den listan kan innehålla viktiga saker: prestanda, säkerhet, tillgänglighet, spårbarhet, integrationer och tekniska begränsningar. Problemet uppstår om listan inte kopplas till teamets prioritering och vardagliga arbete. Då riskerar arkitekturkraven att bli något som upptäcks vid granskning, snarare än något som påverkar lösningen under arbetets gång.

Bakom dessa beteenden finns ofta en legitim oro: att viktiga egenskaper ska tappas bort när kraven blir kortare, mer rörliga och mer verksamhetsnära. Den oron ska tas på allvar. Men svaret är inte alltid att återgå till mer omfattande kravdokument. Ofta behöver arkitekten i stället hjälpa organisationen att göra rätt krav tydliga vid rätt tidpunkt.

Varför det kan skapa friktion i agil utveckling

När arkitekten kräver en mer komplett kravbild innan teamet går vidare kan det skapa friktion på flera nivåer.

För det första kan det bromsa återkoppling. Om organisationen inte vet hur medborgare tolkar statusinformationen kan en lång kravfas ge skenbar säkerhet. Den beskriver vad man tror behövs, men prövar inte om det faktiskt fungerar. I sådana lägen kan en enkel implementation, användartestning eller genomgång med handläggare ge bättre kunskap än ytterligare dokumentation.

För det andra kan det skapa parallella verkligheter. Teamet arbetar i backloggen, produktägaren prioriterar utifrån nytta och arkitekten för en separat diskussion om kravkvalitet. Då uppstår lätt glapp. Det som arkitekten ser som nödvändiga lösningsförutsättningar kan teamet uppfatta som senare granskning eller extraarbete. Det som produktägaren ser som snabb nytta kan arkitekten uppfatta som otillräckligt analyserat.

För det tredje kan kravens form bli viktigare än deras funktion. En välskriven kravspecifikation kan vara värdefull, men bara om den hjälper människor att fatta bättre beslut. Ett kort backloggobjekt kan vara otillräckligt, men det kan också vara fullt tillräckligt om teamet, produktägaren och arkitekten har en gemensam förståelse för vad som ska läras, vilka begränsningar som gäller och vilka beslut som ännu är öppna.

För det fjärde kan arkitekturfrågor komma in för sent om arkitekten väntar på "rätt" kravunderlag. I agil utveckling formas många krav i samtal, refinement, demonstrationer och återkoppling. *Refinement* betyder här att teamet och berörda roller stegvis förtydligar och delar upp arbetet så att det blir begripligt och möjligt att genomföra. Om arkitekten inte deltar i dessa samtal riskerar viktiga arkitekturfrågor att dyka upp först när teamet redan har valt riktning.

Friktionen beror alltså inte på att kravarbete har blivit oviktigt. Den beror på att kravarbete har flyttat närmare utvecklingsflödet, och att arkitekten behöver följa med dit.

Ett mer agilt förhållningssätt

Ett mer agilt förhållningssätt till kravarbete börjar med en enkel insikt: alla krav behöver inte vara lika detaljerade samtidigt.

Vissa krav behöver vara tydliga tidigt, eftersom de påverkar lösningens grundstruktur. Det kan handla om informationssäkerhet, rättsliga begränsningar, centrala informationsobjekt, integrationsriktning, behörighetsmodell, spårbarhet eller krav på arkivering. Om dessa frågor hanteras för sent kan de påverka stora delar av lösningen.

Andra krav kan förfinas stegvis. Det kan handla om exakt formulering av texter, ordning på fält, detaljer i användarflöden eller vilka statusmeddelanden som ger bäst förståelse för medborgaren. Dessa krav kan ofta bli bättre genom prototyper, användaråterkoppling och successiv utveckling.

Arkitektens uppgift blir att hjälpa organisationen att skilja mellan dessa kravtyper.

Löpande kravförfining

Löpande kravförfining innebär att krav inte ses som färdiga en gång för alla. De förtydligas när ny kunskap uppstår. Det betyder inte att allt är flytande eller informellt. Det betyder att kravens detaljnivå anpassas efter risk, närhet till implementation och beslutens konsekvens.

För Sara innebär det att hon inte behöver kräva en komplett kravspecifikation för hela medborgartjänsten. Däremot behöver hon vara tydlig med vilka frågor som måste upp på bordet innan teamet bygger vissa delar.

Hon kan till exempel säga:

“För att börja visa ärendestatus för en begränsad ärendetyp behöver vi inte ha hela tjänsten färdigkravställd. Men vi behöver förstå vilka statusvärden som får visas externt, om de behöver översättas från interna begrepp och vilken spårbarhet som krävs när status visas för medborgaren.”

Det är en annan typ av kravdialog. Den frågar inte efter allt. Den frågar efter det som behövs för nästa meningsfulla beslut.

Icke-funktionella krav som kontinuerlig dialog

Icke-funktionella krav är krav på egenskaper hos lösningen, till exempel säkerhet, prestanda, tillgänglighet, robusthet, användbarhet, spårbarhet och förvaltningsbarhet. I en traditionell kravspecifikation får de ofta ett eget avsnitt. I agil utveckling behöver de också finnas i samtalen där arbetet planeras, delas upp och genomförs.

Det är särskilt viktigt i myndighetsmiljö. Krav på rättssäkerhet, informationssäkerhet, tillgänglighet och spårbarhet är inte detaljer som kan läggas till i slutet. De påverkar designen.

Samtidigt fungerar det sällan bra att formulera alla icke-funktionella krav som generella kravsatser från början. “Systemet ska vara säkert” eller “lösningen ska vara förvaltningsbar” hjälper inte teamet särskilt mycket. Arkitektens bidrag är att göra sådana krav situationsspecifika.

För bilageuppladdning kan Sara till exempel hjälpa teamet att ställa frågor som:

- Vilka filtyper ska tillåtas?
- Hur hanteras skadlig kod?
- Vilken information får bilagan innehålla?
- När blir bilagan en allmän handling?
- Hur kopplas bilagan till rätt ärende och rätt person?
- Vilken loggning behövs för att kunna följa vad som hänt?
- Hur ska fel hanteras om uppladdningen avbryts?

På så sätt blir icke-funktionella krav inte en lista vid sidan av. De blir en del av designen.

Krav som beslutsunderlag

I ett fasorienterat arbetssätt kan krav ibland uppfattas som beställning: verksamheten beskriver vad som ska byggas, IT tar fram lösningen. I ett mer agilt arbetssätt behöver krav ofta fungera mer som beslutsunderlag.

Det innebär att krav hjälper team, produktägare och arkitekt att förstå vilka avvägningar som behöver göras. Ett krav kan visa att det finns en konflikt mellan snabb leverans och långsiktig förvaltningsbarhet. Ett annat krav kan visa att en till synes enkel funktion egentligen kräver informationsklassning, juridisk tolkning eller förändring i ett äldre system.

När Sara ser backloggposten "Skicka avisering när status ändras" kan hon behandla den som en beställning: bygg avisering. Men hon kan också använda den som beslutsunderlag:

- Vilka statusändringar är viktiga för medborgaren?
- Får alla ärendetyper aviseras på samma sätt?
- Vilka kontaktvägar är tillåtna?
- Vad får stå i aviseringen?
- Behöver medborgaren samtycka till vissa typer av avisering?
- Vad händer om aviseringen misslyckas?
- Är aviseringen en del av myndighetens formella kommunikation eller bara en servicefunktion?

Dessa frågor behöver inte alla besvaras innan något arbete får börja. Men de behöver synliggöras så att teamet inte bygger in antaganden som senare visar sig fel.

Exempel: Myndigheten för samhällstjänster

Sara föreslår ett arbetssätt för de kommande veckorna. Hon vill inte stoppa teamets första inkrement, men hon vill göra kravförfiningen mer arkitektoniskt medveten.

Hon ber Erik och Lina att tillsammans med henne gå igenom de högst prioriterade backloggposterna. För varje post ställer de tre frågor:

1. Vilken nytta eller vilket lärande ska denna post ge?
2. Vilka arkitektoniska eller myndighetsnära risker kan den beröra?
3. Vilka frågor måste vara besvarade innan teamet bygger just denna del?

När de går igenom "Visa aktuell ärendestatus" kommer de fram till att teamet kan börja med en begränsad ärendetyp. Men innan implementation behöver de veta om interna statuskoder kan visas direkt eller om de måste översättas till medborgarvänliga statusbegrepp. De behöver också veta om vissa statuslägen inte får visas på grund av sekretess, pågående handläggning eller risk för missförstånd.

När de går igenom "Ladda upp bilaga" blir gruppen mer försiktig. Här finns fler frågor om informationssäkerhet, handlingar, lagring, viruskontroll och koppling till ärendet. Sara föreslår därför att teamet inte börjar med full bilageuppladdning

direkt. I stället kan de först undersöka flödet, informationsobjekten och säkerhetskraven. Det kan göras genom en begränsad teknisk och verksamhetsmässig genomgång, kanske tillsammans med Amir från informationssäkerhet.

När de går igenom "Skicka avisering" ser de att nyttan är tydlig, men att det finns flera oklara regel- och kommunikationsfrågor. Erik tar med sig att verksamheten behöver avgöra vilka aviseringar som är serviceinformation och vilka som kan påverka formell kommunikation med medborgaren.

Resultatet blir inte en komplett kravspecifikation. Men det blir en tydligare backlogg, där vissa poster kan gå vidare, vissa behöver delas upp och vissa behöver kompletteras med arkitektur- eller riskfrågor.

Sara dokumenterar inte allt som ett stort kravdokument. I stället kompletterar hon varje berörd backloggpost med korta arkitekturkommentarer, öppna frågor och beslut som behöver fattas. Hon skapar också en enkel lista över återkommande icke-funktionella krav som teamet ska använda vid refinement:

- informationsklassning,
- behörighet,
- loggning,
- spårbarhet,
- felhantering,
- tillgänglighet,
- förvaltningsbarhet,
- integrationspåverkan.

Det viktiga är att listan används i samtalen. Den blir inte en bilaga som någon läser senare, utan ett stöd för löpande kravförfining.

Vanliga fallgropar

- Fallgrop: Att kräva färdiga krav innan arkitekten kan bidra.
 - Varför den uppstår: Arkitekten är van vid att krav är den stabila grund som designen vilar på.
 - Vad arkitekten kan göra i stället: Identifiera vilka krav som behöver vara tydliga nu och vilka som kan förfinas genom lärande.
- Fallgrop: Att acceptera för tunna backloggposter utan arkitekturdialog.
 - Varför den uppstår: Arkitekten vill inte bromsa teamet eller uppfattas som fasoorienterad.
 - Vad arkitekten kan göra i stället: Delta i refinement och ställa frågor som synliggör risker, beroenden och kvalitetskrav.
- Fallgrop: Att lägga icke-funktionella krav i en separat lista som inte påverkar arbetet.
 - Varför den uppstår: Det känns ordnat att samla sådana krav på ett ställe.
 - Vad arkitekten kan göra i stället: Koppla kraven till konkreta backloggposter, designbeslut och acceptanskriterier där det är relevant.
- Fallgrop: Att arkitekten själv fyller i verksamhetens kravluckor.
 - Varför den uppstår: Erfarna arkitekter ser snabbt konsekvenser och vill hjälpa arbetet framåt.
 - Vad arkitekten kan göra i stället: Formulera antaganden öppet och be produktägare, verksamhet eller specialister bekräfta, avfärda eller undersöka dem.

- Fallgrop: Att behandla alla krav som lika viktiga att precisera tidigt.
 - Varför den uppstår: Det är svårt att veta vilka krav som påverkar grundstrukturen.
 - Vad arkitekten kan göra i stället: Sortera krav efter risk, reversibilitet, påverkan på arkitektur och närhet till implementation.

Frågor att ställa i situationen

När kravarbete blir löpande kan arkitekten använda frågor som hjälper gruppen att komma vidare utan att låsa för mycket.

Frågor om kravets funktion

- Vad behöver vi förstå för att kunna ta nästa steg?
- Ska detta krav ge användarnytta, minska risk, skapa lärande eller möjliggöra senare arbete?
- Vilket beslut ska kravet hjälpa oss att fatta?
- Vad händer om vi missförstår detta krav?

Frågor om arkitektonisk påverkan

- Påverkar kravet informationsmodell, integrationer, behörighet, loggning eller lagring?
- Påverkar kravet hur lösningen ska förvaltas?
- Skapar kravet beroenden till andra system, team, myndigheter eller leverantörer?
- Är detta ett krav som kan ändras lätt senare, eller påverkar det svårreversibla beslut?

Frågor om myndighetskontext

- Finns krav på rättssäkerhet, spårbarhet eller likabehandling?
- Berör kravet personuppgifter, sekretess eller informationsklassning?
- Behöver juridik, informationssäkerhet eller arkivfunktion involveras?
- Behöver beslutet dokumenteras för att kunna förstås i efterhand?

Frågor om arbetsform

- Behöver kravet delas upp?
- Behöver det kompletteras med acceptanskriterier?
- Behöver vi en kort utforskning innan vi bygger?
- Vem behöver delta i nästa refinement för att kravet ska bli tillräckligt tydligt?

Reflektionsfrågor

1. Vilka typer av krav brukar du själv vilja ha tydliga innan du känner dig trygg med arkitekturen?
2. När har du sett att ett krav blev tydligare först efter att teamet började bygga eller visa något?

3. Vilka icke-funktionella krav riskerar oftast att komma in för sent i din organisation?
4. Hur brukar du som arkitekt delta i refinement eller motsvarande kravdialog i dag?
5. Vilka kravluckor brukar du fylla i själv, snarare än att synliggöra dem som antaganden?
6. Hur kan du hjälpa produktägare och team att se arkitektoniska konsekvenser utan att göra kravdialogen tung?
7. Vilka krav behöver dokumenteras mer formellt i din miljö, och vilka kan hanteras genom levande dialog och tydliga beslut?
8. Vad skulle vara ett litet första steg mot mer löpande kravförfining i ditt nuvarande sammanhang?

Snabb sammanfattning

När kravarbete blir löpande försvinner inte behovet av tydlighet. Det förändras.

Arkitekten behöver inte kräva att alla krav är färdiga innan utveckling kan börja. Däremot behöver arkitekten hjälpa organisationen att förstå vilka krav som påverkar arkitekturbeslut, vilka som kan förfinas stegvis och vilka som måste kopplas till säkerhet, spårbarhet, förvaltning och myndighetsansvar.

Ett mer agilt förhållningssätt innebär att krav, arkitektur och design utvecklas i samspel. Backloggen blir inte bara en lista över funktioner, utan ett ställe där nytta, risk, lärande och beslut möts. Arkitektens bidrag är att göra de osynliga konsekvenserna synliga i tid.

Det centrala skiftet är:

Från: "Kraven ska vara tillräckligt kompletta för att arkitekturen ska kunna tas fram."

Till: "Arkitektur- och kravarbete behöver löpande hjälpa varandra att bli tydligare."

Nästa steg

I nästa kapitel flyttas fokus från kravens detaljnivå till startpunkten för själva utvecklingen. Vad händer när teamet vill börja bygga innan arkitekturen känns klar? Då behöver arkitekten kunna avgöra vilka arkitektoniska startvillkor som faktiskt krävs, vilka frågor som kan undersökas genom implementation och hur "tillräcklig arkitektur" kan fungera utan att bli en ny fasgrind.

Kapitel 6: När teamet vill börja bygga innan arkitekturen är klar

Varför detta kapitel finns

En av de vanligaste friktionerna i övergången från ett mer fasorienterat arbetssätt till agil IT-utveckling uppstår när teamet vill börja bygga innan arkitekten upplever att arkitekturen är klar. Situationen är lätt att känna igen: produktägaren vill få fram något konkret, teamet vill minska osäkerhet genom att börja implementera och arkitekten ser fortfarande öppna frågor kring integrationer, informationsmodell, säkerhet, behörighet, drift, förvaltning och beroenden.

I ett fasorienterat arbetssätt är det ofta rimligt att utveckling startar först när centrala lösningsfrågor är utredda och dokumenterade. Det ger en tydlig övergång från analys och design till byggande. I ett mer agilt arbetssätt kan samma logik däremot skapa väntan. Om allt måste vara färdigtänkt innan något får byggas riskerar organisationen att skjuta upp lärande, upptäcka fel antaganden sent och skapa stora beslutsunderlag för frågor som egentligen kunde ha prövats i mindre skala.

Men motsatsen är inte heller hållbar. Att "bara börja bygga" utan arkitektonisk riktning kan skapa teknikskuld, svåra beroenden och lösningar som fungerar lokalt men blir problematiska i helheten. För en myndighet kan det dessutom leda till brister i spårbarhet, informationssäkerhet, rättssäkerhet och förvaltningsbarhet.

Det här kapitlet handlar därför om ett praktiskt mellanläge: **tillräcklig arkitektur**. Det betyder inte minimal arkitektur eller frånvaro av styrning. Det betyder att arkitekten hjälper teamet att förstå vilka beslut och förutsättningar som behöver vara tydliga för att utveckling ska kunna börja på ett kontrollerat sätt, och vilka frågor som kan undersökas genom stegvis implementation.

Kapitlets kärnfråga är:

Vad behöver vara arkitektoniskt klart för att teamet ska kunna börja lära sig utan att skapa oacceptabel risk?

Situationen: "Vi behöver börja bygga nu"

På Myndigheten för samhällstjänster har arbetet med den nya medborgartjänsten kommit till en punkt där teamet vill skapa en första fungerande version. Målet är

begränsat: en medborgare ska kunna logga in och se grundläggande status för ett ärende.

Erik, produktägaren, vill komma vidare:

“Vi har pratat länge nog. Vi behöver få upp något som verksamheten och några användare kan reagera på.”

Lina, teamets utvecklingsledare, håller med:

“Vi kan börja med en smal version. En ärendetyp, ett fåtal statusvärden, bara läsning. Då lär vi oss hur integrationen fungerar och vad som är svårt.”

Sara ser nyttan med detta. Samtidigt ser hon flera öppna arkitekturfrågor:

- Vilket system ska vara källa för ärendestatus?
- Ska tjänsten läsa direkt från handläggningssystemet eller via ett integrationslager?
- Vilka statusvärden får visas externt?
- Hur ska behörighet och identitet hanteras?
- Vilken information får cachas?
- Vilka händelser behöver loggas?
- Hur påverkas förvaltning och support?
- Hur ska lösningen fungera om handläggningssystemet är otillgängligt?
- Vilka designval riskerar att bli svåra att ändra senare?

I en tidigare projektlogik hade Sara sannolikt velat ta fram en mer komplett lösningsbeskrivning innan teamet började implementera. Hon hade velat analysera integrationsmönster, informationsflöden, säkerhetskrav och målarkitektur. Det är inte orimligt. Frågorna är viktiga.

Men nu finns också en annan risk: om allt ska utredas först kan teamet tappa tempo, verksamheten tappa engagemang och flera antaganden förbli oprövade. Det är inte säkert att en mer omfattande förstudie skulle ge bättre svar. Vissa svar kanske bara går att få genom att bygga något begränsat och se vad som händer.

Sara behöver därför inte välja mellan fullständig arkitektur och att släppa taget. Hon behöver formulera vad som är **tillräckligt för att börja**.

Det invanda sättet att agera

När arkitekten möter ett team som vill börja bygga innan arkitekturen är klar är det invanda agerandet ofta att bromsa. Inte av ovilja, utan av ansvarskänsla.

Arkitekten kan tänka:

- “Vi vet inte tillräckligt om helheten.”
- “Om teamet bygger fel nu blir det dyrt att rätta senare.”
- “Vi behöver säkerställa målarkitekturen först.”
- “Vi måste förstå alla beroenden innan implementationen startar.”
- “Det här kommer att bli en punktlösning om vi inte håller ihop designen.”

Det ligger mycket klokskap i dessa reaktioner. Arkitekten har ofta sett vad som händer när team bygger lokalt fungerande lösningar som senare visar sig skapa problem i integration, drift, säkerhet eller förvaltning. I en myndighetsmiljö kan

konsekvenserna bli större än enbart tekniskt omarbete. Felaktiga vägval kan påverka informationshantering, medborgarnas förtroende, rättssäkerhet eller möjligheten att följa upp beslut.

Det invanda sättet att skapa trygghet blir därför att kräva mer utredning innan byggstart. Arkitekten försöker reducera osäkerhet genom analys, dokumentation, granskning och förankring.

Det är ofta rätt när besluten är svårreversibla och riskerna stora. Men det kan bli problematiskt när samma arbetssätt används för alla frågor, även sådana som är mindre riskfyllda, reversibla eller möjliga att undersöka i liten skala.

Varför det kan skapa friktion i agil utveckling

I agil utveckling är byggande inte bara produktion. Det är också ett sätt att lära. En begränsad implementation kan visa om en integration fungerar, om informationen är begriplig, om ett API är tillräckligt, om användarna förstår begreppen eller om en teknisk lösning håller.

När arkitekten kräver att arkitekturen ska vara "klar" innan något byggs kan flera typer av friktion uppstå.

För det första kan arkitekturbegreppet bli för stort. Allt som är oklart hamnar under rubriken "arkitektur", och därmed blir nästan ingenting möjligt att börja med. Frågor som faktiskt måste avgöras tidigt blandas med frågor som kan utforskas senare.

För det andra kan teamets lärande försenas. Vissa problem upptäcks först när någon försöker implementera, integrera eller testa. Om teamet väntar på fullständig klarhet kan organisationen lägga mycket tid på antaganden som ändå behöver ändras.

För det tredje kan arkitekten bli en fasgrind, även om organisationen säger sig arbeta agilt. Teamet lär sig då att utveckling startar när arkitekten har godkänt lösningen, snarare än när riskerna är tillräckligt förstådda och arbetet är upplagt för kontrollerat lärande.

För det fjärde kan dialogen mellan arkitekt och team försämrats. Om arkitekten uppfattas som den som säger nej kan teamet börja undvika arkitekturdialogen eller komma med färdiga lösningar för sent. Det minskar arkitektens möjlighet att påverka när det verkligen behövs.

Det agila alternativet är inte att börja utan riktning. Det är att skilja mellan tre frågor:

1. Vad måste vi veta innan vi börjar?
2. Vad kan vi lära oss genom att börja begränsat?
3. Vad får vi absolut inte råka låsa utan medvetet beslut?

Ett mer agilt förhållningssätt

Ett mer agilt förhållningssätt innebär att arkitekten hjälper teamet att skapa en trygg startpunkt, inte en färdig slutbild. Det kan göras genom tre begrepp: **till-**

räcklig arkitektur, arkitektoniska startvillkor och utforskande implementation.

Tillräcklig arkitektur

Tillräcklig arkitektur är den nivå av arkitektonisk riktning som behövs för att utveckling ska kunna starta utan oacceptabel risk.

Det innebär att arkitekten inte frågar:

Är arkitekturen klar?

Utan snarare:

Är arkitekturen tillräckligt tydlig för nästa steg?

Det är en viktig skillnad. "Klar" antyder ett slutläge. "Tillräcklig för nästa steg" kopplar arkitekturarbetet till det som faktiskt ska göras nu.

För Saras team innebär det att första versionen inte behöver ha hela framtida medborgartjänstens arkitektur färdig. Men teamet behöver förstå vissa gränser:

- vilken information de får visa,
- vilken systemkälla de får använda,
- hur identitet och behörighet ska hanteras i första steget,
- vilka antaganden som inte får byggas in permanent,
- vilka beslut som måste dokumenteras,
- vilka risker som behöver följas upp innan nästa steg.

Tillräcklig arkitektur är alltså inte en ursäkt för otydlighet. Det är ett sätt att vara tydlig på rätt nivå.

Arkitektoniska startvillkor

Arkitektoniska startvillkor är de minsta förutsättningar som behöver vara på plats för att teamet ska kunna börja arbeta ansvarsfullt.

De kan exempelvis handla om:

- vilka system eller datakällor som får användas,
- vilka säkerhetsprinciper som gäller,
- vilka integrationer som får vara tillfälliga,
- vilka delar som måste vara spårbara från början,
- vilka beslut som kräver särskild förankring,
- vilka begränsningar teamet inte får överskrida,
- vilka frågor som ska undersökas i första inkrementet.

Bra startvillkor är konkreta, begränsade och möjliga att använda. De ska inte vara en förtäckt kravspecifikation eller en ny fasgrind. De ska hjälpa teamet att komma i gång på ett sätt som bevarar handlingsutrymme.

Ett exempel på startvillkor för Saras team kan vara:

- Första versionen får endast läsa information, inte ändra ärendedata.
- Endast en ärendetyp ingår.
- Informationen hämtas via en avgränsad integrationslösning, inte genom direkt åtkomst till handläggningssystemets databas.

- Endast statusvärden som verksamhet och juridik har bedömt som lämpliga får visas externt.
- Alla arkitekturbeslut och öppna antaganden dokumenteras i en enkel beslutslogg.
- Teamet ska efter första inkrementet återkomma med lärdomar om integration, datakvalitet och användarförståelse.

Detta är inte hela arkitekturen. Men det kan vara tillräckligt för att börja.

Utforskande implementation

En utforskande implementation är en begränsad implementation som görs för att lära, inte för att låtsas att hela lösningen redan är färdig.

Det kan vara en prototyp, en teknisk spike, en smal första version eller ett kontrollerat inkrement. Det viktiga är att syftet är tydligt:

- Vilket antagande vill vi testa?
- Vilken risk vill vi minska?
- Vilket beslut vill vi kunna fatta bättre efteråt?
- Vad får implementationen inte bli utan nytt beslut?

I Saras fall kan teamet använda en utforskande implementation för att pröva hur ärendestatus kan hämtas och visas. Men då behöver alla vara överens om vad som är utforskande och vad som är långsiktigt vägval.

Det är här arkitektens roll är central. Sara kan hjälpa teamet att undvika att en tillfällig lösning smyger sig in som permanent arkitektur. Hon kan också hjälpa produktägaren att se att ett första inkrement inte bara levererar funktion, utan även kunskap.

Exempel: Myndigheten för samhällstjänster

Sara bokat ett kort arbetsmöte med Erik, Lina och Amir från informationssäkerhet. Syftet är inte att granska en färdig lösning, utan att formulera startvillkor för första inkrementet.

Hon inleder:

“Jag tror att teamet kan börja bygga en smal version. Men vi behöver vara tydliga med vad vi vet, vad vi antar och vad vi absolut inte ska låsa i första steget.”

De delar upp tavlan i fyra rubriker:

1. Beslut som måste tas före byggstart.
2. Antaganden som får testas.
3. Begränsningar som inte får överskridas.
4. Frågor som ska följas upp efter första inkrementet.

Under första rubriken skriver de:

- Tjänsten ska endast visa information, inte ändra ärenden.
- Endast medborgare med säker inloggning får se ärendestatus.
- Första inkrementet gäller en avgränsad ärendetyp.
- Direkt databasåtkomst till handläggningssystemet är inte tillåten.

- Loggning av åtkomst ska finnas från början.

Under antaganden skriver de:

- De befintliga statusvärdena går att översätta till begripligt medborgarspråk.
- Handläggningssystemet kan leverera statusinformation med tillräcklig aktualitet.
- En enkel integrationslösning räcker för första inkrementet.

Under begränsningar skriver de:

- Ingen generell integrationsplattform väljs i detta steg.
- Ingen permanent informationsmodell fastställs för alla ärendetyper.
- Ingen lösning för aviseringar byggs innan rättsliga och informationsmässiga frågor är tydligare.
- Ingen statusinformation visas externt utan verksamhetsmässig och juridisk bedömning.

Under uppföljning skriver de:

- Vad lärde vi oss om datakvalitet?
- Vilka statusvärden var svåra att förstå?
- Vilka integrationsproblem uppstod?
- Vilka beslut behöver fattas inför nästa inkrement?
- Vilka delar av lösningen kan behållas, och vilka var endast utforskande?

Efter mötet säger Lina:

“Det här hjälper. Nu vet teamet vad vi får göra, vad vi inte ska röra och vad vi ska lära oss.”

Erik säger:

“Och jag kan förklara för styrgruppen varför vi börjar smalt utan att låtsas att hela lösningen är beslutad.”

Sara har inte skapat en fullständig arkitekturbeskrivning. Men hon har skapat riktning, gränser och ett sätt att lära utan att tappa kontrollen. Det är tillräcklig arkitektur i praktiken.

Att skilja mellan startbeslut och framtidsbeslut

En viktig färdighet i den här situationen är att skilja mellan beslut som behövs för att starta och beslut som hör hemma senare.

Startbeslut handlar om att göra nästa steg möjligt och säkert nog. Framtidsbeslut handlar om hur hela lösningen ska utvecklas över tid. Problemet uppstår när framtidsbeslut krävs innan teamet ens har börjat lära sig.

För arkitekten kan det vara hjälpsamt att dela in öppna frågor i fyra kategorier.

1. Måste beslutas före start

Detta är frågor där felaktigt eller uteblivet beslut kan skapa oacceptabel risk direkt. Det kan handla om säkerhet, åtkomst, dataskydd, systemgränser, rättsliga begränsningar eller svårreversibla tekniska vägval.

Exempel:

- Får den här informationen visas för medborgaren?
- Vilken identitets- och behörighetsprincip gäller?
- Vilka systemgränser får teamet inte bryta?
- Finns det ett vägval som blir mycket dyrt att ändra senare?

2. Kan beslutas efter utforskning

Detta är frågor där organisationen behöver mer kunskap innan ett bra beslut kan fattas. Här kan en begränsad implementation ge bättre underlag än ytterligare analys.

Exempel:

- Vilket integrationsmönster fungerar bäst i praktiken?
- Hur behöver statusvärden översättas för att vara begripliga?
- Vilken svarstid upplever användare som acceptabel?
- Hur ofta behöver informationen uppdateras?

3. Kan vara tillfälligt med tydlig märkning

Vissa lösningar kan accepteras tillfälligt om de dokumenteras och avgränsas. Det farliga är inte alltid att göra något tillfälligt. Det farliga är att det tillfälliga blir permanent utan beslut.

Exempel:

- En enkel integrationslösning för första inkrementet.
- Manuell hantering av en viss kontroll under kort tid.
- Begränsat stöd för endast en ärendetyp.
- En förenklad informationsmodell för att testa begrepp.

4. Ska inte beslutas ännu

Vissa beslut bör uttryckligen skjutas upp. Det kan kännas ovant för arkitekten, men är ibland det mest ansvarsfulla. Ett beslut som fattas för tidigt kan minska handlingsutrymmet utan att minska risken.

Exempel:

- Slutlig målarkitektur för alla medborgartjänster.
- Val av generell integrationsstrategi för hela myndigheten.
- Permanent modell för alla ärendetyper.
- Fullständig framtida förvaltningsorganisation för en lösning som ännu inte är validerad.

Den här indelningen gör arkitektens resonemang mer konkret. I stället för att säga "arkitekturen är inte klar" kan Sara säga:

"De här tre besluten behöver vi fatta före start. De här fyra frågorna kan teamet undersöka. Den här tillfälliga lösningen är acceptabel i ett inkrement om vi märker det tydligt. Och de här två besluten bör vi inte låsa ännu."

Det är ett mer användbart bidrag än ett generellt ja eller nej.

Tillräcklig arkitektur är inte samma sak som svag arkitektur

En vanlig missuppfattning är att tillräcklig arkitektur betyder mindre arkitektur. Det stämmer inte. I många fall kräver tillräcklig arkitektur mer professionellt omdöme än en omfattande förhandsdesign.

En omfattande förhandsdesign kan ibland dölja osäkerhet bakom detaljerade beskrivningar. Tillräcklig arkitektur behöver i stället göra osäkerheten synlig:

- Vad vet vi?
- Vad antar vi?
- Vad behöver vi lära?
- Vilka risker accepterar vi?
- Vilka risker accepterar vi inte?
- Vilket handlingsutrymme vill vi bevara?

Det kräver att arkitekten är tydlig, inte vag. Det kräver också att arkitekten vågar säga både ja och nej.

Ett agilt arkitekturbidrag kan därför låta så här:

“Ja, teamet kan börja bygga en första läsande vy. Nej, vi ska inte välja permanent integrationsmönster för alla framtida tjänster i det här steget. Ja, vi ska dokumentera vilka antaganden vi testar. Nej, vi får inte visa statusvärden externt innan verksamhet och juridik har bedömt dem.”

Det är varken full kontroll i förväg eller fri byggstart utan gränser. Det är kontrollerat lärande.

När arkitekten bör bromsa

Det finns situationer där arkitekten faktiskt bör bromsa byggstart. Att arbeta agilt betyder inte att alla risker ska tas genom implementation.

Arkitekten bör vara tydlig när:

- teamet riskerar att hantera skyddsvärd information utan tillräcklig kontroll,
- ett tekniskt vägval är svårreversibelt och underlaget är för svagt,
- en tillfällig lösning sannolikt kommer att bli permanent utan styrning,
- teamet saknar förståelse för viktiga beroenden,
- lösningen kan påverka rättssäkerhet, integritet eller myndighetens ansvar,
- det saknas mandat för ett beslut som teamet är på väg att fatta,
- ingen har formulerat vad implementationen ska lära organisationen.

Skillnaden ligger i hur arkitekten bromsar. Ett fasorienterat bromsande kan bli:

“Ni kan inte börja förrän arkitekturen är klar.”

Ett mer agilt bromsande kan bli:

“Ni kan börja med vissa delar, men inte med den här delen innan vi har hanterat de här riskerna.”

Eller:

“Jag föreslår att vi gör första inkrementet smalare så att teamet kan börja utan att vi låser ett svårreversibelt vägval.”

Det andra sättet skyddar fortfarande helheten, men försöker samtidigt bevara flöde och lärande.

Frågor att ställa i situationen

När ett team vill börja bygga innan arkitekturen är klar kan arkitekten använda följande frågor.

Om nästa steg

- Vad är det minsta meningsfulla nästa steget?
- Är syftet att leverera funktion, minska risk, testa ett antagande eller skapa beslutsunderlag?
- Vilka delar behöver fungera på riktigt redan nu?
- Vilka delar kan vara avgränsade, tillfälliga eller manuella?

Om risk

- Vilka beslut i detta steg är svårreversibla?
- Vilka risker är acceptabla för ett begränsat inkrement?
- Vilka risker är inte acceptabla ens i liten skala?
- Vad kan gå fel om teamet börjar utan ytterligare arkitekturarbete?

Om gränser

- Vilka systemgränser får teamet inte bryta?
- Vilka data får användas?
- Vilka integrationer får teamet skapa eller anropa?
- Vilka säkerhets- eller regelefterlevnadskrav gäller från första dagen?

Om lärande

- Vad ska vi veta efter första inkrementet som vi inte vet i dag?
- Hur ska vi dokumentera lärdomarna?
- När ska vi ompröva de tillfälliga vägvalen?
- Vilka beslut ska första inkrementet hjälpa oss att fatta bättre?

Om ansvar

- Vem har mandat att acceptera riskerna?
- Vem behöver vara med i dialogen innan teamet startar?
- Hur kommuniceras startvillkoren till team, produktägare och styrning?
- Hur säkerställer vi att tillfälliga lösningar inte blir permanenta av misstag?

Vanliga fallgropar

- Fallgrop: Att kräva fullständig arkitektur före varje byggstart.

- Varför den uppstår: Arkitekten vill minska risk och skapa trygghet genom tydliga underlag.
- Vad arkitekten kan göra i stället: Skilj mellan beslut som måste tas före start och frågor som kan undersökas genom ett begränsat inkrement.
- Fallgrop: Att låta teamet börja utan tydliga gränser.
 - Varför den uppstår: Organisationen vill undvika att arkitektur uppfattas som bromsande.
 - Vad arkitekten kan göra i stället: Formulera arkitektoniska startvillkor som ger teamet frihet inom tydliga ramar.
- Fallgrop: Att kalla allt för "tillfälligt" utan uppföljning.
 - Varför den uppstår: Tillfälliga lösningar gör det lättare att komma i gång.
 - Vad arkitekten kan göra i stället: Märk tillfälliga vägval, dokumentera varför de görs och bestäm när de ska omprövas.
- Fallgrop: Att blanda ihop prototyp, spike och produktionslösning.
 - Varför den uppstår: Teamet bygger något som fungerar, och organisationen börjar använda det.
 - Vad arkitekten kan göra i stället: Var tydlig med vad som är utforskande implementation och vad som är avsett att leva vidare.
- Fallgrop: Att arkitekten säger nej för sent.
 - Varför den uppstår: Arkitekten involveras först när teamet redan har börjat implementera.
 - Vad arkitekten kan göra i stället: Var närvarande tidigt i planering och refinement så att risker och startvillkor formuleras innan teamet investerar för mycket.

Reflektionsfrågor

1. När har du själv upplevt att ett team ville börja bygga innan du tyckte att arkitekturen var tillräckligt klar?
2. Vad var det egentligen du saknade: beslut, analys, dokumentation, förankring, riskhantering eller egen trygghet?
3. Vilka arkitekturfrågor i din miljö måste vara besvarade före byggstart?
4. Vilka frågor skulle ni kunna lära er mer om genom en begränsad implementation?
5. Hur tydligt brukar ni skilja mellan tillfälliga vägval och långsiktiga arkitekturbeslut?
6. Vilka startvillkor skulle hjälpa team i din organisation att komma i gång utan att skapa oacceptabel risk?
7. När behöver du som arkitekt bli bättre på att säga "ja, men inom de här ramar" i stället för bara ja eller nej?
8. Vilka signaler visar att en tillfällig lösning håller på att bli permanent utan medvetet beslut?

Snabb sammanfattning

När teamet vill börja bygga innan arkitekturen är klar behöver arkitekten inte välja mellan att bromsa allt eller släppa kontrollen. Det mer användbara bidraget är att skapa **tillräcklig arkitektur** för nästa steg.

Det innebär att arkitekten hjälper team och produktägare att förstå:

- vilka beslut som måste fattas före start,
- vilka frågor som kan undersökas genom byggande,
- vilka risker som är acceptabla i liten skala,
- vilka gränser teamet inte får överskrida,
- vilka tillfälliga vägval som behöver märkas och följas upp.

Arkitektoniska startvillkor gör det möjligt att börja utan att låtsas att allt är klart. Utforskande implementation gör det möjligt att använda byggande som lärande, inte bara som produktion.

För arkitekten handlar skiftet om att gå från frågan:

“Är arkitekturen klar?”

Till frågan:

“Är arkitekturen tillräcklig för att vi ska kunna ta nästa steg på ett ansvarsfullt sätt?”

Nästa steg

I nästa kapitel fördjupas frågan om design. När teamen börjar arbeta mer iterativt blir design inte längre en separat fas som avslutas före implementation. I stället behöver design ske kontinuerligt, nära teamens vardag, men fortfarande med riktning, principer och helhetssyn.

Nästa kapitel handlar därför om situationen: **när design inte längre är en separat fas.**

Kapitel 7: När design inte längre är en separat fas

Varför detta kapitel finns

I många fasorienterade arbetssätt har design en tydlig plats. Först analyseras behov och krav, därefter tas en lösning eller design fram, och sedan bygger teamet utifrån den. Det kan ge ordning, spårbarhet och en känsla av kontroll. För en arkitekt är det också ett välbekant sätt att bidra: samla in information, väga alternativ, formulera designprinciper, beskriva lösningen och lämna över ett genomtänkt underlag till dem som ska realisera den.

När utvecklingen blir mer agil förändras designens rytm. Design sker inte längre bara före utveckling. Den sker också under utveckling, i små beslut, i tekniska avvägningar, i samtal mellan teammedlemmar, i refinement, i kodgranskning, i testning, i incidenter och i återkoppling från användare. Design blir mindre av en separat fas och mer av en kontinuerlig praktik.

Det betyder inte att design blir mindre viktigt. Tvärtom kan design bli viktigare, eftersom många beslut fattas närmare implementationen och därför behöver stöd av tydliga principer, gemensam förståelse och arkitektonisk närvaro. Om designarbetet blir helt lokalt och osynligt kan lösningen gradvis glida isär. Om arkitekten å andra sidan försöker hålla kvar all design som en separat kontrollpunkt kan flödet stanna och teamets lärande försvagas.

Det här kapitlet handlar om hur arkitekten kan bidra till **kontinuerlig design**: designarbete som sker löpande, men inte slumpmässigt. Målet är att skapa en balans där teamet kan fatta många lokala beslut, samtidigt som lösningen hålls ihop över tid.

Kapitlets kärnfråga är:

Hur kan arkitekten bidra till sammanhållen design när designbesluten uppstår löpande i utvecklingsarbetet?

Situationen: “Det där löser vi när vi bygger”

På Myndigheten för samhällstjänster arbetar teamet vidare med den nya medborgartjänsten. Efter kapitel 6 har Sara och teamet kommit överens om några arkitektoniska startvillkor. Teamet får börja med en smal läsvy för ärendestatus, men med tydliga ramar för identitet,loggning, informationskällor och vad som får visas externt.

Efter några veckor märker Sara något nytt. Teamet rör sig snabbt framåt, men många designbeslut fattas i vardagen utan att de syns särskilt tydligt.

Lina säger på ett möte:

“Vi löste statusöversättningen i tjänstelagret tills vidare. Det var enklast.”

En utvecklare fyller i:

“Och vi lade till en liten lokal tabell för visningsnamn, annars blev det för mycket beroende till gamla systemet.”

Erik, produktägaren, är nöjd:

“Bra. Då kan vi visa något för verksamheten nästa vecka.”

Sara är också nöjd med framdriften. Men hon får en gnagande känsla. Varje beslut verkar rimligt i stunden. Samtidigt påverkar besluten helheten:

- Var ska verksamhetsregler egentligen ligga?
- Hur många lokala översättningar och specialfall är acceptabla?
- När blir en tillfällig tabell en del av den långsiktiga lösningen?
- Vem ser att flera små beslut tillsammans skapar en ny arkitektur?
- Hur dokumenteras varför teamet valde just denna väg?
- Hur vet nästa team vilka principer som gäller?

Det är inte så att teamet gör fel. De försöker leverera värde och lösa konkreta problem. Men design håller på att ske i många små steg. Om Sara inte är närvarande riskerar hon att upptäcka helhetsmönstret först när det redan är svårt att ändra.

Samtidigt vill hon inte säga:

“Stopp, vi behöver en designfas.”

Det skulle skicka arbetet tillbaka till ett mönster som organisationen försöker lämna. Sara behöver i stället hitta ett sätt att göra designen synlig, gemensam och styrbar utan att dra ut den ur flödet.

Det invanda sättet att agera

Det invanda sättet att skapa god design är ofta att samla designarbetet i en avgränsad aktivitet. Arkitekten tar fram en lösningsdesign, beskriver komponenter, gränssnitt, informationsflöden, säkerhetslösning, integrationer och centrala regler. Därefter granskas designen och används som underlag för implementation.

Detta arbetssätt har flera styrkor:

- Det tvingar fram helhetstänkande.
- Det gör viktiga avvägningar synliga innan de omsätts i kod eller konfiguration.
- Det skapar dokumentation som kan granskas och förankras.
- Det minskar risken för att olika delar av lösningen utvecklas åt olika håll.
- Det ger arkitekten möjlighet att se beroenden och konsekvenser.

I en myndighetsmiljö är dessa styrkor viktiga. Det finns ofta krav på spårbarhet, förvaltningsbarhet, säkerhet, informationshantering och ansvar. Design kan inte bara vara något som “råkar bli” medan teamet bygger.

Problemet uppstår när designfasen blir den enda plats där design anses få ske. Då kan arkitekten hamna i ett mönster där design behöver vara färdig innan utveckling får börja, och där förändringar under utvecklingen uppfattas som avvikelser från den tänkta lösningen.

Det invanda agerandet kan då bli att:

- efterfråga mer komplett design innan teamet går vidare,
- vilja granska varje större designbeslut innan implementation,
- dokumentera lösningen i detalj för att undvika missförstånd,
- hålla designansvaret nära arkitektrollen,
- se teamets löpande designbeslut som risker snarare än som nödvändiga delar av lärandet.

Detta kan vara förståeligt. Arkitekten försöker skydda helheten. Men i ett agilt utvecklingsflöde kan det också skapa distans mellan arkitektur och faktisk lösning. Designen på papper blir då något som finns bredvid arbetet, medan den verkliga designen växer fram i teamets dagliga beslut.

Varför det kan skapa friktion i agil utveckling

Friktionen uppstår när design behandlas som en leverans i stället för som en förmåga.

I en fasorienterad logik är det ofta naturligt att fråga:

“Är designen klar?”

I en mer agil logik behöver frågan ofta vara:

“Har vi tillräcklig designriktning för nästa steg, och har vi ett arbetssätt som fångar upp nya designbeslut?”

Skillnaden är stor. Om design ska vara klar tidigt behöver arkitekten försöka förutse många detaljer innan teamet har lärt sig vad som faktiskt är svårt. Det kan leda till omfattande antaganden. Några av dem kommer att visa sig riktiga. Andra kommer att ändras när teamet möter verkliga data, äldre system, användarbeteenden, prestandakrav eller säkerhetsbegränsningar.

Om design däremot lämnas helt till teamets löpande arbete finns en annan risk: många små lokala beslut kan tillsammans skapa en lösning som ingen egentligen valt medvetet. Det kan handla om duplicerade regler, olika mönster för felhantling, varierande integrationssätt, olika tolkningar av säkerhetskrav eller otydliga ansvar mellan komponenter.

Friktionen visar sig ofta i vardagliga kommentarer:

- “Det där löser vi i teamet.”
- “Vi hinner inte ta upp varje detalj med arkitektur.”
- “Arkitekten är inte tillräckligt nära för att förstå varför vi valde så här.”
- “Teamet har byggt något som inte följer målbilden.”
- “Målbilden var för generell för att hjälpa i beslutet.”
- “Vi dokumenterar senare när lösningen stabiliserat sig.”

Alla dessa kommentarer kan vara rimliga ur sitt perspektiv. De visar bara att organisationen saknar ett fungerande sätt att göra löpande designbeslut synliga och gemensamma.

För arkitekten blir utmaningen att inte svara med mer kontroll än situationen behöver. I stället behöver arkitekten skapa strukturer som hjälper teamet att designa bra i vardagen.

Ett mer agilt förhållningssätt

Ett mer agilt förhållningssätt till design börjar med en viktig insikt:

Designansvar kan delas utan att helhetsansvar försvinner.

Det betyder att arkitekten inte behöver fatta alla designbeslut själv. Däremot behöver arkitekten bidra till att rätt beslut fattas på rätt nivå, att besluten hänger ihop och att viktiga konsekvenser blir synliga.

Kontinuerlig design bygger på tre delar:

1. **Tydliga designprinciper**
2. **Återkommande designsamtal**
3. **Synliga designbeslut**

Tydliga designprinciper

Designprinciper är inte detaljerade lösningar. De är vägledande regler som hjälper teamet att fatta många mindre beslut utan att behöva fråga om allt.

För Myndigheten för samhällstjänster kan sådana principer till exempel vara:

- Medborgartjänsten ska inte innehålla egna kopior av information som är juridiskt styrande.
- Verksamhetsregler som påverkar ärendehantering ska vara spårbara till ansvarig verksamhetsprocess.
- Integrationer mot äldre system ska kapslas in så att framtida modernisering underlättas.
- Säkerhets- och behörighetsbeslut ska vara explicita, inte inbyggda som antaganden i gränssnitt.
- Tillfälliga lösningar ska ha en dokumenterad ägare och ett tydligt omprövningstillfälle.

Sådana principer hjälper teamet att agera självständigt. De säger inte exakt hur varje komponent ska byggas, men de gör det tydligare vilka avvägningar som är viktiga.

Återkommande designsamtal

Kontinuerlig design kräver forum, men inte nödvändigtvis tunga forum. Det kan räcka med korta, regelbundna samtal där arkitekt, team och ibland produktägare eller säkerhetsroller tittar på aktuella designfrågor.

Syftet är inte att arkitekten ska godkänna allt. Syftet är att upptäcka mönster, avvikelser och nya beslut i tid.

Frågor i ett sådant samtal kan vara:

- Vilka designbeslut har teamet fattat sedan sist?
- Vilka beslut närmar sig som kan påverka andra team eller system?

- Finns det något tillfälligt som riskerar att bli permanent?
- Har vi brutit mot någon princip, och var det i så fall medvetet?
- Behöver något beslut dokumenteras som ett arkitekturbeslut?
- Behöver någon annan roll kopplas in tidigare?

Det viktiga är rytmen. Om designsamtalet sker ofta och nära arbetet behöver det inte bli stort. Om det sker sällan tenderar det att bli en granskning av mycket som redan är byggt.

Synliga designbeslut

I agil utveckling fattas många designbeslut löpande. Det är inte ett problem i sig. Problemet är när besluten försvinner.

Alla beslut behöver inte dokumenteras på samma sätt. Ett lokalt val av intern kodstruktur kanske inte behöver lyftas. Men beslut som påverkar beroenden, informationsansvar, säkerhet, integration, förvaltning eller framtida handlingsutrymme behöver fångas.

En enkel beslutslogg kan räcka:

- Vad beslutades?
- Varför beslutades det?
- Vilka alternativ övervägdes?
- Vilka konsekvenser accepterar vi?
- När behöver beslutet omprövas?
- Vem behöver känna till det?

Detta gör designen lärbar. Nästa team kan förstå varför lösningen ser ut som den gör. Förvaltningen kan förstå vilka avvägningar som gjorts. Arkitekturforum kan se mönster utan att behöva godkänna varje detalj.

Exempel: Myndigheten för samhällstjänster

Sara bestämmer sig för att inte försöka återinföra en designfas. I stället bokar hon ett kort veckovis designsamtal med Lina och två utvecklare från teamet. Erik deltar när frågorna påverkar prioritering eller verksamhetsregler. Amir från informationssäkerhet bjuds in vid behov, särskilt när behörighet,loggning eller personuppgifter påverkas.

Första samtalet börjar konkret.

Sara säger:

”Jag vill inte att vi stoppar flödet. Men jag vill att vi ser vilka designbeslut vi faktiskt håller på att fatta.”

Teamet går igenom tre beslut från veckan:

1. Statusvärden översätts i tjänstelagret.
2. En lokal tabell används för visningsnamn.
3. Fel från handläggningssystemet visas som generella meddelanden för medborgaren.

Varje beslut hade ett gott skäl. Men i samtalet ser gruppen att de tillsammans rör vid en större fråga: var ska översättning mellan interna ärendebegrepp och medborgarnära begrepp äga rum?

Det är inte bara en teknisk fråga. Det påverkar verksamhetsansvar, spårbarhet, kommunikation med medborgare och framtida möjlighet att återanvända begrepp i andra digitala tjänster.

Sara föreslår inte en lång utredning. Hon formulerar i stället ett designbeslut:

“För första inkrementet får teamet hantera visningsnamn i tjänstelagret, men begreppsöversättning som påverkar juridisk eller verksamhetsmässig innebörd ska ägas av verksamhetsprocessen och dokumenteras som en separat regel. Beslutet omprövas innan fler ärendetyper läggs till.”

Detta blir varken en fullständig målarkitektur eller ett fritt lokalt val. Det är ett medvetet, begränsat och omprövningsbart designbeslut.

Efter några veckor har teamet en liten lista med designprinciper:

- Skilj medborgarnära begrepp från interna handläggningsbegrepp.
- Undvik direkta beroenden från medborgartjänsten till handläggningssystemets interna datamodell.
- Dokumentera tillfälliga lösningar med omprövningsdatum.
- Lyft beslut som påverkar andra tjänster, inte varje lokal koddetalj.
- Bjud in säkerhet och juridik när designen påverkar informationens exponering eller spårbarhet.

Sara märker att hennes roll förändras. Hon designar fortfarande, men inte genom att äga varje lösningsdetalj. Hon hjälper teamet att se mönster, formulera principer och göra viktiga beslut synliga.

Vanliga fallgropar

- **Fallgrop: Att kalla allt teamet gör för “bara implementation”.**
 - Varför den uppstår: I en fasorienterad logik skiljs design och implementation ofta tydligt åt.
 - Vad arkitekten kan göra i stället: Se att implementation ofta innehåller designbeslut och hjälp teamet att identifiera vilka beslut som har arkitektonisk betydelse.
- **Fallgrop: Att försöka granska varje designbeslut.**
 - Varför den uppstår: Arkitekten vill skydda helheten och undvika framtida problem.
 - Vad arkitekten kan göra i stället: Skilj mellan lokala beslut, beslut som påverkar teamets lösning och beslut som påverkar flera system eller framtida handlingsutrymme.
- **Fallgrop: Att designprinciper blir för abstrakta.**
 - Varför den uppstår: Principer formuleras ibland som allmänna ideal, till exempel “lösningen ska vara flexibel”.
 - Vad arkitekten kan göra i stället: Formulera principer så att de hjälper teamet att välja mellan konkreta alternativ.
- **Fallgrop: Att tillfälliga designval blir permanenta av misstag.**
 - Varför den uppstår: Teamet behöver lösa ett problem snabbt och saknar en tydlig rytm för omprövning.

- Vad arkitekten kan göra i stället: Dokumentera tillfälliga beslut med ägare, konsekvens och tidpunkt för omprövning.
- **Fallgrop: Att gemensamt designansvar blir otydligt ansvar.**
 - Varför den uppstår: När fler deltar i designen kan det bli oklart vem som ser helheten.
 - Vad arkitekten kan göra i stället: Tydliggör vilka beslut teamet äger, vilka arkitekten stödjer och vilka som behöver lyftas till bredare forum.

Frågor att ställa i situationen

När design inte längre är en separat fas kan arkitekten använda frågor som gör designarbetet synligt utan att bromsa det:

- Vilka designbeslut har vi faktiskt fattat den senaste veckan?
- Vilka av dessa beslut påverkar bara teamet, och vilka påverkar andra?
- Vilka principer använder teamet när det väljer mellan olika lösningar?
- Finns det tillfälliga lösningar som behöver ett omprövningsdatum?
- Har vi skapat nya beroenden som inte var synliga tidigare?
- Har vi flyttat verksamhetsregler, säkerhetslogik eller informationsansvar utan att märka det?
- Behöver något beslut dokumenteras för förvaltning, spårbarhet eller framtida team?
- Vilka personer behöver vara med i nästa designsamtal för att beslutet ska bli tillräckligt belyst?

Reflektionsfrågor

1. I vilka delar av ditt arbete behandlas design fortfarande som en separat fas?
2. Var sker de verkliga designbesluten i din organisation i dag?
3. Vilka designbeslut fattar teamen utan att de blir synliga för arkitektur eller förvaltning?
4. Vilka designprinciper skulle hjälpa teamen att fatta bättre beslut utan att fråga om allt?
5. När brukar du själv känna behov av att granska, och vad är det egentligen du försöker skydda?
6. Hur kan du skapa en rytm för designsamtal som stödjer flödet i stället för att bli en ny kontrollpunkt?
7. Vilka tillfälliga lösningar i din miljö borde få ett tydligt omprövningsdatum?
8. Hur kan du se skillnad på design som behöver styras centralt och design som bör växa fram nära teamet?

Snabb sammanfattning

- I agil utveckling sker design inte bara före implementation, utan löpande under arbetets gång.
- Det betyder inte att design blir mindre viktig; det betyder att design behöver göras synlig i flödet.
- Arkitekten behöver gå från att äga alla designprodukter till att skapa förutsättningar för bra designbeslut över tid.

- Kontinuerlig design kräver tydliga designprinciper, återkommande design-samtal och synliga designbeslut.
- Team kan fatta många lokala beslut självständigt, men beslut som påverkar beroenden, säkerhet, information, förvaltning eller framtida handlingsutrymme behöver uppmärksammas.
- Tillfälliga lösningar behöver ägare, konsekvensbeskrivning och tidpunkt för omprövning.
- Gemensamt designansvar fungerar bara om ansvarsnivåer och eskaleringsvägar är tydliga.

Nästa steg

När designbeslut sker löpande blir beroenden ofta synligare. Vissa beroenden är tekniska, till exempel integrationer mellan system. Andra är organisatoriska, till exempel väntan på beslut, leverantörer, forum eller andra team. I nästa kapitel går vi vidare till situationen där beroenden börjar bromsa flödet, och hur arkitekten kan bidra genom att inte bara kartlägga beroenden utan också minska, frikoppla och göra dem mer hanterbara.

Kapitel 8: När beroenden bromsar flödet

Varför detta kapitel finns

I ett fasorienterat arbetssätt hanteras beroenden ofta genom planering, samordning och beslutspunkter. Man kartlägger vilka system, leverantörer, verksamhetsdelar och beslut som påverkar varandra. Därefter försöker man lägga en plan där beroendena hanteras i rätt ordning. För en arkitekt är detta välbekant arbete: att se kopplingarna, förstå konsekvenserna och hjälpa organisationen att undvika överraskningar.

I agil utveckling försvinner inte beroendena. Ofta blir de tvärtom mer synliga. När team försöker leverera i kortare cykler märks det snabbt om de måste vänta på ett annat system, ett beslut, en miljö, en integration, en säkerhetsbedömning eller en leverantör. Det som i en lång projektplan kunde se hanterbart ut blir i ett iterativt flöde till konkret väntan.

Det här kapitlet handlar om arkitektens roll när beroenden bromsar utvecklingsflödet. Fokus ligger inte bara på att identifiera beroenden, utan på att förstå vilka beroenden som kan minskas, vilka som behöver göras tydligare och vilka som måste hanteras genom bättre samspel.

Kapitlets kärnfråga är:

Hur kan arkitekten hjälpa organisationen att gå från beroendekartläggning till aktiv beroendehantering?

Situationen: “Vi är klara, men vi väntar på tre andra”

På Myndigheten för samhällstjänster har teamet kommit vidare med den nya medborgartjänsten. De har byggt en första version av en vy där medborgaren kan se status på sitt ärende. Funktionen är smal, men användbar. Den visar inte allt, men den räcker för att börja testa nytta och förståelse med en begränsad grupp användare.

På ett avstämningsmöte sammanfattar Lina läget:

“Själva vyn är nästan klar. Men vi kan inte släppa den förrän integrationen mot handläggningssystemet är ändrad, loggningen är godkänd och den externa myndigheten har bekräftat sitt datagränssnitt.”

Erik, produktägaren, ser frustrerad ut:

“Så vi är klara, men ändå inte klara?”

Sara känner igen situationen. Teamet har arbetat iterativt och fokuserat. Ändå står leveransen still. Det finns beroenden till:

- det äldre handläggningssystemet,
- en integrationsplattform,
- informationssäkerhetsfunktionen,
- en extern myndighet,
- ett separat team som ansvarar för behörighet,
- en förvaltningsgrupp som behöver acceptera nya driftflöden,
- ett arkitekturforum som ska ta ställning till ett undantag.

I en projektplan hade dessa beroenden kanske funnits som aktiviteter, milstolpar och samordningspunkter. I teamets vardag blir de i stället väntan, osäkerhet och omprioriteringar. Teamet kan inte leverera det de har byggt, och produktägaren får svårt att skapa tydliga förväntningar.

Sara märker att hennes första impuls är att göra en mer komplett beroendekarta. Det är inte fel. Men hon inser också att en karta inte räcker. Om beroendena är inbyggda i lösningen, organisationen och beslutsvägarna behöver de hanteras mer aktivt.

Hon börjar därför ställa andra frågor:

- Vilka beroenden är nödvändiga?
- Vilka beroenden har vi själva skapat genom designval?
- Vilka beroenden kan ersättas av tydligare gränssnitt?
- Vilka beroenden kan hanteras genom tillfälliga lösningar medan vi lär oss?
- Vilka beroenden kräver beslut på en annan nivå?
- Vilka beroenden behöver göras synliga för styrning och prioritering?

Det är här arkitektens bidrag blir centralt. Inte genom att ensam lösa alla beroenden, utan genom att hjälpa organisationen att förstå vilka beroenden som går att reducera, vilka som behöver koordineras och vilka som visar på djupare strukturella problem.

Det invanda sättet att agera

Det invanda sättet att arbeta med beroenden är ofta att kartlägga dem, beskriva dem och planera runt dem. Arkitekten identifierar vilka system som påverkas, vilka gränssnitt som behövs, vilka beslut som måste fattas och vilka parter som behöver involveras. Resultatet kan bli beroendekartor, integrationsöversikter, tidplaner, ansvarsmatriser och risklistor.

Detta kan vara mycket värdefullt. I en myndighetsmiljö finns ofta verkliga beroenden som inte kan ignoreras. Äldre system kan innehålla verksamhetskritisk logik. Andra myndigheter kan äga data eller processer. Säkerhet, juridik och förvaltning behöver ofta involveras. Vissa beroenden är helt enkelt en del av uppdragets natur.

Problemet uppstår när beroendehantering stannar vid kartläggning och koordinering. Då kan arkitekten hjälpa organisationen att förstå varför arbetet är svårt, men inte nödvändigtvis hjälpa den att bli mindre beroende.

I en faserorienterad logik kan det kännas tillräckligt att veta vilka beroenden som finns och när de ska hanteras. I en agil logik behöver man oftare fråga om beroen-

det kan förändras. Annars riskerar teamet att få ett agilt arbetssätt på ytan, men ett långsamt beroendenät under ytan.

Typiska invanda reaktioner är:

- att skapa en mer detaljerad beroendeplan,
- att kalla till fler samordningsmöten,
- att eskalera beroenden till styrgrupp,
- att vänta in andra parter innan teamet får gå vidare,
- att acceptera beroenden som givna eftersom "så ser landskapet ut",
- att behandla alla beroenden som planeringsproblem snarare än designproblem.

Alla dessa åtgärder kan behövas ibland. Men de räcker inte alltid. Om beroendena återkommer i leverans efter leverans behöver arkitekten också undersöka om lösningens struktur, ansvarsfördelning eller tekniska gränser behöver förändras.

Varför det kan skapa friktion i agil utveckling

Agil utveckling bygger på att team kan skapa värde stegvis, få återkoppling och anpassa sig. Många beroenden gör detta svårt. Varje gång ett team behöver vänta på någon annan innan det kan testa, leverera eller ändra något minskar möjligheten till lärande.

Beroenden skapar friktion på flera sätt.

För det första skapar de väntan. Teamet kan ha gjort sitt arbete, men ändå inte kunna leverera eftersom ett annat system, beslut eller godkännande saknas.

För det andra skapar de omarbete. När ett beroende ändras sent kan teamet behöva ändra design, implementation eller prioritering.

För det tredje skapar de otydligt ansvar. Om flera team och funktioner behöver agera samtidigt blir det lätt oklart vem som faktiskt äger framdriften.

För det fjärde skapar de falsk lokal effektivitet. Varje team kan vara upptaget och produktivt, men helheten rör sig långsamt eftersom arbetet fastnar mellan team, system och beslutspunkter.

För det femte kan beroenden dölja arkitektoniska problem. Ett team som alltid väntar på samma system kanske inte bara har ett planeringsproblem. Det kan ha ett kopplingsproblem. Ett team som alltid måste gå via samma forum kanske inte bara har ett styrningsproblem. Det kan ha ett mandatproblem.

För arkitekten innebär detta en viktig förskjutning:

Beroenden är inte bara något som ska samordnas. De är signaler om hur lösningen och organisationen faktiskt är strukturerade.

När arkitekten ser beroenden på det sättet blir frågan större än "hur planerar vi runt detta?". Den blir också "vad i vår arkitektur gör att detta beroende fortsätter att bromsa oss?".

Ett mer agilt förhållningssätt

Ett mer agilt förhållningssätt börjar med att skilja mellan olika typer av beroenden. Alla beroenden är inte likadana, och de ska inte hanteras på samma sätt.

En enkel indelning är:

1. **Tekniska beroenden**

Beroenden till system, plattformar, integrationer, miljöer, datakällor eller tekniska komponenter.

2. **Informationsberoenden**

Beroenden till datamodeller, informationsägare, begreppsdefinitioner, informationsklassning eller datakvalitet.

3. **Organisatoriska beroenden**

Beroenden till team, enheter, leverantörer, förvaltning, drift eller externa myndigheter.

4. **Beslutsberoenden**

Beroenden till forum, styrgrupper, arkitekturgranskningar, säkerhetsbeslut eller formella godkännanden.

5. **Kompetensberoenden**

Beroenden till enskilda personer eller specialistfunktioner som behövs för att arbetet ska kunna gå vidare.

När beroendet är synligt kan arkitekten hjälpa till att välja strategi. Det finns åtminstone fem möjliga strategier.

1. Acceptera beroendet och gör det synligt

Vissa beroenden är verkliga och rimliga. En myndighet kan inte ensidigt ändra ett gränssnitt hos en annan myndighet. Ett team kan inte bortse från informations-säkerhetskrav. En juridisk bedömning kan vara nödvändig innan viss information exponeras.

Då är målet inte att låtsas att beroendet kan tas bort. Målet är att göra det synligt tidigt, formulera vad som behövs och skapa realistiska förväntningar.

Frågor att ställa:

- Vad väntar vi på?
- Vem äger beroendet?
- När behöver vi besked?
- Vad händer om beskedet dröjer?
- Finns det ett mindre steg vi kan ta under tiden?

2. Minska beroendet genom design

Vissa beroenden finns därför att lösningen är starkt kopplad. Om varje liten ändring i medborgartjänsten kräver ändring i handläggningssystemet kommer teamet att bromsas gång på gång.

Här kan arkitekten bidra genom beroendereducerande design. Det kan handla om tydligare gränssnitt, mellanlager, händelsebaserad kommunikation, stabilare kontrakt, bättre informationsmodeller eller att flytta ansvar till rätt del av lösningen.

Poängen är inte att införa en teknisk lösning för sakens skull. Poängen är att minska den praktiska väntan och göra teamens arbete mer självständigt där det är rimligt.

Frågor att ställa:

- Varför behöver detta team vänta på detta system?
- Är beroendet tekniskt nödvändigt eller historiskt uppkommet?
- Kan ett stabilt gränssnitt minska koordineringen?
- Kan vi dela upp ansvaret annorlunda?
- Vilken koppling kostar mest i återkommande väntan?

3. Skapa en tillfällig frikoppling för att lära

Ibland behöver teamet komma vidare för att testa ett antagande, även om alla integrationer inte är färdiga. Då kan en tillfällig lösning vara rimlig, till exempel en begränsad testdatakälla, en simulerad tjänst, en manuell kontroll i ett tidigt skede eller en smal implementation som inte kräver full integration.

Detta är inte samma sak som att bygga slarvigt. Skillnaden ligger i tydlighet. En tillfällig frikoppling behöver ha ett syfte, en tidsgräns och ett beslut om hur den ska hanteras senare.

Frågor att ställa:

- Vad behöver vi lära oss innan beroendet är löst?
- Kan vi testa användarvärde utan full integration?
- Vilken tillfällig lösning är acceptabel ur riskperspektiv?
- Hur dokumenterar vi att detta är tillfälligt?
- När behöver den tillfälliga lösningen ersättas?

4. Flytta beslut närmare kunskap

Vissa beroenden uppstår för att beslut måste tas långt från dem som har mest aktuell kunskap. Om teamet behöver vänta på ett forum för beslut som egentligen ryms inom etablerade principer kan styrningen skapa onödig väntan.

Arkitekten kan då hjälpa till att tydliggöra vilka beslut teamet får fatta själv, vilka som kräver arkitektdialog och vilka som behöver formellt forum.

Frågor att ställa:

- Behöver detta beslut verkligen fattas i ett forum?
- Finns det principer som gör att teamet kan fatta beslutet själv?
- Vilken risk motiverar en högre beslutsnivå?
- Kan forumet ge ramar i stället för att godkänna varje enskilt vägval?

5. Förändra planeringen utifrån beroendets natur

När ett beroende inte kan tas bort kan arbetet behöva planeras annorlunda. Det kan innebära att ta beroendefrågor tidigare, skapa gemensamma planeringspunkter, synkronisera med externa parter eller välja ett annat första inkrement.

Det viktiga är att planeringen inte bara utgår från vad teamet vill bygga, utan också från vad beroendena gör möjligt att lära och leverera.

Frågor att ställa:

- Vilket inkrement ger värde med minst kritiska beroenden?
- Vilket beroende behöver lösas först för att öppna flera senare steg?
- Kan vi välja en annan ordning som minskar väntan?
- Vilka beroenden behöver vara en del av prioriteringen, inte bara genomförandet?

Exempel: Myndigheten för samhällstjänster

Sara samlar Lina, Erik och Amir för att titta på varför den första versionen av medborgartjänsten har fastnat. Hon ritar inte en fullständig beroendekarta över hela myndigheten. I stället börjar hon med den konkreta leveransen som står still.

De identifierar fyra beroenden.

Det första är integrationen till det äldre handläggningssystemet. Teamet behöver läsa ärendestatus, men handläggningssystemet är inte byggt för tät extern åtkomst. Varje ändring kräver samordning med förvaltningsgruppen.

Det andra är loggning. Amir påpekar att åtkomst till ärendestatus behöver kunna följas upp, särskilt eftersom informationen kan vara känslig.

Det tredje är begreppsdefinitioner. Verksamheten använder flera interna statuskoder som inte är begripliga för medborgare.

Det fjärde är en extern myndighet som äger vissa kompletterande uppgifter. Den integrationen kommer inte att vara klar inom den närmaste tiden.

Tidigare hade Sara kanske främst sammanställt beroendena och lyft dem till styrning. Nu väljer hon att sortera dem.

- Integrationen till handläggningssystemet är både tekniskt och organisatoriskt beroende.
- Loggningen är ett säkerhets- och regelefterlevnadsberoende.
- Statuskoderna är ett informationsberoende.
- Den externa myndigheten är ett externt beroende som teamet inte kan styra över.

När de sorterar beroendena ser de att alla inte behöver lösas på samma sätt.

För den första versionen beslutar de att bara visa status för ett begränsat antal ärendetyper. Det minskar beroendet till handläggningssystemets mest komplexa delar.

För loggningen tar Sara och Amir fram ett arkitektoniskt startvillkor: ingen extern visning utan spårbar åtkomstlogg. Men de gör också klart vilken minimnivå som räcker för den första begränsade användartesten.

För statuskoderna skapar teamet en enkel översättningstabell, men dokumenterar att den är en medveten mellanlösning. Sara markerar beslutet i beslutsloggen eftersom det kan påverka framtida informationsarkitektur.

För den externa myndigheten väljer Erik att inte vänta. Den första versionen får tydligt avgränsas till ärenden där den externa informationen inte behövs.

Resultatet är inte att alla beroenden försvinner. Men teamet går från att vara blockerat till att kunna leverera ett smalare inkrement med tydliga begränsningar.

Sara sammanfattar:

“Vi har inte löst hela beroendebilden. Men vi har skilt på det som måste vara på plats, det som kan avgränsas bort, det som kan göras tillfälligt och det som behöver ett långsiktigt arkitekturbeslut.”

Det är ett annat sätt att arbeta med beroenden. Mindre fokus på att allt ska vara koordinerat innan utvecklingen fortsätter. Mer fokus på att förstå beroendena tillräckligt väl för att kunna minska väntan och fatta bättre stegvisa beslut.

Vanliga fallgropar

- **Fallgrop: Att behandla alla beroenden som planeringsfrågor**
 - Varför den uppstår: Det är naturligt att vilja lösa väntan genom bättre koordinering, möten och tidplaner.
 - Vad arkitekten kan göra i stället: Undersök om beroendet också är ett designproblem, ett mandatproblem eller ett informationsproblem.
- **Fallgrop: Att acceptera historiska kopplingar som naturlagar**
 - Varför den uppstår: Äldre systemlandskap har ofta vuxit fram under lång tid, och kopplingarna känns givna.
 - Vad arkitekten kan göra i stället: Fråga vilka kopplingar som faktiskt behövs för värdeskapandet och vilka som bara finns på grund av tidigare designval.
- **Fallgrop: Att minska beroenden genom att skapa nya dolda beroenden**
 - Varför den uppstår: En snabb mellanlösning kan verka frikoppla teamet, men samtidigt skapa ny teknikskuld eller otydligt ansvar.
 - Vad arkitekten kan göra i stället: Dokumentera tillfälliga lösningar, tidsätt dem och följ upp dem som arkitekturbeslut.
- **Fallgrop: Att eskalera för många beslut**
 - Varför den uppstår: När riskerna är oklara känns det tryggt att lyfta frågor till forum eller styrning.
 - Vad arkitekten kan göra i stället: Skilj på beslut som kräver formellt mandat och beslut som kan fattas inom etablerade principer.
- **Fallgrop: Att optimera ett team men glömma helheten**
 - Varför den uppstår: Agila team kan bli starkt fokuserade på sitt eget flöde och sin egen leverans.
 - Vad arkitekten kan göra i stället: Titta på hur lokala förenklingar påverkar andra team, förvaltning, säkerhet och framtida ändringsbarhet.

Frågor att ställa i situationen

När ett beroende bromsar arbetet kan arkitekten använda följande frågor:

1. Vad är det konkret som teamet väntar på?
2. Är beroendet tekniskt, organisatoriskt, informationsmässigt, beslutsamässigt eller kompetensmässigt?
3. Är beroendet nödvändigt, eller ett resultat av tidigare design- eller organisationsval?
4. Kan beroendet minskas genom ett tydligare gränssnitt eller en annan ansvarsfördelning?
5. Kan teamet lära något värdefullt genom ett smalare inkrement?
6. Vilken risk uppstår om vi går vidare utan att beroendet är helt löst?
7. Vem behöver fatta beslut om avgränsning, risk eller prioritering?

8. Hur dokumenterar vi beroendet så att det inte försvinner ur sikte?
9. Är detta ett enstaka hinder eller ett återkommande mönster?
10. Vad skulle göra nästa liknande leverans mindre beroende?

Reflektionsfrågor

1. Vilka beroenden bromsar oftast utvecklingsflödet i din organisation?
2. Vilka av dessa beroenden brukar beskrivas som planeringsproblem, men kan egentligen vara arkitekturproblem?
3. När har du sett ett team vara lokalt effektivt men ändå fastna i helhetens beroenden?
4. Vilka beroenden är rimliga och nödvändiga i din myndighetskontext?
5. Vilka beroenden skulle kunna minska genom tydligare gränssnitt, ansvar eller principer?
6. Hur brukar du som arkitekt agera när ett team väntar på andra?
7. Vilken fråga skulle du kunna börja ställa oftare: "Hur koordinerar vi detta?" eller "Varför är vi beroende på det här sättet?"
8. Vilket beroende borde lyftas som ett långsiktigt arkitekturbeslut snarare än som ett återkommande leveranshinder?

Snabb sammanfattning

Beroenden är inte bara något som ska kartläggas. De påverkar teamens möjlighet att leverera, lära och anpassa sig.

I en fasorienterad logik hanteras beroenden ofta genom planering, samordning och beslutspunkter. I en mer agil logik behöver arkitekten också fråga om beroendena kan minskas, frikopplas eller hanteras på ett sätt som stödjer flöde.

Alla beroenden ska inte tas bort. Vissa är nödvändiga, särskilt i myndighetsmiljöer med säkerhet, juridik, förvaltning och externa aktörer. Men beroenden behöver förstås tillräckligt väl för att organisationen ska kunna avgöra vilka som ska accepteras, vilka som ska göras synliga, vilka som ska reduceras och vilka som kräver långsiktiga arkitekturbeslut.

Arkitektens bidrag är att hjälpa organisationen se skillnaden mellan:

- beroenden som måste samordnas,
- beroenden som kan minskas genom design,
- beroenden som kan hanteras genom avgränsade inkrement,
- beroenden som kräver tydligare mandat,
- beroenden som visar att arkitekturen behöver förändras.

Nästa steg

När beroenden hanteras mer aktivt blir dokumentationen viktig på ett nytt sätt. Det räcker inte att dokumentera lösningen som den borde se ut i slutet. Organisationen behöver också kunna förstå vilka beslut som har fattats, varför vissa beroenden accepterats, vilka mellanlösningar som finns och vilka risker som ska följas upp.

Nästa kapitel handlar därför om **dokumentation som stöd för lärande och ansvar**. Där flyttas fokus från dokumentation som leverans eller godkännande till dokumentation som ett praktiskt stöd för beslut, samordning, spårbarhet och fortsatt utveckling.

Kapitel 9: Dokumentation som stöd för lärande och ansvar

Varför detta kapitel finns

I en myndighetsmiljö är dokumentation sällan något som går att välja bort. Beslut behöver kunna följas upp. Ansvar behöver kunna förstås i efterhand. Förvaltning behöver veta vad som har byggts och varför. Säkerhet, juridik, dataskydd och verksamhet behöver kunna se vilka avvägningar som har gjorts.

Samtidigt blir dokumentation lätt en källa till friktion när utvecklingen blir mer agil. Teamet upplever att dokumentationen tar tid från leverans. Styrning och förvaltning upplever att teamet bygger snabbare än organisationen hinner förstå. Arkitekten hamnar ofta mitt emellan: någon behöver skapa helhet, spårbarhet och långsiktig begriplighet, men utan att återinföra en tung dokumentationsfas.

Det här kapitlet handlar om hur dokumentation kan bli ett stöd för lärande och ansvar i stället för en parallell produkt vid sidan av utvecklingen.

Kapitlets centrala skifte är:

Från dokumentation som bevis på att arbetet är färdigt till dokumentation som stöd för beslut, samordning, lärande och ansvar över tid.

Situationen: dokumentation efterfrågas men ingen vet riktigt för vem

Sara sitter i ett möte om moderniseringen av handläggningsstödet på Myndigheten för samhällstjänster. Teamet har under de senaste veckorna byggt en första intern version av ett nytt ärendeflöde. Lösningen är inte färdig, men den har gett flera viktiga insikter: vissa antaganden om integrationen med det gamla systemet stämde inte, behörighetsmodellen blev mer komplex än väntat och handläggarna använde flödet på ett annat sätt än verksamheten först beskrev.

På mötet säger styrgruppen:

“Vi behöver en uppdaterad arkitekturbeskrivning så att vi vet vad som byggs.”

Teamet svarar:

“Vi har inte tid att skriva dokument som ändå är inaktuella om två veckor.”

Förvaltningen säger:

“Vi behöver förstå vad som förändras, annars kan vi inte ta ansvar senare.”

Säkerhetsfunktionen säger:

“Vi behöver se hur ni har tänkt kring behörighet och loggning.”

Alla har rimliga behov. Problemet är att ordet dokumentation betyder olika saker för olika personer.

För styrgruppen kan dokumentation betyda kontroll och insyn. För teamet kan det betyda avbrott. För förvaltningen betyder det framtida ansvar. För säkerhetsfunktionen betyder det riskförståelse och spårbarhet. För Sara blir frågan inte bara “vilket dokument ska vi skriva?”, utan “vilket arbete ska dokumentationen hjälpa oss att göra?”

Det invanda sättet att agera

I en mer fasorienterad utvecklingslogik är dokumentation ofta kopplad till tydliga övergångar:

- en förstudie dokumenterar behov och lösningsinriktning,
- en kravspecifikation dokumenterar vad som ska byggas,
- en arkitekturbeskrivning dokumenterar hur lösningen ska utformas,
- en designbeskrivning dokumenterar detaljer,
- en överlämning dokumenterar vad förvaltningen ska ta emot.

Det invanda arbetssättet kan skapa trygghet. Det finns dokument att hänvisa till. Beslut kan förankras. Granskning kan ske mot ett underlag. Ansvar kan formuleras.

För en erfaren arkitekt är det därför naturligt att vilja skapa ett samlat dokument som beskriver lösningen ordentligt. Det känns ansvarsfullt. Det skapar ordning. Det minskar risken för missförstånd.

Men i ett mer agilt arbetssätt förändras dokumentationens rytm. Lösningen växer fram successivt. Vissa beslut är preliminära. Vissa antaganden testas genom implementation. Vissa delar är stabila, medan andra behöver ändras ofta. Om dokumentationen fortfarande behandlas som om allt vore färdigutrett riskerar den att antingen bli för tung, för sen eller för snabbt inaktuell.

Det invanda sättet att agera kan därför se ut så här:

1. Arkitekten försöker samla ihop allt i ett större dokument.
2. Dokumentet blir svårt att hålla aktuellt.
3. Teamet upplever dokumentationen som en extern börda.
4. Styrning och förvaltning får ändå inte den insyn de behöver.
5. Dokumentationen börjar användas för godkännande snarare än lärande.

Varför det kan skapa friktion i agil utveckling

Friktionen uppstår inte för att dokumentation är oviktig. Den uppstår för att dokumentationen inte längre passar arbetets tempo, osäkerhet eller beslutsbehov.

Dokumentationen blir för stor

Ett omfattande arkitekturdokument kan vara användbart när lösningen är relativt stabil och behöver förankras samlat. Men om mycket fortfarande förändras kan ett stort dokument bli svårt att uppdatera. Det riskerar att innehålla både stabila beslut, tillfälliga antaganden, öppna frågor och gamla formuleringar utan tydlig skillnad mellan dem.

Då blir dokumentet inte ett stöd. Det blir ett arkiv av blandad aktualitet.

Dokumentationen kommer för sent

Om dokumentation skapas först efter att teamet har byggt, kommer viktiga beslut redan att vara fattade. Då blir dokumentationen mer av en efterhandsbeskrivning än ett stöd i beslutsprocessen.

Det kan fortfarande vara värdefullt, men det hjälper inte teamet att fatta bättre beslut när besluten faktiskt uppstår.

Dokumentationen skrivs för fel mottagare

Samma dokument förväntas ofta uppfylla många behov samtidigt: styrning, granskning, teamkommunikation, förvaltning, säkerhet, juridik och historik. Resultatet blir lätt ett dokument som är för detaljerat för vissa, för övergripande för andra och för otydligt för alla.

En viktig fråga är därför: vem ska använda dokumentationen, och till vad?

Dokumentationen ersätter samtal

När organisationen blir osäker kan den be om mer dokumentation. Det är förståeligt. Men vissa oklarheter löses inte genom fler dokument, utan genom bättre samtal mellan rätt personer.

Dokumentation kan bära resultatet av ett samtal. Den kan synliggöra ett beslut. Den kan göra ett antagande spårbart. Men den bör inte bli en ersättning för den dialog som krävs för att förstå problemet.

Dokumentationen blir en kontrollpunkt

I en agil utvecklingslogik behöver dokumentation ofta vara en del av flödet. Om den i stället blir en separat kontrollpunkt kan den skapa väntan. Teamet bygger, stannar upp, dokumenterar, väntar på granskning, ändrar, väntar igen.

Det betyder inte att granskning är fel. Men granskningen behöver ske på rätt nivå, med rätt underlag och vid rätt tidpunkt.

Ett mer agilt förhållningssätt

Ett mer agilt förhållningssätt handlar inte om mindre dokumentation som princip. Det handlar om mer användbar dokumentation.

Sara behöver därför byta grundfråga.

Inte:

“Vilket dokument förväntas av oss?”

Utan:

“Vilka beslut, risker, beroenden och ansvar behöver bli begripliga för rätt personer över tid?”

Det leder till tre centrala begrepp i kapitlet:

- levande dokumentation,
- dokumentation för användning,
- beslutslogg.

Levande dokumentation

Levande dokumentation är dokumentation som används, uppdateras och hjälper arbetet framåt. Den är inte levande bara för att den ligger i ett digitalt verktyg. Den är levande för att den har en funktion i vardagen.

Levande dokumentation kännetecknas av att den:

- har tydliga mottagare,
- uppdateras när beslut eller förståelse ändras,
- skiljer mellan beslut, antaganden och öppna frågor,
- är lätt att hitta,
- är tillräckligt kort för att faktiskt användas,
- är kopplad till arbetets flöde,
- hjälper både teamet och andra roller att förstå lösningen.

För Sara innebär det att hon inte behöver välja mellan “ingen dokumentation” och “fullständig dokumentation”. Hon behöver skapa en dokumentationsstruktur som rymmer olika typer av information på rätt nivå.

En enkel uppdelning kan vara:

Typ av innehåll	Syfte	Exempel
Stabil riktning	Visa vart lösningen är på väg	målbild, principer, centrala begränsningar
Aktuella beslut	Visa vad som är beslutat just nu	val av integrationsmönster, behörighetsprincip
Öppna frågor	Visa vad som ännu inte är avgjort	datakvalitet, ansvar för masterdata
Antaganden	Visa vad teamet tror men ännu inte vet	förväntad belastning, användarbeteende
Risker	Visa vad som behöver följas upp	personuppgifter, beroenden, teknisk skuld
Förvaltningskunskap	Visa vad som behövs över tid	driftpåverkan, ansvar, beroenden, kontaktvägar

Poängen är inte att skapa fler dokument. Poängen är att skapa begriplighet.

Dokumentation för användning

Dokumentation blir lätt abstrakt när den skrivs "för att den ska finnas". Ett mer användbart angreppssätt är att skriva dokumentation för en konkret användning.

Sara kan ställa fem frågor innan hon dokumenterar:

1. Vem ska använda detta?
2. Vilket beslut, vilken dialog eller vilket ansvar ska dokumentationen stödja?
3. Hur länge behöver informationen vara aktuell?
4. Hur ofta kommer den att ändras?
5. Vilken detaljnivå är tillräcklig?

Det gör stor skillnad.

En styrgrupp behöver kanske inte förstå alla tekniska detaljer i behörighetslösningen. Den behöver förstå vilken risknivå som accepteras, vilka vägval som påverkar framtida kostnad och vilka beslut som kräver mandat.

Ett utvecklingsteam behöver kanske inte en lång målarkitekturbeskrivning. Det behöver tydliga principer, gränssnitt, begränsningar och beslut som påverkar det dagliga designarbetet.

Förvaltningen behöver kanske inte alla diskussioner som ledde fram till lösningen. Den behöver förstå ansvar, beroenden, driftpåverkan, förändringspunkter och varför vissa vägval gjordes.

Säkerhetsfunktionen behöver kanske inte varje detalj i implementationen. Den behöver förstå informationsflöden, skyddsvärden, risker, kontroller och beslutade avvägningar.

Dokumentation för användning gör att arkitekten kan minska mängden text men öka värdet.

Beslutslogg

En beslutslogg är ett av arkitektens mest användbara verktyg i en agil miljö. Den behöver inte vara avancerad. Den behöver vara begriplig, aktuell och använd.

En beslutslogg svarar på frågor som:

- Vad har vi beslutat?
- Varför beslutade vi det?
- Vilka alternativ övervägde vi?
- Vilka konsekvenser accepterar vi?
- Vilka antaganden bygger beslutet på?
- När behöver beslutet omprövas?

Det viktiga är att beslutsloggen inte bara dokumenterar beslutets rubrik. Den behöver fånga sammanhanget.

Ett enkelt format kan vara:

```
## Beslut: [kort namn]
```

```
**Datum:**
```

```
**Status:** Föreslaget / Beslutat / Omprövas / Ersatt
```

****Sammanhang:****

Vilken situation eller vilket problem ledde till beslutet?

****Beslut:****

Vad har vi valt att göra?

****Motiv:****

Varför valde vi detta?

****Alternativ som övervägdes:****

Vilka andra vägar fanns?

****Konsekvenser:****

Vilka effekter, begränsningar eller risker accepterar vi?

****Antaganden:****

Vad tror vi, men vet ännu inte säkert?

****Omprövning:****

När eller under vilka förutsättningar ska beslutet ses över?

I en myndighetskontext kan en sådan beslutslogg vara särskilt värdefull. Den hjälper organisationen att visa att beslut inte har fattats slarvigt, även om de har fattats stegvis. Den gör det möjligt att förstå varför lösningen ser ut som den gör. Den hjälper nya personer att komma in i arbetet. Den minskar beroendet av muntligt minne.

Exempel: Myndigheten för samhällstjänster

Sara samlar representanter från teamet, förvaltningen, säkerhetsfunktionen och produktägaren Erik. Hon börjar inte med att föreslå ett nytt dokument. Hon börjar med att fråga vad de behöver kunna göra med dokumentationen.

Styrgruppen behöver förstå vilka större vägval som påverkar kostnad, risk och tidplan.

Förvaltningen behöver veta vilka delar av det gamla handläggningssystemet som påverkas och vilka nya beroenden som skapas.

Säkerhetsfunktionen behöver se hur behörighet, loggning och personuppgifter hanteras.

Teamet behöver veta vilka principer som gäller när de fattar designbeslut i vardagen.

Sara föreslår därför fyra enkla dokumentationsytor:

1. **En kort lösningsöversikt** som visar målbild, systemkontext, huvudflöden och centrala begränsningar.
2. **En beslutslogg** för viktiga arkitekturbeslut.
3. **En lista över öppna frågor och antaganden** som behöver följas upp.
4. **En förvaltningsnära konsekvensbild** som visar beroenden, ansvar och påverkan på befintliga system.

Hon är nog med att beskriva detta som arbetsmaterial, inte som en ny tung dokumentationsfas. Teamet uppdaterar vissa delar löpande. Sara tar ansvar för att beslutsloggen och lösningsöversikten hålls begripliga. Förvaltningen och säkerhetsfunktionen får kommentera när deras behov inte täcks.

Efter några veckor märker gruppen att diskussionerna förändras. Styrgruppen frågar inte längre bara efter "arkitekturdokumentet". Den frågar efter vilka beslut som är fattade, vilka risker som är öppna och vad som behöver mandat. Teamet börjar själva hänvisa till beslutsloggen när nya designfrågor uppstår. Säkerhetsfunktionen kommer in tidigare eftersom öppna riskfrågor blir synliga innan de blir sena hinder.

Dokumentationen har inte försvunnit. Den har blivit mer användbar.

Vanliga fallgropar

- Fallgrop: Att tro att agil utveckling betyder att dokumentation inte behövs.
 - Varför den uppstår: Team kan ha negativa erfarenheter av dokumentation som varit tung, sen eller oanvänd.
 - Vad arkitekten kan göra i stället: Prata om vilken dokumentation som behövs för beslut, ansvar och lärande, inte om dokumentation som generell princip.
- Fallgrop: Att försöka ersätta alla gamla dokument med ett nytt stort agilt dokument.
 - Varför den uppstår: Organisationen vill behålla tryggheten i ett samlat underlag.
 - Vad arkitekten kan göra i stället: Dela upp dokumentationen efter användning: beslut, riktning, öppna frågor, risker och förvaltningskonsekvenser.
- Fallgrop: Att dokumentera lösningen men inte besluten.
 - Varför den uppstår: Det är ofta lättare att beskriva vad som byggs än varför det blev så.
 - Vad arkitekten kan göra i stället: Använd beslutslogg för viktiga vägval, särskilt när beslutet påverkar risk, beroenden eller framtida handlingsutrymme.
- Fallgrop: Att dokumentationen bara uppdateras inför granskning.
 - Varför den uppstår: Dokumentation ses som ett godkännandeunderlag snarare än som en del av arbetet.
 - Vad arkitekten kan göra i stället: Koppla uppdatering av dokumentation till naturliga händelser: beslut, förändrade antaganden, nya risker och större designändringar.
- Fallgrop: Att skriva för många målgrupper samtidigt.
 - Varför den uppstår: Myndighetsmiljöer har många legitima intressenter med olika behov.
 - Vad arkitekten kan göra i stället: Skapa olika vyer eller nivåer av samma kunskap i stället för ett dokument som försöker passa alla.

Frågor att ställa i situationen

När dokumentation efterfrågas i ett agilt arbete kan arkitekten ställa frågor som hjälper organisationen att bli mer precis:

1. Vem behöver dokumentationen?
2. Vilket beslut eller vilket ansvar ska den stödja?
3. Vad behöver vara stabilt över tid?
4. Vad är just nu ett antagande?
5. Vilka beslut behöver kunna förstås i efterhand?
6. Vilka risker behöver vara synliga för fler än teamet?
7. Vilken dokumentation hjälper teamet att fatta bättre beslut i vardagen?
8. Vilken dokumentation behöver förvaltningen för att kunna ta ansvar senare?
9. Vad kan dokumenteras lättviktigt nu och fördjupas när det blir mer stabilt?
10. Hur vet vi att dokumentationen faktiskt används?

Reflektionsfrågor

1. Vilken dokumentation i din organisation används faktiskt, och vilken produceras mest för att den förväntas finnas?
2. När har du senast sett ett arkitekturdokument bli inaktuellt snabbare än arbetet hann använda det?
3. Vilka beslut i ditt nuvarande arbete skulle behöva dokumenteras bättre, inte bara som beslut utan med motiv och konsekvenser?
4. Vilka mottagare skriver du oftast för: teamet, styrning, förvaltning, säkerhet, juridik eller någon annan?
5. Vilken dokumentation skulle kunna kortas ned om syftet blev tydligare?
6. Var i ert utvecklingsflöde borde dokumentation uppdateras naturligt, utan att bli en separat fas?
7. Vilka öppna frågor eller antaganden borde göras mer synliga just nu?
8. Hur kan du som arkitekt hjälpa organisationen att prata om dokumentation som stöd för ansvar och lärande, snarare än som en administrativ börda?

Snabb sammanfattning

- Dokumentation är fortfarande viktig i agil utveckling, särskilt i myndighetsmiljö.
- Problemet är sällan dokumentation i sig, utan dokumentation som är för stor, för sen, för statisk eller skriven för otydliga mottagare.
- Levande dokumentation används, uppdateras och hjälper arbetet framåt.
- Dokumentation för användning börjar med frågan vem som behöver informationen och vilket arbete den ska stödja.
- En beslutslogg är ett enkelt men kraftfullt sätt att skapa spårbarhet kring arkitekturbeslut.
- Arkitekten kan bidra genom att skapa begriplighet, inte genom att producera mer text än situationen kräver.
- I en agil myndighetskontext behöver dokumentation stödja både flöde och ansvar.

Nästa steg

Dokumentation gör arkitekturbeslut, risker och ansvar synliga. Men vissa frågor kräver mer än dokumentation. I myndighetsutveckling behöver säkerhet, regel-

terlevnad, dataskydd och rättssäkerhet byggs in i arbetssättet, inte bara kontrolleras i efterhand.

Nästa kapitel handlar därför om **säkerhet, regelefterlevnad och arkitektur i kortare cykler**.

Kapitel 10: Säkerhet, regelefterlevnad och arkitektur i kortare cykler

Varför detta kapitel finns

I statlig myndighet är säkerhet, integritet, rättssäkerhet och regelefterlevnad inte sidofrågor. De är en del av uppdraget. Ett digitalt stöd som hanterar ärenden, personuppgifter, beslut eller samhällsviktig information behöver inte bara fungera tekniskt. Det behöver också kunna förklaras, granskas, skyddas och förvaltas över tid.

När utvecklingen blir mer agil uppstår därför en särskild spänning. Teamen vill leverera stegvis, lära sig av användning och minska ledtider. Säkerhets-, juridik- och dataskyddsfunktioner vill försäkra sig om att lösningen inte skapar oacceptabla risker. Arkitekten hamnar ofta mitt i denna spänning.

Det invanda svaret är ofta att försöka "säkerställa allt" tidigt eller att lägga in tydliga kontrollpunkter sent i flödet. Båda strategierna kan kännas ansvarsfulla. Men i kortare utvecklingscykler räcker de sällan. För mycket tidig kontroll bygger på antaganden som senare förändras. För sen kontroll upptäcker risker när det redan är dyrt att ändra.

Det här kapitlet handlar om hur arkitekten kan bidra till att säkerhet och regelefterlevnad blir en levande del av utvecklingsarbetet.

Kapitlets centrala skifte är:

Från säkerhet och regelefterlevnad som tidiga krav och sena kontroller till säkerhet och regelefterlevnad som återkommande riskdialog, designbegränsningar och inbyggd kontroll.

Situationen: alla vill göra rätt, men ingen vill stoppa flödet

Sara är med i ett planeringsmöte för den nya medborgartjänsten på Myndigheten för samhällstjänster. Teamet har byggt en första intern version där en medborgare kan följa status i sitt ärende. Tjänsten hämtar information från det äldre handläggningssystemet och visar en förenklad vy.

Erik, produktägaren, vill gå vidare med en pilot för en begränsad användargrupp.

Lina, teamets utvecklingsledare, säger att den tekniska lösningen är tillräckligt stabil för att testas.

Amir, som arbetar med informationssäkerhet och dataskyddsfrågor, är tveksam. Han frågar:

“Vilka personuppgifter exponeras? Hur vet vi att rätt person ser rätt ärenden? Hur loggar vi åtkomst? Finns det risk att vi visar mer information än nödvändigt?”

Teamet svarar att flera av frågorna finns med i backloggen. Erik säger att piloten är begränsad. Amir vill se tydligare underlag innan tjänsten används utanför teamet.

Sara känner igen situationen. Alla har rimliga perspektiv:

- Erik vill skapa nytta och få återkoppling.
- Lina vill undvika att utvecklingen fastnar i långa utredningar.
- Amir vill undvika att myndigheten tar risker som inte är förstådda.
- Sara vill att arkitekturen ska stödja både säkerhet och stegvis utveckling.

Frågan är inte om säkerhet och regelefterlevnad är viktiga. Frågan är hur de kommer in i arbetet på ett sätt som både skapar trygghet och gör fortsatt lärande möjligt.

Det invanda sättet att agera

I en mer fasorienterad utvecklingslogik hanteras säkerhet och regelefterlevnad ofta genom särskilda moment:

- krav på säkerhet och regelverk samlas in tidigt,
- lösningen analyseras mot dessa krav,
- risker dokumenteras,
- kontrollfunktioner granskar underlag,
- godkännande sker inför nästa fas, inför test eller inför produktionssättning.

Detta kan ge tydlighet. Det blir klart vem som behöver granska vad. Det finns underlag att hänvisa till. Ansvar kan dokumenteras. För en myndighet är detta ofta nödvändigt i någon form.

Men när utvecklingen sker i kortare cykler blir det problematiskt om säkerhet och regelefterlevnad bara kommer in som separata kontrollpunkter. Då uppstår lätt två oönskade mönster.

Det första mönstret är att allt försöker lösas för tidigt. Arkitekten, säkerhetsfunktionen och juridiken försöker förutse alla tänkbara risker innan teamet börjar bygga. Resultatet blir ofta omfattande analyser av en lösning som ännu inte finns, eller av antaganden som senare ändras.

Det andra mönstret är att kontrollen kommer in för sent. Teamet bygger vidare, och först nära leverans upptäcks att behörighet, loggning, informationsklassning, gallring, spårbarhet eller rättslig grund inte är tillräckligt genomtänkta. Då upplevs säkerhet och regelefterlevnad som bromsar, trots att frågorna egentligen borde ha påverkat designen från början.

Det invanda sättet kan därför skapa en falsk uppdelning:

- teamet “bygger”,

- arkitekten "designar",
- säkerhet och juridik "granskar",
- styrningen "godkänner".

I ett agilt arbetssätt behöver dessa aktiviteter i stället kopplas samman oftare och tidigare, utan att allt måste formaliseras lika tungt varje gång.

Varför det kan skapa friktion i agil utveckling

Friktionen uppstår ofta för att olika delar av organisationen arbetar med olika rytm.

Teamet arbetar i korta cykler. Det vill kunna dela upp arbetet, testa antaganden och leverera stegvis. Säkerhets- och regelefterlevnadsfrågor arbetar ofta med en annan logik: konsekvenser behöver förstås, ansvar behöver vara tydligt och vissa risker kan inte lämnas öppna.

När dessa rytmer inte möts skapas väntan och frustration. Teamet upplever att kontrollfunktioner ställer frågor som kommer för sent eller är för omfattande för den aktuella förändringen. Kontrollfunktionerna upplever att teamet rör sig för snabbt och att viktiga risker göms bakom ord som "pilot", "MVP" eller "första inkrement".

Arkitekten kan då lockas att gå in som översättare och problemlösare på egen hand. Sara kan till exempel börja skriva ett större säkerhets- och arkitekturunderlag för att lugna Amir och samtidigt försöka skydda teamets tempo. Det kan hjälpa kort-siktigt, men riskerar att förstärka bilden av att säkerhet är något arkitekten "tar hand om" vid sidan av teamets arbete.

I stället behöver arkitekten hjälpa organisationen att föra in säkerhet och regelefterlevnad i själva arbetssättet. Det betyder inte att alla i teamet ska bli jurister eller säkerhetsspecialister. Det betyder att riskfrågor, designbegränsningar, kontrollbehov och dokumentationskrav behöver bli synliga i rätt tid och på rätt nivå.

Tre begrepp är särskilt viktiga i det skiftet:

- **Inbyggd kontroll:** kontroller som byggs in i lösning, arbetssätt, automatisering eller återkommande dialog i stället för att bara ske som sena granskningar.
- **Kontinuerlig riskdialog:** återkommande samtal om risker, avvägningar och åtgärder mellan arkitektur, team, säkerhet, juridik, dataskydd och verksamhet.
- **Säkerhet som designbegränsning:** synsättet att säkerhet inte bara är krav att bocka av, utan begränsningar som påverkar hur lösningen bör utformas.

Ett mer agilt förhållningssätt

Ett mer agilt förhållningssätt börjar med att skilja mellan tre typer av frågor:

1. Frågor som måste vara tillräckligt utredda innan utveckling eller pilot får gå vidare.
2. Frågor som kan hanteras stegvis, om riskerna är synliga och avgränsade.
3. Frågor som bör byggas in i teamets löpande arbetssätt.

För Sara innebär det att hon inte bara frågar: "Är lösningen godkänd ur säkerhetssynpunkt?" Hon frågar också:

- Vilka risker påverkar arkitekturen redan nu?
- Vilka beslut är svåra att ändra senare?
- Vilka kontroller kan byggas in i lösningen?
- Vilka kontroller behöver ske genom granskning eller beslut?
- Vilka antaganden behöver testas innan vi skalar upp?
- Vilka roller behöver prata med varandra oftare?

Det handlar om att skapa en rytm där säkerhet och regelefterlevnad inte blir en separat fas, men heller inte reduceras till en punkt på en checklista.

1. Gör riskerna synliga tidigt, men på rätt detaljnivå

Tidigt i arbetet behöver Sara inte ha alla svar. Men hon behöver hjälpa teamet att se vilka säkerhets- och regelefterlevnadsfrågor som kan påverka lösningens form.

För medborgartjänsten kan det handla om:

- vilken information som visas för medborgaren,
- hur identitet och behörighet säkerställs,
- vilka händelser som behöver loggas,
- om tjänsten hanterar känsliga personuppgifter,
- hur länge information behöver sparas,
- vilka integrationer som öppnar nya angreppsytor,
- om det finns beslut eller statusuppgifter som kan misstolkas,
- vilka fel som kan få rättsliga eller praktiska konsekvenser för individen.

Detta behöver inte innebära ett stort dokument i första steget. Det kan börja som en enkel riskkarta: vilka frågor är kritiska, vilka är osäkra, vilka kan vänta och vilka behöver en särskild dialog?

Det viktiga är att riskerna blir synliga medan designen fortfarande kan påverkas.

2. Behandla säkerhet som en del av lösningens form

Ibland beskrivs säkerhet som något som "läggs till" efteråt. Det leder lätt fel. Behörighet, loggning, informationsflöden, dataminimering, integrationer och spårbarhet påverkar lösningens struktur.

För arkitekten betyder det att säkerhetsfrågor behöver översättas till designfrågor.

Exempel:

- Om endast vissa handläggningsuppgifter får visas för medborgaren behöver lösningen ha en tydlig informationsmodell och filtrering.
- Om åtkomst måste kunna följas upp behöver loggning vara en del av designen, inte något som läggs till inför driftsättning.
- Om olika ärendetyper har olika sekretessnivåer behöver behörighetsmodellen stödja den variationen.
- Om externa integrationer används behöver felhantering och ansvarsfördelning vara tydlig.
- Om information inte ska sparas längre än nödvändigt behöver livscykel och gallring påverka både dataflöde och lagring.

När säkerhet behandlas som designbegränsning blir den inte bara ett kontrollkrav. Den hjälper teamet att forma en lösning som är möjlig att använda ansvarsfullt.

3. Skapa kontinuerlig riskdialog

Kontinuerlig riskdialog innebär inte att alla riskfrågor ska diskuteras på varje möte. Det innebär att det finns en återkommande, lättillgänglig form för att lyfta risker när de påverkar arbetet.

Sara kan till exempel hjälpa till att etablera en enkel rutin:

- Vid större förändringar i informationsflöde, behörighet eller integration bokas en kort riskdialog.
- Teamet tar med aktuell skiss, öppna frågor och tänkt nästa steg.
- Amir eller annan specialist hjälper till att sortera risker och avgöra vad som kräver mer underlag.
- Beslut och antaganden dokumenteras kort i beslutsloggen eller arkitekturdokumentationen.
- Frågor som inte behöver lösas direkt läggs som synliga uppföljningspunkter.

Det viktiga är att dialogen sker medan den fortfarande kan påverka design och prioritering. Om säkerhetsfunktionen bara ser lösningen när teamet vill gå i produktion blir dialogen nästan alltid svårare.

4. Bygg in kontroll där det går

Inbyggd kontroll kan vara teknisk, organisatorisk eller dokumentationsmässig.

Teknisk inbyggd kontroll kan till exempel vara:

- automatiska kontroller i bygg- och leveransflöden,
- loggning av känsliga händelser,
- behörighetskontroller som återanvänds konsekvent,
- testfall för säkerhetskritiska regler,
- övervakning av avvikelser,
- konfigurationsstyrning.

Organisatorisk inbyggd kontroll kan vara:

- att säkerhet och dataskydd deltar i återkommande riskdialoger,
- att riskfrågor finns med i refinement för relevanta backloggposter,
- att teamet har tydliga kriterier för när specialiststöd ska kopplas in,
- att produktägaren förstår vilka riskbeslut som behöver eskaleras.

Dokumentationsmässig inbyggd kontroll kan vara:

- en beslutslogg för viktiga risk- och arkitekturbeslut,
- en enkel informationsflödesbild som hålls uppdaterad,
- en lista över öppna riskantaganden,
- tydliga länkar mellan krav, designbeslut och riskåtgärder.

Poängen är inte att all kontroll ska automatiseras eller förenklas. Poängen är att kontrollen ska finnas där den gör mest nytta och inte enbart som sen granskning.

5. Skilj mellan pilot, experiment och skarp användning

I agila sammanhang används ofta ord som pilot, experiment, MVP och första version. De kan vara värdefulla, men de får inte bli ett sätt att otydliggöra ansvar.

Sara behöver därför hjälpa organisationen att vara tydlig med vad ett steg faktiskt innebär.

En intern teknisk prototyp har en typ av risk. En pilot med verkliga handläggare har en annan. En medborgartjänst som visar verklig ärendestatus för en begränsad grupp har en tredje. En fullskalig produktionssättning har ytterligare en annan riskbild.

Frågan är inte bara hur liten leveransen är. Frågan är vilken information, vilka användare, vilka beslut och vilka konsekvenser som berörs.

Ett begränsat steg kan absolut vara rätt väg. Men begränsningen behöver vara verklig och begriplig:

- Vilka användare omfattas?
- Vilka data används?
- Vilka funktioner är aktiva?
- Vilka beslut kan påverkas?
- Hur upptäcker vi fel?
- Hur avbryter eller rullar vi tillbaka?
- Vem äger riskbeslutet?

När dessa frågor är tydliga blir stegvis leverans inte ett sätt att komma runt ansvar. Det blir ett sätt att ta ansvar med kontrollerat lärande.

Exempel: Myndigheten för samhällstjänster

Efter mötet samlar Sara, Lina, Erik och Amir en kort arbetsdialog. Sara ritar upp tre enkla vyer på en digital whiteboard:

1. Vilken information medborgaren ska se.
2. Varifrån informationen hämtas.
3. Vilka beslut och kontroller som krävs innan nästa steg.

Det visar sig snabbt att ordet "ärendestatus" betyder olika saker. För Erik betyder det att medborgaren ska kunna se om ärendet är mottaget, under handläggning eller beslutat. För teamet betyder det ett tekniskt statusfält från det äldre systemet. För Amir betyder det en fråga om vilken information som kan avslöja mer än myndigheten avsett.

Sara ställer frågan:

"Vilken minsta statusinformation ger medborgaren nytta utan att vi exponerar känsliga detaljer?"

Det leder till ett annat designalternativ. I stället för att visa det interna statusfältet direkt skapar teamet en förenklad extern statusmodell. Den visar färre lägen, med språk som är begripligt för medborgaren och som inte avslöjar interna handläggningsdetaljer.

Amir påpekar att åtkomst behöver loggas, särskilt när ärenden innehåller skyddsvärd information. Lina svarar att teamet redan har teknisk loggning, men att den inte är utformad för uppföljning av medborgaråtkomst. Sara fångar det som ett arkitekturbeslut:

"Medborgaråtkomst till ärendestatus ska loggas som en verksamhetsrelevant händelse, inte enbart som teknisk systemlogg."

Beslutet läggs i beslutsloggen med tre delar:

- varför beslutet fattades,
- vilken konsekvens det får för designen,
- vilka frågor som fortfarande är öppna.

Erik vill fortfarande göra piloten. Amir säger att det kan vara möjligt om piloten begränsas till ärenden utan särskilda skyddsmarkeringar, om åtkomst sker via befintlig säker inloggning, om loggningen är på plats och om teamet dokumenterar kvarvarande risker.

Sara sammanfattar:

“Vi stoppar inte piloten, men vi gör den smalare och tydligare. Vi använder den för att lära, inte för att smyga ut en full lösning.”

Det blir ett konkret exempel på inbyggd kontroll och kontinuerlig riskdialog. Säkerhet och regelefterlevnad blir inte en sen spärr. De påverkar själva utformningen av nästa steg.

Vanliga fallgropar

- Fallgrop: Att använda “agilt” som skäl för att skjuta upp säkerhetsfrågor.
 - Varför den uppstår: Teamet vill undvika tidiga utredningar och tänker att detaljer kan lösas senare.
 - Vad arkitekten kan göra i stället: Skilj mellan frågor som kan vänta och frågor som påverkar lösningens grundläggande form.
- Fallgrop: Att försöka säkerställa allt innan något får byggas.
 - Varför den uppstår: Osäkerhet känns riskabel, särskilt i myndighetsmiljö.
 - Vad arkitekten kan göra i stället: Identifiera vilka risker som måste hanteras före nästa steg och vilka som kan undersökas genom kontrollerad utveckling.
- Fallgrop: Att behandla säkerhet som en checklista vid sidan av designen.
 - Varför den uppstår: Säkerhetskrav dokumenteras ofta separat från arkitektur- och designbeslut.
 - Vad arkitekten kan göra i stället: Översätt säkerhetskrav till designbegränsningar, principer och konkreta beslut.
- Fallgrop: Att kalla något pilot utan att tydliggöra risknivån.
 - Varför den uppstår: Ordet pilot låter begränsat och ofarligt.
 - Vad arkitekten kan göra i stället: Beskriv exakt vilka användare, data, funktioner, miljöer och konsekvenser piloten omfattar.
- Fallgrop: Att göra säkerhetsfunktionen till en sen godkännare.
 - Varför den uppstår: Organisationen har invanda gransknings- och beslutsvägar.
 - Vad arkitekten kan göra i stället: Skapa återkommande riskdialoger där specialister kommer in när designen fortfarande går att påverka.
- Fallgrop: Att arkitekten själv försöker bära hela risköversättningen.
 - Varför den uppstår: Arkitekten vill skydda teamets flöde och samtidigt tillfredsställa kontrollbehoven.
 - Vad arkitekten kan göra i stället: Gör risker, antaganden och beslut synliga så att rätt roller kan ta ansvar tillsammans.

Frågor att ställa i situationen

När säkerhet, regelefterlevnad och arkitektur behöver hanteras i kortare cykler kan arkitekten använda följande frågor:

1. Vilken information hanteras, visas, lagras eller skickas vidare?
2. Vilka användare, roller eller externa aktörer berörs?
3. Vilka fel kan få störst konsekvens för individ, verksamhet eller myndighet?
4. Vilka säkerhets- och regelkrav påverkar lösningens struktur redan nu?
5. Vilka beslut är svåra att ändra senare?
6. Vilka risker kan vi minska genom att begränsa nästa steg?
7. Vilka kontroller kan byggas in i lösningen eller arbetssättet?
8. Vilka risker behöver dokumenteras som medvetna avvägningar?
9. Vilka specialister behöver delta i dialogen innan vi går vidare?
10. Vad behöver vara sant för att nästa inkrement ska vara ansvarsfullt?

Frågorna är inte tänkta som en komplett granskningsmall. De är ett sätt att hjälpa arkitekten föra rätt samtal i rätt tid.

Reflektionsfrågor

1. När kommer säkerhet, juridik, dataskydd eller regelefterlevnad typiskt in i dina utvecklingsflöden i dag?
2. Vilka frågor brukar upptäckas för sent?
3. Vilka frågor försöker organisationen ibland lösa för tidigt, innan lösningen är tillräckligt begriplig?
4. Hur kan du som arkitekt göra risker synliga utan att omedelbart skapa en tung utredningsfas?
5. Vilka säkerhets- eller regelefterlevnadskrav påverkar arkitekturen i dina vanligaste lösningar?
6. Vilka kontroller skulle kunna byggas in i teamets arbetssätt i stället för att göras som sena granskningar?
7. Hur tydligt skiljer ni i dag mellan prototyp, pilot, begränsad produktionssättning och fullskalig användning?
8. Vilken återkommande riskdialog skulle skapa mest nytta i din organisation just nu?

Snabb sammanfattning

- I myndighetsmiljö är säkerhet, integritet, rättssäkerhet och regelefterlevnad en del av lösningens kvalitet, inte externa tillägg.
- Kortare utvecklingscykler kräver att riskfrågor kommer in återkommande, inte bara tidigt eller sent.
- Arkitekten behöver hjälpa team och kontrollfunktioner att mötas medan designen fortfarande kan påverkas.
- Säkerhet bör ofta behandlas som designbegränsning: något som formar lösningens struktur, informationsflöden, behörighet, loggning och spårbarhet.
- Inbyggd kontroll kan finnas i teknik, arbetssätt och dokumentation.
- En pilot eller ett experiment behöver tydliga gränser för användare, data, funktioner, risker och ansvar.

- Målet är inte att minska myndighetens ansvar, utan att ta ansvar på ett sätt som fungerar i iterativ utveckling.

Nästa steg

När säkerhet och regelefterlevnad blir en del av det löpande arbetet blir nästa fråga hur beslut ska fattas utan att organisationen fastnar i väntan. Arkitekturen behöver forum, mandat och styrning, men dessa behöver stödja flöde snarare än skapa parallella köer.

Nästa kapitel handlar därför om arkitekturforum, styrning och beslut utan flaskhalsar.

Kapitel 11: Arkitekturforum, styrning och beslut utan flaskhalsar

Varför detta kapitel finns

I många myndigheter finns etablerade forum för arkitektur, säkerhet, portföljstyrning, prioritering och förändringsbeslut. De finns ofta av goda skäl. Myndigheten behöver kunna ta ansvar för helheten, följa regler, använda resurser klokt och undvika att enskilda initiativ skapar problem för andra delar av verksamheten.

När utvecklingen blir mer agil uppstår ändå en återkommande spänning. Teamen behöver kunna fatta beslut nära arbetet och röra sig framåt utan lång väntan. Samtidigt behöver organisationen säkerställa att viktiga arkitekturfrågor inte löses lokalt på ett sätt som skapar risker, dubbelarbete eller långsiktiga låsningar.

Arkitekten hamnar ofta mellan dessa behov. Å ena sidan vill Sara inte att teamen ska behöva vänta på forum för frågor de själva kan hantera. Å andra sidan vet hon att vissa beslut påverkar fler än ett team, flera system, säkerhet, förvaltning och framtida handlingsutrymme. Då räcker det inte att säga att teamet "äger lösningen".

Det här kapitlet handlar om hur arkitekturforum och styrning kan stödja agil utveckling utan att bli en flaskhals. Fokus ligger inte på att avskaffa forum, utan på att tydliggöra vad de ska vara bra på.

Kapitlets centrala skifte är:

Från arkitekturforum som granskande godkännandepunkt till arkitekturforum som stödstruktur för riktning, mandat, lärande och beslut i rätt tid.

Situationen: forumet både behövs och bromsar

På Myndigheten för samhällstjänster finns ett arkitekturforum som träffas varannan torsdag. Forumet består av flera arkitekter, representanter från IT-styrning, säkerhet, förvaltning och ibland verksamhet. I forumets uppdrag står att det ska "säkerställa arkitekturell kvalitet och följsamhet mot myndighetens principer".

I praktiken har forumet blivit en plats där många typer av frågor hamnar:

- team som vill få en lösningsskiss godkänd,
- projekt som behöver ett arkitekturutlåtande inför styrgrupp,

- integrationsfrågor som påverkar flera system,
- undantag från tekniska riktlinjer,
- säkerhetsfrågor som inte hittat någon annan beslutsväg,
- förvaltningsfrågor som kräver samordning,
- frågor där ingen riktigt vet vem som har mandat.

Sara ser att forumet gör nytta. Det fångar upp beroenden, skapar gemensam riktning och gör att viktiga frågor inte försvinner i lokala teamdiskussioner.

Samtidigt börjar teamen uppleva forumet som en kö.

Lina, teamets utvecklingsledare, säger:

“Vi vill gärna göra rätt, men om varje större fråga måste vänta på nästa forum tappar vi tempo. Ibland behöver vi bara veta om vi är på rätt väg.”

Erik, produktägaren, uttrycker en annan frustration:

“Vi får ofta nya frågor från forumet, men inte alltid ett tydligt beslut. Då blir det svårt att planera nästa steg.”

Amir, som representerar säkerhet och dataskydd, tycker samtidigt att teamen ibland kommer för sent:

“När frågan väl kommer hit har lösningen redan börjat byggas. Då blir det svårt att påverka utan att det upplevs som stopp.”

Sara känner igen mönstret. Forumet är inte problemet i sig. Problemet är att det används för många olika syften samtidigt: rådgivning, granskning, beslut, förankring, eskalering, informationsdelning och ibland risköverföring.

När ett forum blir allt detta på en gång blir det lätt en flaskhals.

Det invanda sättet att agera

I en mer fasorienterad utvecklingslogik är det vanligt att forum används som tydliga kontroll- och beslutspunkter. Ett projekt tar fram underlag, presenterar en lösning, får synpunkter och går sedan vidare till nästa fas eller beslut.

Det kan fungera väl när arbetet rör sig i större steg och när underlagen är relativt stabila. Forumet får då en tydlig roll: att granska, kvalitetssäkra, förankra och fatta beslut innan nästa större åtagande görs.

Det invanda sättet att agera kan se ut så här:

1. Teamet eller projektet tar fram en lösningsbeskrivning.
2. Arkitekten sammanställer risker, beroenden och vägval.
3. Ärendet bokas in till arkitekturforum.
4. Forumet ställer frågor och begär kompletteringar.
5. Teamet återkommer vid nästa tillfälle.
6. Beslut eller rekommendation dokumenteras.
7. Arbetet går vidare.

I en fasorienterad logik kan detta skapa trygghet. Det finns en tydlig punkt där organisationen samlar sig, granskar och bekräftar att lösningen är tillräckligt genomtänkt.

Men när utvecklingen sker i kortare cykler förändras förutsättningarna. Frågorna kommer oftare, underlagen är mer rörliga och många beslut är mindre än de beslut forumet tidigare var byggt för. Då riskerar det invanda arbetssättet att ge samma processvikt till små, medelstora och stora beslut.

Det leder till tre vanliga problem.

För det första blir forumet överbelastat. Om alla frågor ska dit får de frågor som verkligen behöver gemensam hantering konkurrera med frågor som kunde ha lösts närmare teamet.

För det andra blir beslut för sena. När forumet väl behandlar frågan kan teamet redan ha gjort antaganden, byggt delar av lösningen eller planerat kommande arbete utifrån en riktning som ännu inte är förankrad.

För det tredje blir ansvar otydligt. Om forumet förväntas godkänna mycket kan team och arkitekter börja vänta på forumet även i frågor där de egentligen har tillräcklig kunskap och mandat.

Varför det kan skapa friktion i agil utveckling

Agil utveckling bygger inte på att alla får bestämma allt lokalt. Den bygger heller inte på att styrning försvinner. Men den förutsätter att beslut kan fattas med rimlig hastighet, nära den kunskap som behövs och med lagom mycket koordinering.

När arkitekturforum blir en generell godkännandepunkt uppstår flera former av friktion.

Väntan ersätter ansvar

Om teamet vet att forumet ändå ska godkänna en lösning kan det bli rationellt att vänta. Frågan flyttas från dem som har mest aktuell kunskap till dem som har formellt mandat. Då tappar organisationen både tempo och lärande.

Det kan också påverka arkitektens roll. Sara riskerar att bli den som "tar frågan till forumet" i stället för den som hjälper teamet att förstå vilket beslut de själva kan fatta, vilket beslut som behöver förankras och vilket beslut som bör eskaleras.

Forumet får frågor i fel mognadsgrad

Ett forum kan få frågor för tidigt eller för sent.

För tidigt betyder att frågan ännu är så osäker att forumet inte kan fatta ett meningsfullt beslut. Då blir mötet mest en diskussion som avslutas med att teamet behöver återkomma.

För sent betyder att lösningen redan har fått form. Då blir forumets synpunkter svåra att ta om hand utan omarbete, irritation eller förhandling.

I båda fallen uppstår friktion eftersom forumet inte är kopplat till rätt beslutstidpunkt.

Granskning blandas ihop med stöd

Ett forum som både ska stötta, granska och besluta behöver vara tydligt med vilket läge det befinner sig i. Annars kan teamet komma för rådgivning men uppleva sig granskat. Eller så kommer teamet för beslut men får i stället allmänna råd.

Det skapar osäkerhet:

- Är detta ett beslut eller en rekommendation?
- Behöver teamet ändra lösningen eller bara överväga synpunkterna?
- Vem äger risken om teamet går vidare?
- Vad betyder det att forumet "inte har några invändningar"?

Agilt arbete kräver inte mindre tydlighet än fasorienterat arbete. Ofta kräver det mer tydlighet, eftersom besluten är fler, mindre och mer utspridda.

Lokala beslut får oavsiktliga helhetseffekter

Om forumet å andra sidan kopplas bort helt kan team fatta beslut som var för sig verkar rimliga, men som tillsammans skapar problem:

- flera team bygger egna integrationsmönster,
- samma information modelleras på olika sätt,
- säkerhetslösningar blir inkonsekventa,
- förvaltningen får fler tekniska varianter att hantera,
- myndigheten tappar överblick över kritiska beroenden.

Det här är skälet till att kapitlets budskap inte är "mindre styrning". Budskapet är bättre placerad styrning.

Ett mer agilt förhållningssätt

Ett mer agilt förhållningssätt till arkitekturforum börjar med att skilja på fyra olika behov:

1. behov av riktning,
2. behov av stöd,
3. behov av beslut,
4. behov av lärande.

Ett forum som blandar ihop dessa behov blir lätt otydligt. Ett forum som skiljer dem åt kan bli ett viktigt stöd för agil utveckling.

Forumet som riktning

Vissa frågor bör inte behöva lösas om och om igen av varje team. Myndigheten behöver gemensamma principer, målarkitekturer, tekniska riktlinjer och överenskomna mönster.

Här kan arkitekturforumet skapa riktning genom att svara på frågor som:

- Vilka integrationsprinciper ska vi normalt följa?
- Vilka tekniska plattformar är strategiska?
- Vilka avvikelser är acceptabla och hur dokumenteras de?
- Vilka kvalitetsattribut är särskilt viktiga i våra digitala tjänster?

- Vilka arkitekturella risker ser vi över flera initiativ?

När forumet är bra på riktning behöver färre frågor gå dit för godkännande.

Forumet som stöd

Många frågor behöver inte beslut direkt. De behöver klargörande, gemensam förståelse eller tidig dialog.

Ett stödjande forum kan hjälpa team att tänka tidigare:

- "Vilka beroenden har ni identifierat?"
- "Vilka beslut är svårreversibla?"
- "Vilka risker behöver säkerhet eller förvaltning titta på?"
- "Vilken dokumentation behöver finnas för att andra ska kunna förstå beslutet?"
- "Vilka alternativ har ni valt bort, och varför?"

När forumet används tidigt som stöd kan senare godkännande ofta bli mindre dramatiskt, eller helt onödigt.

Forumet som beslutspunkt

Vissa frågor behöver ändå formellt beslut eller tydlig förankring. Det gäller särskilt beslut som påverkar flera team, myndighetens gemensamma plattformar, informationshantering, säkerhet, långsiktiga kostnader eller avsteg från beslutade principer.

Men då behöver forumet vara tydligt:

- Vilket beslut ska fattas?
- Vilka alternativ finns?
- Vad rekommenderas?
- Vilka konsekvenser får beslutet?
- Vem har mandat att fatta beslutet?
- Hur dokumenteras beslutet?
- Hur kommuniceras det till berörda?

Ett agilt arkitekturforum bör inte fatta fler beslut än nödvändigt. Men de beslut det faktiskt fattar behöver vara tydliga nog att hjälpa arbetet framåt.

Forumet som lärande

När utvecklingen sker iterativt behöver även arkitekturforum lära sig. Forumet bör inte bara fråga om teamen följer riktlinjer. Det bör också fånga upp när riktlinjerna inte längre hjälper.

Exempel på lärandefrågor:

- Vilka frågor kommer ofta tillbaka?
- Var skapar våra principer otydlighet?
- Vilka beslut hamnar för ofta för sent?
- Vilka typer av ärenden fastnar i forumet?
- Vilka beroenden borde lösas strukturellt i stället för att hanteras ärende för ärende?

På så sätt blir forumet inte bara en plats där team granskas. Det blir en plats där organisationens arkitekturförmåga utvecklas.

Beslutsflöde

Ett centralt begrepp i kapitlet är beslutsflöde.

Med beslutsflöde menar vi hur en fråga rör sig från att den upptäcks till att den blir ett fattat, kommunicerat och omsatt beslut.

I ett svagt beslutsflöde händer ofta något av följande:

- ingen vet om frågan är ett beslut, en risk eller bara en diskussion,
- frågan studsar mellan team, arkitekt, forum och styrgrupp,
- beslut fattas men kommuniceras inte tydligt,
- beslut dokumenteras men påverkar inte arbetet,
- forumet ger synpunkter men ingen vet om de är bindande,
- samma fråga återkommer i flera forum.

Ett starkare beslutsflöde gör det tydligt:

1. Vad är frågan?
2. Varför behöver den hanteras?
3. Vilken typ av beslut handlar det om?
4. Vem har kunskap?
5. Vem har mandat?
6. När behöver beslutet fattas?
7. Hur ska beslutet dokumenteras?
8. Hur ska beslutet följas upp?

För Sara innebär detta att hennes uppgift inte bara är att förbereda underlag till forumet. Hon behöver hjälpa organisationen att forma själva vägen från fråga till beslut.

Det kan vara en enkel förbättring att varje arkitekturärende till forumet tydligt märks med ärendetyp:

Ärendetyp	Syfte	Exempel
Tidig dialog	Få perspektiv innan vägval låses	"Vi ser tre möjliga integrationsmönster och vill pröva resonemanget."
Rekommendation	Få stöd för en föreslagen riktning	"Teamet rekommenderar alternativ B, men vill förankra konsekvenserna."
Beslut	Få ett tydligt ja, nej eller villkorat beslut	"Vi behöver besluta om undantag från standardmönstret."
Eskalering	Lyfta en fråga som inte kan lösas lokalt	"Två initiativ prioriterar samma plattformresurs."

Lärande	Återföra erfarenheter till gemensamma principer	“Det beslutade mönstret fungerar inte bra i den här typen av ärenden.”
---------	---	--

En sådan enkel klassning minskar risken att forumet behandlar alla frågor på samma sätt.

Mandat nära kunskap

Ett annat centralt begrepp är mandat nära kunskap.

Principen betyder att beslut bör fattas så nära relevant kunskap som möjligt, men inom tydliga ramar. Det betyder inte att alla beslut ska fattas av teamet. Det betyder att varje beslut bör placeras där kombinationen av kunskap, konsekvensförståelse och mandat är bäst.

Vissa beslut hör hemma nära teamet:

- val av intern kodstruktur,
- detaljer i implementationen,
- mindre designbeslut som följer etablerade principer,
- tekniska förbättringar inom teamets komponenter,
- justeringar som är lätta att ändra senare.

Andra beslut behöver arkitektens aktiva stöd:

- mönster för integration,
- informationsmodellering som påverkar flera system,
- avvägningar mellan kortsiktig leverans och teknikskuld,
- designbeslut med påverkan på säkerhet, spårbarhet eller förvaltning,
- frågor där flera team behöver samordnas.

Ytterligare andra beslut behöver forum eller styrning:

- avsteg från gemensamma arkitekturprinciper,
- val av ny strategisk plattform,
- beslut som påverkar flera förvaltningsområden,
- riskacceptans på nivåer teamet inte äger,
- större beroenden mellan initiativ,
- beslut som påverkar myndighetens långsiktiga kostnader eller ansvar.

Sara kan hjälpa genom att inte fråga “ska detta till forumet?” för tidigt. En bättre första fråga är:

“Var finns den kunskap och det mandat som behövs för att fatta ett bra beslut?”

Ibland är svaret teamet. Ibland är svaret arkitekten tillsammans med teamet. Ibland är svaret ett forum. Ibland är svaret att forumet inte ska fatta beslutet, men behöver skapa riktning så att andra kan fatta beslut snabbare.

Arkitekturforum som stödstruktur

Ett arkitekturforum blir en stödstruktur när det hjälper arbetet framåt även när det inte fattar beslut.

Det kan göra det genom att:

- tydliggöra principer,
- förenkla beslutsvägar,
- synliggöra beroenden,
- hjälpa team att hitta rätt personer tidigt,
- upptäcka återkommande hinder,
- minska behovet av sena granskningar,
- skapa gemensamt språk för arkitekturbeslut,
- följa upp om beslut faktiskt fungerar.

Ett stödande forum är inte passivt. Det är inte bara ett rådgivande samtal utan konsekvens. Men dess primära värde är inte att säga ja eller nej. Värdet är att öka organisationens förmåga att fatta bra beslut där besluten hör hemma.

Det kan låta mindre kraftfullt än ett godkännandeorgan, men i agil utveckling kan det vara mer effektivt.

Om forumet lyckas skapa tydliga principer, snabba återkopplingsvägar och rätt mandat nära kunskap behöver färre frågor vänta. Samtidigt får myndigheten bättre helhet än om varje team löser allt själv.

Exempel: Myndigheten för samhällstjänster

Efter flera möten där ärenden fastnat bestämmer sig Sara för att föreslå en förändring i hur arkitekturforumet arbetar.

Hon börjar inte med att föreslå en ny organisationsmodell. I stället tar hon med tre observationer till forumet:

1. Många ärenden kommer in som "beslutsärenden", men visar sig egentligen vara behov av tidig dialog.
2. Teamen upplever att forumet ger många synpunkter men ibland otydliga beslut.
3. Forumet lägger mycket tid på frågor som kunde ha hanterats med tydligare principer eller mandat.

Sara föreslår tre små förändringar.

För det första ska varje ärende märkas med ärendetyp: tidig dialog, rekommendation, beslut, eskalering eller lärande.

För det andra ska varje beslutsärende innehålla en kort beslutsruta:

Beslut som behövs:

Rekommenderat alternativ:

Alternativ som valts bort:

Konsekvenser:

Berörda team/system:

Föreslaget beslutsmandat:

Hur beslutet dokumenteras:

För det tredje ska forumet avsluta varje ärende med en tydlig formulering:

- "Beslut fattat: ..."
- "Rekommendation: ..."
- "Ingen invändning, teamet har mandat att gå vidare."
- "Komplettering behövs före beslut: ..."
- "Frågan hör hemma i annat forum eller hos annan beslutsägare: ..."

Vid nästa möte tar Lina upp en fråga om hur den nya medborgartjänsten ska hämta ärendestatus från det äldre handläggningssystemet. Tidigare hade teamet kanske försökt få hela lösningen godkänd. Nu märks ärendet som tidig dialog.

Teamet presenterar tre alternativ:

1. direkt anrop mot det äldre systemet,
2. ett nytt API-lager,
3. en händelsebaserad lösning där statusändringar publiceras till en gemensam komponent.

Forumet fattar inget slutligt beslut. Däremot hjälper det teamet att se att alternativ 1 kan vara snabbt men riskerar stark koppling, att alternativ 2 kan vara rimligt som första steg och att alternativ 3 kan vara en långsiktig riktning men kräver mer samordning.

Sara sammanfattar:

"Teamet har mandat att gå vidare och testa alternativ 2 som begränsad lösning, men beslutet ska dokumenteras som ett arkitekturbeslut. Forumet vill se en återkoppling om vilka beroenden som kvarstår och om lösningen bör utvecklas mot den händelsebaserade riktningen senare."

Det viktiga är inte att forumet löser allt. Det viktiga är att teamet kan gå vidare med tydligare ramar, och att organisationen lär sig något som kan påverka framtida principer.

Vanliga fallgropar

- Fallgrop: Alla större frågor ska till forumet.
 - Varför den uppstår: Organisationen vill undvika lokala beslut som skapar långsiktiga problem.
 - Vad arkitekten kan göra i stället: Hjälp till att skilja mellan beslut som kräver forum, frågor som bara behöver tidig dialog och beslut som teamet kan fatta inom givna principer.
- Fallgrop: Forumet ger synpunkter men inga beslut.
 - Varför den uppstår: Det är oklart om forumet är rådgivande, beslutande eller granskande i det aktuella ärendet.
 - Vad arkitekten kan göra i stället: Kräv att ärendetyp och önskat utfall är tydligt innan frågan tas upp.
- Fallgrop: Teamen kommer för sent.
 - Varför den uppstår: Forumet upplevs som en sen kontrollpunkt snarare än ett stöd i tidiga vägval.
 - Vad arkitekten kan göra i stället: Skapa lättare former för tidig dialog, till exempel korta arkitektursamtal innan formella ärenden.
- Fallgrop: Forumet blir en ersättning för mandat.

- Varför den uppstår: Ingen vill äga svåra beslut, och forumet blir en plats där ansvar sprids ut.
- Vad arkitekten kan göra i stället: Synliggör vem som har kunskap, vem som har mandat och vem som behöver acceptera risk.
- Fallgrop: Styrning tolkas som godkännande.
 - Varför den uppstår: Organisationen har lärt sig att kvalitetssäkring sker genom att någon högre upp säger ja.
 - Vad arkitekten kan göra i stället: Visa att styrning också kan ske genom principer, ramar, beslutsloggar, återkoppling och uppföljning.
- Fallgrop: Forumet optimerar för enskilda ärenden.
 - Varför den uppstår: Varje ärende behandlas för sig, utan att återkommande mönster fångas.
 - Vad arkitekten kan göra i stället: Avsätt tid för lärande: vilka ärenden återkommer, vilka principer saknas och vilka hinder behöver lösas strukturellt?

Frågor att ställa i situationen

När du som arkitekt står inför frågan om ett ärende ska till forum, kan följande frågor hjälpa:

1. Vad är det egentliga beslutet?
2. Är frågan mogen för beslut, eller behöver den tidig dialog?
3. Vilka alternativ finns?
4. Vilka konsekvenser får beslutet för andra team, system eller förvaltningsområden?
5. Är beslutet reversibelt eller svårreversibelt?
6. Vem har bäst kunskap om frågan?
7. Vem har mandat att acceptera konsekvenserna?
8. Vilka principer eller tidigare beslut påverkar frågan?
9. Behöver forumet fatta beslut, ge rekommendation, undanröja hinder eller bara bli informerat?
10. Hur ska beslutet dokumenteras så att andra kan förstå det senare?

En särskilt användbar fråga är:

“Vad behöver hända efter forumet för att arbetet faktiskt ska kunna gå vidare?”

Om svaret är oklart är ärendet troligen inte tillräckligt tydligt formulerat.

Reflektionsfrågor

1. Vilka typer av frågor hamnar oftast i arkitekturforum eller motsvarande styrningsforum i din organisation?
2. Vilka av dessa frågor behöver verkligen forumets beslut?
3. Vilka frågor skulle kunna hanteras snabbare med tydligare principer eller mandat nära teamen?
4. När upplever teamen att forumet hjälper dem framåt?
5. När upplever teamen att forumet främst skapar väntan?
6. Hur tydligt skiljer ni mellan rådgivning, rekommendation och beslut?
7. Vilka beslut fattas i dag för långt från den kunskap som behövs?

8. Vilka återkommande ärenden visar att organisationen saknar en gemensam princip eller riktning?
9. Hur kan du som arkitekt bidra till bättre beslutsflöde utan att själv bli den nya flaskhalsen?

Snabb sammanfattning

- Arkitekturforum och styrning behövs även i agil utveckling, särskilt i myndighetsmiljö där ansvar, säkerhet, spårbarhet och helhet är viktiga.
- Friktion uppstår när forumet används som generell godkännandepunkt för frågor som egentligen behöver olika typer av hantering.
- Ett mer agilt forum skiljer mellan riktning, stöd, beslut och lärande.
- Beslutsflöde handlar om hur en fråga rör sig från upptäckt till tydligt beslut, kommunikation och uppföljning.
- Mandat nära kunskap innebär att beslut placeras där relevant kunskap och rätt mandat möts.
- Arkitekturforum bör fungera som stödstruktur: skapa riktning, undanröja hinder, synliggöra beroenden och hjälpa organisationen att fatta bättre beslut över tid.
- Arkitektens uppgift är inte bara att ta ärenden till forum, utan att hjälpa organisationen förstå vilka beslut som behövs, var de hör hemma och hur de ska följas upp.

Nästa steg

I det här kapitlet har vi sett hur arkitekturforum och styrning kan gå från flaskhals till stödstruktur. Det avslutar bokens tredje del om hållbart agilt arkitekturarbete i myndighetsmiljö.

Nästa kapitel samlar bokens viktigaste arbetssätt i en praktisk verktygslåda. Där återvänder vi till Sara och de återkommande situationerna för att formulera frågor, principer och förhållningssätt som arkitekten kan bära med sig i vardagen.

Kapitel 12: Den agila arkitektens praktiska verktygslåda

Varför detta kapitel finns

De tidigare kapitlen har rört sig genom situationer som många lösnings- och IT-arkitekter känner igen: osäkra behov, löpande kravarbete, team som vill börja bygga, design som sker under arbetets gång, beroenden som bromsar, dokumentation som både behövs och skaver, säkerhet som måste byggas in och forum som behöver stödja snarare än stoppa.

Det här avslutande kapitlet samlar bokens viktigaste arbetssätt i en praktisk verktygslåda. Syftet är inte att introducera ännu en modell som ska ersätta allt annat. Syftet är att ge arkitekten ett antal frågor, perspektiv och enkla arbetssätt som går att bära med sig in i vardagen.

För Sara på Myndigheten för samhällstjänster har resan inte handlat om att sluta vara arkitekt. Den har handlat om att använda sin erfarenhet på ett annat sätt. Hon har fortfarande ansvar för helhet, kvalitet, beroenden och långsiktighet. Men hon har behövt släppa föreställningen att trygghet alltid skapas genom mer förhandsanalys, mer detaljerade dokument och fler formella beslut innan utvecklingen startar.

Kapitlets centrala skifte är:

Från att arkitekten främst skapar arkitekturprodukter till att arkitekten utvecklar organisationens förmåga att fatta bra arkitektur- och designbeslut över tid.

Det betyder inte att arkitekturdokument, principer, beslutsunderlag eller forum blir oviktiga. Det betyder att de behöver förstås som delar av en levande praktik.

Situationen: när allt är samtidigt

Mot slutet av moderniseringsarbetet på Myndigheten för samhällstjänster sitter Sara i ett möte med Erik, Lina och Amir. Teamen har levererat flera inkrement av den nya medborgartjänsten. Det äldre handläggningssystemet är fortfarande kvar, men vissa delar har frikopplats. Dokumentationen är inte lika omfattande som i tidigare projekt, men den används oftare. Arkitekturforumet har börjat lägga mer tid på riktning och hinder än på sena godkännanden.

Samtidigt är arbetet långt ifrån friktionsfritt.

Ett team vill göra en teknisk förenkling som kan påverka framtida integrationer. Verksamheten vill prioritera en ny funktion som inte fanns med i den ursprungliga målbilden. Säkerhetsfunktionen vill se tydligare riskbeslut kring loggning och behörighet. Förvaltningen undrar hur lösningen ska kunna tas om hand långsiktigt. En extern part har ändrat sina tidsplaner, vilket påverkar ett beroende.

Det är en typisk agil arkitektursituation: flera perspektiv är sanna samtidigt.

Sara märker att hennes viktigaste bidrag inte är att ensam formulera den perfekta lösningen. Hennes viktigaste bidrag är att hjälpa gruppen att förstå vilken typ av fråga de står inför, vilket beslut som behövs nu, vilka risker som är verkliga, vilka antaganden som behöver testas och vilka ramar som behöver vara tydliga för att teamen ska kunna fortsätta.

Det är här den agila arkitektens verktygslåda behövs.

Det invanda sättet att agera

I ett mer fasorienterat arbetssätt har arkitekten ofta lärt sig att skapa trygghet genom att:

- analysera helheten tidigt,
- identifiera beroenden i förväg,
- dokumentera målbild och lösningsdesign,
- ta fram beslutsunderlag,
- förankra lösningen i forum,
- säkerställa att krav och lösning hänger ihop,
- minska risken för överraskningar senare.

Det är värdefulla förmågor. Boken har inte argumenterat för att de ska överges.

Men när utvecklingen blir mer iterativ kan samma förmågor skapa friktion om de används på ett sätt som förutsätter att lösningen först ska förstås fullt ut och därefter realiseras. Arkitekten kan då hamna i ett mönster där varje osäkerhet leder till mer utredning, varje risk leder till mer kontroll och varje otydlighet leder till att teamen behöver vänta.

Det invanda sättet kan sammanfattas så här:

“Jag skapar trygghet genom att göra lösningen tydligare innan andra går vidare.”

I en mer agil miljö behöver arkitekten ofta komplettera detta med ett annat förhållningssätt:

“Jag skapar trygghet genom att hjälpa organisationen lära, besluta och justera utan att tappa riktning.”

Skillnaden är liten i ord, men stor i vardagligt beteende.

Varför det kan skapa friktion i agil utveckling

Friktionen uppstår inte för att arkitekten bryr sig för mycket om kvalitet. Den uppstår när kvalitet bara kan hanteras genom tidig kontroll.

I agil utveckling kommer vissa behov, risker och lösningsdetaljer att bli tydliga först när arbetet pågår. Det gäller särskilt när organisationen arbetar med äldre system, många intressenter, externa beroenden, starka regelkrav och oklara verksamhetseffekter. En myndighet kan inte lösa all sådan osäkerhet genom att planera mer i början.

Det betyder inte att "allt får växa fram". Det betyder att arkitekten behöver skilja mellan olika typer av osäkerhet.

Vissa frågor behöver tydliga svar tidigt:

- Vilken information får hanteras i tjänsten?
- Vilka säkerhets- och behörighetsprinciper gäller?
- Vilka beroenden kan stoppa första leveransen?
- Vilka beslut är svåra att ändra senare?
- Vilka delar av det äldre systemet får inte påverkas utan särskild kontroll?

Andra frågor kan med fördel undersökas stegvis:

- Vilken användarresa ger störst nytta först?
- Hur detaljerad behöver integrationen vara i första inkrementet?
- Vilken teknisk lösning ger bäst balans mellan snabb leverans och långsiktig hållbarhet?
- Vilken dokumentationsnivå räcker för att nästa team ska förstå beslutet?
- Vilket forum behöver informeras, och vilket behöver faktiskt fatta beslut?

När allt behandlas som om det måste lösas före byggstart blir arkitekten lätt en broms. När allt behandlas som om det kan lösas senare skapas risk, omarbete och bristande ansvar. Den agila arkitektens praktik ligger i att skilja dessa frågor åt.

Ett mer agilt förhållningssätt

Ett mer agilt arkitektförhållningssätt kan sammanfattas i tre huvudbegrepp: arkitektonisk närvaro, reflekterande arkitektpraktik och situationsbaserat arkitektledarskap.

Arkitektonisk närvaro

Arkitektonisk närvaro betyder att arkitekten finns nära de sammanhang där viktiga beslut växer fram. Det handlar inte om att delta i alla möten eller svara på alla frågor. Det handlar om att vara tillräckligt nära för att upptäcka när ett lokalt beslut håller på att få större konsekvenser.

Arkitektonisk närvaro kan innebära att Sara:

- deltar i utvalda refinement-samtal där krav påverkar arkitektur,
- har korta återkommande avstämningar med teamet,
- följer beslutsloggen och reagerar på mönster,
- hjälper produktägaren att förstå arkitektoniska konsekvenser av prioriteringar,
- bjuder in säkerhet och förvaltning tidigt när risker börjar synas,
- formulerar principer som teamen kan använda utan att fråga varje gång.

Närvaro är inte samma sak som kontroll. En närvarande arkitekt behöver inte äga varje beslut. Men hon behöver förstå vilka beslut som är på väg att fattas, vilka

antaganden som styr dem och när ett samtal behöver breddas.

Reflekterande arkitektpraktik

Reflekterande arkitektpraktik betyder att arkitekten regelbundet undersöker sitt eget sätt att skapa värde.

För Sara innebär det att hon inte bara frågar "är lösningen bra?" utan också:

- Vilket beteende hos mig själv förstärker den här situationen?
- Skapar jag tydlighet eller väntan?
- Hjälper jag teamet att fatta bättre beslut, eller gör jag dem beroende av mig?
- Dokumenterar vi för faktisk användning eller för att känna oss trygga?
- Har jag gjort skillnad mellan risk, osäkerhet och obehag?
- Försöker jag lösa en fråga som egentligen behöver utforskas?

Det här är inte självkritik för sakens skull. Det är professionell kalibrering. En erfaren arkitekt har starka mönster och vältränade reflexer. I många situationer är de en tillgång. I andra situationer behöver de justeras.

Situationsbaserat arkitektledarskap

Situationsbaserat arkitektledarskap betyder att arkitekten anpassar sitt agerande efter situationens karaktär.

Ibland behöver arkitekten vara tydlig och avgränsande:

"Det här beslutet påverkar informationssäkerhet och förvaltning. Det behöver inte bli en lång process, men vi behöver fatta det med rätt personer i rummet."

Ibland behöver arkitekten vara utforskande:

"Vi vet inte ännu vilken integrationslösning som är bäst. Låt oss avgränsa ett inkrement som hjälper oss lära utan att låsa målbilden."

Ibland behöver arkitekten vara möjliggörande:

"Teamet kan fatta de här designbesluten själva om vi är överens om principerna och dokumenterar avvikelser."

Ibland behöver arkitekten vara bromsande:

"Här riskerar vi att skapa ett beroende som blir dyrt att ändra. Innan vi går vidare behöver vi förstå konsekvensen."

Poängen är inte att alltid vara snabbare, lättare eller mer tillåtande. Poängen är att agera medvetet utifrån vilken sorts fråga organisationen faktiskt står inför.

Exempel: Myndigheten för samhällstjänster

Sara börjar använda en enkel reflektionsmodell i sitt arbete. Hon kallar den för "frågekortet", men det är egentligen bara fem återkommande frågor som hon använder i möten, forum och samtal med team.

Fråga 1: Vilken typ av fråga är detta?

När en fråga dyker upp försöker Sara först förstå om den främst handlar om:

- verksamhetsnytta,
- användarbeteende,
- juridik eller regelefterlevnad,
- informationssäkerhet,
- integration,
- data och informationsmodell,
- teknisk design,
- förvaltning,
- beroenden,
- mandat och beslut.

Det gör att gruppen snabbare ser vilka perspektiv som behöver vara med.

När Lina säger att teamet vill förenkla integrationen mot handläggningssystemet frågar Sara inte först efter en detaljerad lösningsskiss. Hon frågar:

“Är det här främst en teknisk förenkling, eller påverkar det vilken information handläggarna kommer kunna lita på?”

Den frågan förändrar samtalet. Det visar sig att förenklingen både påverkar teknisk komplexitet och informationskvalitet. Då behöver Erik och Amir också vara med i diskussionen.

Fråga 2: Vad behöver beslutas nu?

I stället för att försöka lösa hela frågan hjälper Sara gruppen att skilja mellan beslut som behövs nu och beslut som kan vänta.

För integrationen behöver teamet besluta hur första inkrementet ska läsa data. Men det behöver ännu inte besluta hela den framtida integrationsstrategin. Däremot behöver de undvika ett vägval som gör framtida frikoppling orimligt dyr.

Sara formulerar det så här:

“Vi behöver inte välja slutlig målbild i dag, men vi behöver välja ett första steg som inte gör målbilden omöjlig.”

Det är ett typiskt uttryck för tillräcklig arkitektur.

Fråga 3: Vad är svårt att ändra senare?

Gruppen går igenom vad som är reversibelt och svårreversibelt.

Det visar sig att vissa delar av teamets förslag är enkla att ändra senare, medan valet av datamodell och behörighetsprinciper kan bli svårare. Sara föreslår därför att teamet får stor frihet i implementationen, men att datamodell, behörighetsprincip och spårbarhet dokumenteras som arkitekturbeslut.

Det gör att teamet kan fortsätta utan att allt behöver lyftas till arkitekturforumet.

Fråga 4: Vilket lärande behöver vi skapa?

Sara frågar vad första inkrementet faktiskt ska lära organisationen.

Är syftet att testa användarvärde? Att minska teknisk risk? Att pröva ett nytt arbetssätt mellan team och säkerhet? Att se om äldre system klarar belastningen? Att förstå handläggarnas informationsbehov?

När gruppen svarar blir det tydligare vad som ska följas upp. Det första inkrementet behöver inte bara leverera funktion. Det behöver också skapa kunskap inför nästa beslut.

Fråga 5: Hur gör vi beslutet synligt?

Till sist hjälper Sara gruppen att dokumentera beslutet kort:

- Vad har vi beslutat?
- Varför?
- Vilka antaganden bygger beslutet på?
- Vilka risker accepterar vi?
- När behöver vi ompröva beslutet?
- Vem behöver känna till det?

Beslutet blir inte ett stort dokument. Det blir en användbar beslutslogg som både teamet, förvaltningen, säkerhetsfunktionen och arkitekturforumet kan förstå.

Verktygslåda: fem praktiska arbetssätt

1. Beslutsradarn

Beslutsradarn är ett enkelt sätt att upptäcka när en vardagsfråga egentligen är ett arkitekturbeslut.

Fråga:

- Påverkar detta fler än ett team?
- Påverkar detta data, säkerhet, integration eller förvaltning?
- Blir detta svårt att ändra senare?
- Skapar detta ett beroende?
- Behöver andra kunna förstå beslutet om sex månader?

Om svaret är ja på någon av frågorna är det troligen inte bara en lokal detalj. Då behöver beslutet synliggöras, även om det fortfarande kan fattas nära teamet.

2. Startvillkor utan fasgrind

När ett team vill börja bygga kan arkitekten använda startvillkor som stöd i stället för som godkännandegrind.

Exempel på startvillkor:

- Vi vet vilken användar- eller verksamhetsnytta första inkrementet ska pröva.
- Vi vet vilka säkerhets- och informationsfrågor som inte får lämnas öppna.
- Vi vet vilka beroenden som kan stoppa arbetet.
- Vi vet vilka arkitekturbeslut som behöver dokumenteras.
- Vi vet vad första inkrementet ska hjälpa oss att lära.

Startvillkor ska vara lätta nog att använda, men tydliga nog att skydda arbetet från onödig risk.

3. Principer före detaljstyrning

När flera team behöver fatta många designbeslut fungerar detaljgodkännande dåligt. Då behöver arkitekten formulera principer.

En bra princip är:

- begriplig,
- användbar i vardagen,
- möjlig att följa upp,
- tydlig med varför den finns,
- inte mer detaljerad än nödvändigt.

Exempel:

“Personuppgifter ska inte kopieras mellan system utan tydligt syfte, ansvar och rensningsregel.”

En sådan princip hjälper teamen att fatta beslut utan att Sara behöver godkänna varje lösningsdetalj.

4. Dokumentation med mottagare

Innan dokumentation tas fram bör arkitekten fråga:

- Vem ska använda detta?
- Vilket beslut, vilken samordning eller vilket ansvar ska dokumentationen stödja?
- Hur ofta behöver den uppdateras?
- Vad händer om den inte finns?
- Vad händer om den finns men inte används?

Det flyttar dokumentation från leveransprodukt till arbetsredskap.

5. Forumfrågan

Innan något lyfts till arkitekturforum kan arkitekten fråga:

- Behöver forumet fatta beslut?
- Behöver forumet ge riktning?
- Behöver forumet undanröja hinder?
- Behöver forumet bara informeras?
- Kan frågan avgöras närmare teamet inom befintliga principer?

Det hjälper forumet att bli en stödstruktur snarare än en kö.

Vanliga fallgropar

- Fallgrop: Att göra verktygslådan till en ny process.
 - Varför den uppstår: Organisationer som vill skapa ordning kan snabbt omvandla frågor och arbetssätt till obligatoriska steg.
 - Vad arkitekten kan göra i stället: Behandla verktygen som stöd för professionellt omdöme, inte som en ny fasmodell.
- Fallgrop: Att tolka agil arkitektur som mindre arkitektur.

- Varför den uppstår: När fokus ligger på snabbare leveranser kan arkitektur misstas för något tungt eller försenande.
- Vad arkitekten kan göra i stället: Visa hur arkitektur kan minska väntan, beroenden och omarbete genom tydliga principer och beslut i rätt tid.
- Fallgrop: Att tro att allt ska beslutas nära teamet.
 - Varför den uppstår: Teamautonomi kan ibland tolkas som att varje team ska kunna fatta alla beslut själv.
 - Vad arkitekten kan göra i stället: Skilj mellan lokala beslut, delade beslut och beslut som påverkar myndighetens helhet.
- Fallgrop: Att fortsätta vara den tysta kvalitetssäkraren.
 - Varför den uppstår: Erfarna arkitekter ser ofta risker tidigt, men väntar med att lyfta dem tills det finns ett tydligt underlag.
 - Vad arkitekten kan göra i stället: Dela observationer tidigt som frågor, inte som färdiga invändningar.
- Fallgrop: Att söka trygghet i dokumentation som ingen använder.
 - Varför den uppstår: Dokumentation kan kännas som kontroll även när den inte påverkar beslut eller beteenden.
 - Vad arkitekten kan göra i stället: Börja med mottagare, användning och livscykel innan dokumentet skapas.

Frågor att ställa i situationen

När du som arkitekt hamnar i en oklar situation kan följande frågor användas som en praktisk checklista:

1. Vilken typ av fråga står vi inför?
2. Vilka perspektiv saknas i samtalet?
3. Vad behöver beslutas nu?
4. Vad kan vänta?
5. Vad behöver undersökas genom ett inkrement?
6. Vad är svårt att ändra senare?
7. Vilka antaganden styr vårt resonemang?
8. Vilka beroenden skapar eller minskar vi?
9. Vilken dokumentation behövs för att beslutet ska kunna förstås senare?
10. Vem behöver mandat, information eller stöd för att nästa steg ska fungera?

Frågorna behöver inte användas varje gång. Värde ligger i att de hjälper arkitekten att sakta ner precis tillräckligt för att förstå situationen, utan att göra arbetet tungt.

Reflektionsfrågor

1. Vilka av bokens situationer känner du starkast igen från din egen vardag?
2. När tenderar du att skapa trygghet genom mer förhandsarbete, och när fungerar det väl?
3. Vilka beslut i din organisation fattas för långt från den kunskap som behövs?
4. Vilka beslut fattas för lokalt och borde synliggöras mer?
5. Vilka arkitekturprinciper skulle hjälpa teamen att fatta bättre beslut utan att fråga dig varje gång?
6. Vilken dokumentation i din miljö används faktiskt, och vilken finns mest för att den förväntas finnas?

7. Hur kan du öka din arkitektoniska närvaro utan att bli en flaskhals?
8. Vilket litet beteende kan du pröva i nästa möte för att bidra mer agilt?

Snabb sammanfattning

- Den agila arkitekten slutar inte ta ansvar för helhet, kvalitet och långsiktighet.
- Ansvarstagandet sker mer genom närvaro, frågor, principer, beslut i rätt tid och stöd till teamens beslutsförmåga.
- Arkitektonisk närvaro handlar om att vara nära viktiga beslut utan att äga alla beslut.
- Reflekterande arkitektpraktik handlar om att undersöka hur det egna beteendet påverkar flöde, lärande och kvalitet.
- Situationsbaserat arkitektledarskap handlar om att kunna vara tydlig, utforskande, möjliggörande eller bromsande beroende på vad situationen kräver.
- Verktygslådan består av praktiska stöd: beslutsradar, startvillkor utan fasgrind, principer före detaljstyrning, dokumentation med mottagare och forumfrågan.
- Bokens kärna är att arkitekten utvecklar organisationens förmåga att fatta bra arkitektur- och designbeslut över tid.

Nästa steg

Boken är nu komplett som första kapitelutkast. Nästa steg är inte att lägga till fler kapitel, utan att granska helheten.

En lämplig fortsättning är att göra en första sammanhängande genomläsning med tre frågor:

1. Håller boken en konsekvent ton: kollegial, reflekterande och myndighetsnära?
2. Introduceras begreppen i rätt ordning och används de konsekvent?
3. Bildar Saras resa genom Myndigheten för samhällstjänster en tydlig progression från fasorienterad trygghet till agil arkitektpraktik?

Efter en sådan granskning kan boken bearbetas kapitel för kapitel, kompletteras med checklistor eller förberedas för export till exempelvis DOCX, PDF eller EPUB.